

深度学习

从数学基础到大模型

基于 RethinkFun 教程的扩展笔记

魏舟

2026 年 7 月 21 日

目录

第 1 章 线性代数基础	1
1.1 标量、向量与矩阵	1
1.1.1 标量	1
1.1.2 向量	1
1.1.3 矩阵	2
1.1.4 张量	3
1.2 矩阵乘法	3
1.2.1 矩阵-向量乘法	3
1.2.2 矩阵-矩阵乘法	4
1.2.3 全连接层中的矩阵乘法	4
1.3 范数	5
1.3.1 向量范数	5
1.3.2 矩阵范数	6
1.4 特殊矩阵	6
1.4.1 正交矩阵	6
1.4.2 正定矩阵	6
1.5 特征分解	7
1.5.1 特征值与特征向量	7
1.5.2 特征分解	7
1.5.3 特征值与深度学习	8
1.6 奇异值分解	9
1.6.1 低秩近似	9
1.7 Moore-Penrose 伪逆	11
1.8 行列式	11
1.9 迹	11
1.10 PCA 与 SVD	12
1.11 向量与矩阵微积分	14
1.12 本章小结	14
第 2 章 微积分基础	17
2.1 极限与连续性	17
2.1.1 极限的定义	17
2.1.2 连续性	17
2.2 导数	17
2.2.1 导数的定义	17
2.2.2 基本导数公式	18
2.2.3 导数的运算法则	18
2.2.4 常见激活函数的导数	18

2.3	偏导数与梯度	19
2.3.1	偏导数	19
2.3.2	梯度	20
2.3.3	Jacobian 矩阵	20
2.4	链式法则	21
2.4.1	一元函数的链式法则	21
2.4.2	多元函数的链式法则	22
2.4.3	反向传播与链式法则	22
2.5	Hessian 矩阵	24
2.6	Taylor 展开	24
2.6.1	一元函数的 Taylor 展开	24
2.6.2	多元函数的 Taylor 展开	24
2.7	凸优化基础	26
2.7.1	凸集与凸函数	26
2.7.2	凸函数的判定条件	26
2.7.3	凸优化的性质	27
2.8	约束优化与 Lagrange 乘子法	28
2.9	本章小结	28
第 3 章	概率与统计基础	29
3.1	随机变量与概率分布	29
3.1.1	离散随机变量	29
3.1.2	连续随机变量	30
3.2	条件概率与贝叶斯定理	31
3.2.1	条件概率	31
3.2.2	全概率公式与贝叶斯定理	32
3.2.3	独立性与条件独立	32
3.3	期望、方差与协方差	33
3.3.1	期望	33
3.3.2	方差与标准差	33
3.3.3	协方差与相关系数	34
3.4	信息论基础	35
3.4.1	信息量与熵	35
3.4.2	KL 散度与交叉熵	36
3.4.3	互信息	36
3.5	最大似然估计	38
3.5.1	似然函数	38
3.5.2	最大似然估计	38
3.5.3	MLE 与损失函数	38
3.6	最大后验估计	39
3.7	随机过程初步	40
3.7.1	马尔可夫链	40
3.8	本章小结	40
第 4 章	线性回归与梯度下降	41
4.1	线性回归模型	41
4.1.1	模型定义	41
4.1.2	损失函数：均方误差	41

4.2	正规方程	42
4.3	梯度下降	43
4.3.1	批量梯度下降	43
4.3.2	随机梯度下降	44
4.3.3	小批量随机梯度下降	44
4.4	动量与自适应优化器	45
4.4.1	带动量的 SGD	45
4.4.2	Nesterov 动量	46
4.4.3	AdaGrad	46
4.4.4	RMSProp	46
4.4.5	Adam	46
4.5	学习率调度	49
4.6	正则化	49
4.6.1	L^2 正则化 (Ridge 回归/权重衰减)	49
4.6.2	L^1 正则化 (Lasso 回归)	49
4.6.3	弹性网络	50
4.7	多项式回归	51
4.8	偏差-方差权衡	51
4.9	本章小结	52
第 5 章	逻辑回归与分类	53
5.1	逻辑回归模型	53
5.1.1	从线性回归到逻辑回归	53
5.1.2	决策边界	54
5.2	交叉熵损失	54
5.2.1	概率解释	54
5.2.2	最大似然推导	55
5.3	多分类: Softmax 回归	57
5.3.1	模型定义	57
5.3.2	交叉熵损失	57
5.4	分类评估指标	59
5.4.1	混淆矩阵	59
5.4.2	准确率、精确率、召回率与 F1 分数	59
5.4.3	ROC 曲线与 AUC	60
5.5	多分类的评估指标	60
5.6	类别不平衡问题	60
5.7	逻辑回归与深度学习	61
5.8	本章小结	62
第 6 章	PyTorch 入门	63
6.1	张量基础	63
6.1.1	创建张量	63
6.1.2	张量属性与运算	64
6.1.3	索引与切片	64
6.2	GPU 加速	65
6.3	自动求导	66
6.3.1	计算图	66
6.3.2	梯度在神经网络中的应用	68

6.4	nn.Module: 构建神经网络	69
6.4.1	常用层	71
6.4.2	Sequential 容器	71
6.5	损失函数与优化器	72
6.6	数据加载	74
6.6.1	内置数据集	75
6.7	训练循环	76
6.8	模型保存与加载	78
6.9	完整实例: MNIST 手写数字识别	80
6.10	本章小结	81
第 7 章	神经网络——深度学习的核心	83
7.1	从感知机到多层神经网络	83
7.1.1	神经元——最基本的计算单元	83
7.1.2	单层网络与异或问题	84
7.1.3	多层网络与表示学习	84
7.2	激活函数——神经网络的灵魂	85
7.2.1	Sigmoid 与 Tanh	85
7.2.2	ReLU 及其变体	86
7.2.3	Softmax——多分类的输出层	86
7.3	前向传播——信息的流动	87
7.3.1	前向传播的矩阵形式	87
7.3.2	向量化——批量前向传播	88
7.4	损失函数——定义优化目标	88
7.4.1	均方误差 (MSE)	88
7.4.2	交叉熵损失	88
7.5	反向传播——学习的引擎	89
7.5.1	链式法则	89
7.5.2	反向传播的逐步推导	90
7.5.3	反向传播的计算复杂度	90
7.5.4	局部误差的直观解释	91
7.6	梯度消失与梯度爆炸	91
7.6.1	问题成因	91
7.6.2	解决方案	91
7.7	正则化——防止过拟合	91
7.7.1	L^1 与 L^2 正则化	91
7.7.2	Dropout	92
7.7.3	Batch Normalization 的正则化效应	92
7.8	用 NumPy 从零实现神经网络	93
7.8.1	代码核心要点说明	95
7.9	用 PyTorch 实现多分类神经网络	96
7.9.1	模块化构建	96
7.9.2	超参数的影响	97
7.10	参数初始化策略	97
7.11	本章小结	97

第 8 章 优化深度神经网络	99
8.1 优化问题的数学描述	99
8.1.1 经验风险最小化	99
8.1.2 非凸优化的挑战——鞍点而非局部极小值	99
8.2 梯度下降法及其变体	100
8.2.1 批量梯度下降	100
8.2.2 随机梯度下降	100
8.2.3 小批量随机梯度下降	100
8.3 动量法——让梯度具有“惯性”	101
8.3.1 经典动量	101
8.3.2 Nesterov 加速梯度	102
8.4 自适应学习率方法	102
8.4.1 AdaGrad	102
8.4.2 RMSProp	103
8.4.3 Adam——当前的事实标准	103
8.4.4 AdamW——解耦权重衰减	103
8.5 学习率调度策略	105
8.5.1 常用调度策略	105
8.6 梯度裁剪	107
8.7 二阶优化方法简介	108
8.7.1 牛顿法	109
8.7.2 L-BFGS——实用的拟牛顿法	109
8.8 超参数调优	109
8.8.1 网格搜索 vs 随机搜索 vs 贝叶斯优化	109
8.8.2 实用超参数调优建议	109
8.9 训练稳定性与调试技巧	110
8.9.1 梯度与激活值监控	110
8.9.2 常见训练问题诊断表	112
8.10 本章小结	112
第 9 章 卷积神经网络——让机器看懂世界	113
9.1 视觉问题的本质	113
9.1.1 全连接网络的失效	113
9.1.2 图像的三个归纳偏置	113
9.2 卷积运算的数学定义	114
9.2.1 一维卷积	114
9.2.2 二维卷积——图像处理的核心	114
9.2.3 多通道卷积	115
9.2.4 步长与填充	115
9.3 用 NumPy 从零实现二维卷积	116
9.3.1 卷积的矩阵乘法实现——im2col	118
9.4 池化层——降采样与平移不变性	120
9.4.1 最大池化与平均池化	120
9.5 PyTorch CNN 完整训练管线	122
9.5.1 感受野的堆叠增长	124
9.6 转置卷积——从低分辨率到高分辨率	124
9.6.1 为什么需要上采样	124

9.6.2	转置卷积的数学原理	124
9.7	特征可视化——理解 CNN 看到了什么	125
9.7.1	卷积核可视化	125
9.7.2	特征图可视化	127
9.7.3	Grad-CAM——类激活映射	129
9.8	CNN 设计中的核心原则	131
9.8.1	感受野的指数增长	131
9.8.2	通道数递增，空间分辨率递减	131
9.8.3	1×1 卷积——通道间的信息交换	132
9.8.4	空洞卷积——扩大感受野而不增加参数	132
9.9	实践建议与常见陷阱	132
9.9.1	计算卷积参数的公式	133
9.10	本章小结	133
第 10 章	经典 CNN 架构——从 LeNet 到 ResNet	135
10.1	LeNet-5——CNN 的起源（1998 年）	135
10.2	AlexNet——深度学习革命的起点（2012 年）	136
10.3	VGG——深度的重要性（2014 年）	137
10.4	GoogLeNet / Inception——多尺度特征融合（2014 年）	138
10.4.1	Inception 模块	138
10.4.2	辅助分类器	138
10.5	ResNet——残差网络，突破深度极限（2015 年）	140
10.5.1	退化问题	140
10.5.2	残差学习——Let the layers fit a residual mapping	140
10.5.3	残差连接解决梯度消失	140
10.5.4	Bottleneck 残差块	140
10.5.5	ResNet 架构配置表	142
10.6	DenseNet——连接一切（2017 年）	142
10.7	轻量化架构——MobileNet 与 EfficientNet	142
10.7.1	深度可分离卷积（Depthwise Separable Convolution）	142
10.7.2	EfficientNet——复合缩放	143
10.8	架构演进路线图与设计趋势	144
10.8.1	设计趋势总结	144
10.9	本章小结	144
第 11 章	循环神经网络与注意力机制	147
11.1	序列建模的核心挑战	147
11.1.1	为什么需要特殊的网络结构	147
11.1.2	RNN 的基本思想	147
11.2	循环神经网络——前向传播与反向传播	148
11.2.1	前向传播	148
11.2.2	时间反向传播（BPTT）	148
11.2.3	RNN 的变体	148
11.3	LSTM——长短期记忆网络	149
11.3.1	LSTM 的核心思想——细胞状态	149
11.3.2	四个门控的精确数学公式	149
11.3.3	梯度流动——为什么 LSTM 有效	149
11.4	GRU——门控循环单元	150

11.5	序列到序列模型 (Seq2Seq)	152
11.6	注意力机制——从瓶颈到动态加权	152
11.6.1	Bahdanau 注意力 (2015 年)	152
11.6.2	Luong 注意力——三种评分方式	153
11.6.3	缩放点积注意力——通往 Transformer 的桥梁	153
11.7	自注意力——序列内部的注意力	155
11.8	本章小结	155
第 12 章	Transformer 与大规模语言模型	157
12.1	Transformer——Attention Is All You Need	157
12.1.1	为什么需要 Transformer	157
12.1.2	整体架构	157
12.1.3	多头自注意力——Transformer 的心脏	158
12.1.4	位置编码——赋予序列顺序信息	158
12.1.5	前馈网络	159
12.1.6	Add & Norm——残差连接与层归一化	159
12.1.7	掩码自注意力——解码器中的因果性	159
12.2	BERT——双向编码器表示	161
12.2.1	预训练任务	161
12.2.2	BERT 微调	161
12.3	GPT——生成式预训练 Transformer	161
12.3.1	GPT 的演进	161
12.3.2	自回归生成与 KV-Cache	162
12.3.3	缩放定律	164
12.4	Llama——开源大模型的前沿	164
12.4.1	Llama 系列的演进	164
12.4.2	RMSNorm——简化的层归一化	164
12.4.3	RoPE——旋转位置编码	165
12.4.4	分组查询注意力	165
12.5	DeepSeek——中国大模型的崛起	165
12.5.1	DeepSeek-V2 与 Multi-head Latent Attention	165
12.5.2	DeepSeek-V3 与混合专家模型	165
12.5.3	DeepSeek-R1——推理增强	165
12.6	Transformer 的变体与优化	166
12.6.1	FlashAttention——IO 感知的注意力优化	166
12.6.2	Mamba 与状态空间模型	166
12.7	指令微调与 RLHF	166
12.7.1	指令微调	166
12.7.2	RLHF——基于人类反馈的强化学习	166
12.8	本章小结	167

线性代数基础

线性代数是深度学习的数学基石。神经网络的前向传播本质上是矩阵乘法与非线性激活的交替，反向传播则依赖于梯度在计算图中的传播。本章系统介绍深度学习所需的线性代数知识，包括标量、向量、矩阵、张量的基本概念，矩阵运算，范数，特征分解与奇异值分解等核心内容。

§1.1

标量、向量与矩阵

1.1.1 标量

标量（Scalar）是一个单独的数值，通常用小写斜体字母表示，如 a 、 n 、 x 。在深度学习中，标量常用于表示损失函数值、学习率、准确率等单一数值。

1.1.2 向量

向量（Vector）是一组有序的数值，可视为一维数组。本书约定向量用粗体小写字母表示，如 $\mathbf{x}$ 。一个 n 维向量可以表示为：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

向量的第 i 个元素记为 x_i 。在深度学习中，向量常用于表示单个样本的特征、权重参数等。例如，一张 28×28 的灰度图像可以被展平为一个 784 维向量。

Python 中的向量操作

```

1 import numpy as np
2
3 # 创建向量
4 x = np.array([1.0, 2.0, 3.0, 4.0])
5 print(f"向量: {x}")
6 print(f"维度: {x.shape}")
7 print(f"第2个元素: {x[1]}")
8
9 # 向量基本运算
10 y = np.array([5.0, 6.0, 7.0, 8.0])
11 print(f"加法: {x + y}")
12 print(f"标量乘法: {2.0 * x}")
13 print(f"点积: {np.dot(x, y)}")
14 print(f"逐元素乘积: {x * y}")

```

1.1.3 矩阵

矩阵 (Matrix) 是一个二维数组, 用粗体大写字母表示, 如 $\mathbf{A}$ 。一个 m 行 n 列的矩阵记为:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

矩阵的重要属性:

- **转置 (Transpose):** 将矩阵的行和列互换, 记为 $\mathbf{A}^\top$ 。若 $\mathbf{A} \in \mathbb{R}^{m \times n}$, 则 $\mathbf{A}^\top \in \mathbb{R}^{n \times m}$, 且 $(\mathbf{A}^\top)_{ij} = A_{ji}$ 。
- **对称矩阵:** 若 $\mathbf{A} = \mathbf{A}^\top$, 则 $\mathbf{A}$ 为对称矩阵。
- **对角矩阵:** 只有主对角线上的元素可能非零, 其余元素均为零。
- **单位矩阵:** 主对角线全为 1 的对角矩阵, 记为 $\mathbf{I}_n$ 。
- **逆矩阵:** 若存在矩阵 $\mathbf{A}^{-1}$ 使得 $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, 则称 $\mathbf{A}$ 可逆。

Python 中的矩阵操作

```

1 import numpy as np
2
3 # 创建矩阵
4 A = np.array([[1, 2, 3],
5               [4, 5, 6],
6               [7, 8, 9]])
7 print(f"矩阵 A:\n{A}")
8 print(f"A 的形状: {A.shape}")
9 print(f"A 的转置:\n{A.T}")
10
11 # 单位矩阵
12 I = np.eye(3)
13 print(f"单位矩阵:\n{I}")
14
15 # 矩阵乘法
16 B = np.array([[2, 0, 1],
17               [0, 3, 2],
18               [1, 1, 0]])
19 C = A @ B # 或 np.dot(A, B)
20 print(f"A @ B:\n{C}")
21
22 # 逐元素乘法 (Hadamard 积)
23 D = A * B
24 print(f"A * B (Hadamard 积):\n{D}")

```

1.1.4 张量

张量 (Tensor) 是标量、向量和矩阵向更高维度的推广。标量是零阶张量，向量是一阶张量，矩阵是二阶张量。在深度学习中，常见的张量包括：

- 一阶张量 (向量): $(n,)$
- 二阶张量 (矩阵): (m, n) ，如图像的灰度表示
- 三阶张量: (H, W, C) ，如彩色图像 (高度 $\times$ 宽度 $\times$ 通道)
- 四阶张量: (B, H, W, C) ，如一批彩色图像
- 五阶张量: (B, T, H, W, C) ，如视频数据 (批大小 $\times$ 时间 $\times$ 高度 $\times$ 宽度 $\times$ 通道)

§ 1.2
矩阵乘法

1.2.1 矩阵-向量乘法

设 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ， $\mathbf{x} \in \mathbb{R}^n$ ，则矩阵-向量乘积 $\mathbf{y} = \mathbf{Ax} \in \mathbb{R}^m$ 定义为：

$$y_i = \sum_{j=1}^n A_{ij}x_j, \quad i = 1, \dots, m$$

从线性变换的角度看， $\mathbf{Ax}$ 表示将向量 $\mathbf{x}$ 从 $\mathbb{R}^n$ 空间变换到 $\mathbb{R}^m$ 空间。

从列向量的角度，若 $\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n]$ ，则：

$$\mathbf{Ax} = \sum_{j=1}^n x_j \mathbf{a}_j$$

即 $\mathbf{Ax}$ 是 $\mathbf{A}$ 的列向量的线性组合，系数为 $\mathbf{x}$ 的分量。

1.2.2 矩阵-矩阵乘法

设 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ， $\mathbf{B} \in \mathbb{R}^{n \times p}$ ，则矩阵乘积 $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$ 定义为：

$$C_{ij} = \sum_{k=1}^n A_{ik}B_{kj}$$

矩阵乘法的计算复杂度为 $O(mnp)$ 。在深度学习中，当 m, n, p 都很大时，矩阵乘法是主要的计算瓶颈，这也是 GPU 被广泛使用的原因——GPU 拥有大量并行计算核心，可以高效地并行执行矩阵乘法。

矩阵乘法的性质：

- 结合律： $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- 分配律： $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$
- 转置： $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$
- 不满足交换律：一般情况下 $\mathbf{AB} \neq \mathbf{BA}$

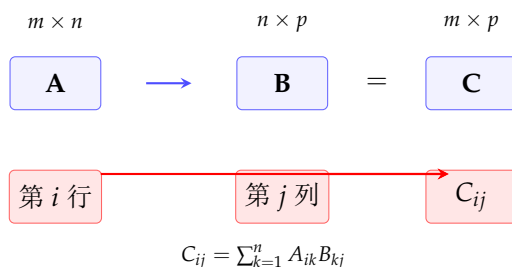


图 1.1: 矩阵乘法示意图： $\mathbf{C} = \mathbf{AB}$ 中 C_{ij} 由 $\mathbf{A}$ 的第 i 行与 $\mathbf{B}$ 的第 j 列的内积得到。

1.2.3 全连接层中的矩阵乘法

深度学习中，全连接层（也称为线性层或稠密层）的核心操作就是矩阵乘法。设输入向量为 $\mathbf{x} \in \mathbb{R}^n$ ，权重矩阵为 $\mathbf{W} \in \mathbb{R}^{m \times n}$ ，偏置为 $\mathbf{b} \in \mathbb{R}^m$ ，则输出为：

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

在实际实现中，通常一次处理一批样本。设批大小为 B ，输入矩阵为 $\mathbf{X} \in \mathbb{R}^{B \times n}$ （每行一个样本），则输出为：

$$\mathbf{Y} = \mathbf{XW}^\top + \mathbf{b}$$

其中 $\mathbf{Y} \in \mathbb{R}^{B \times m}$ 。注意这里需要将 $\mathbf{W}$ 转置以满足维度匹配。

§ 1.3
范数

范数（Norm）是衡量向量或矩阵“大小”的函数，满足非负性、齐次性和三角不等式。

1.3.1 向量范数

L^p 范数定义为：

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad p \geq 1$$

常用的向量范数：

 L^p 向量范数

- L^0 “范数”（不是真正的范数）：非零元素的个数

$$\|\mathbf{x}\|_0 = |\{i : x_i \neq 0\}|$$

- L^1 范数（曼哈顿距离）： $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$
- L^2 范数（欧几里得距离）： $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- L^∞ 范数（最大范数）： $\|\mathbf{x}\|_\infty = \max_i |x_i|$

在深度学习中，范数有两个重要作用：

1. **正则化**：在损失函数中加入 L^1 或 L^2 范数惩罚项，防止过拟合。
2. **梯度裁剪**：当梯度的 L^2 范数超过阈值时，对其进行缩放，防止梯度爆炸。

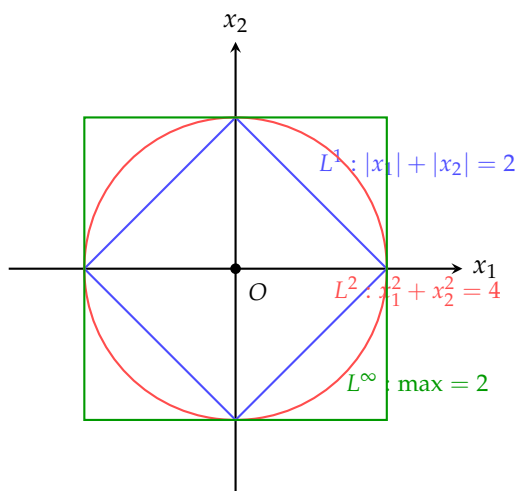


图 1.2: 二维空间中 L^1 、 L^2 和 L^∞ 范数的单位等高线。 L^1 范数的等高线呈菱形，易于产生稀疏解； L^2 范数的等高线为圆形。

1.3.2 矩阵范数

Frobenius 范数

矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 的 Frobenius 范数定义为所有元素平方和的平方根：

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2}$$

Frobenius 范数可以视为将矩阵展平为向量后求 L^2 范数。它在深度学习中常用于衡量权重矩阵的大小，作为权重衰减（weight decay）的正则化项。

Python 中的范数计算

```
1 import numpy as np
2
3 x = np.array([3.0, 4.0])
4 A = np.array([[1, 2], [3, 4]])
5
6 # 向量范数
7 print(f"L1 范数: {np.linalg.norm(x, ord=1)}")
8 print(f"L2 范数: {np.linalg.norm(x, ord=2)}")
9 print(f"Linf 范数: {np.linalg.norm(x, ord=np.inf)}")
10
11 # 矩阵范数
12 print(f"Frobenius 范数: {np.linalg.norm(A, ord='fro')}")
13 print(f"Frobenius (手动): {np.sqrt(np.sum(A**2))}")
```

§ 1.4 特殊矩阵

1.4.1 正交矩阵

若方阵 $\mathbf{Q} \in \mathbb{R}^{n \times n}$ 满足 $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ （即 $\mathbf{Q}^{-1} = \mathbf{Q}^\top$ ），则称其为正交矩阵。正交矩阵的列向量构成一组标准正交基。

正交矩阵的一个重要性质是保范性：

$$\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$$

这意味着用一个正交矩阵乘以向量不会改变向量的长度，只是旋转或反射。

1.4.2 正定矩阵

若对称矩阵 $\mathbf{A} \in \mathbb{R}^{n \times n}$ 满足对所有非零 $\mathbf{x} \in \mathbb{R}^n$ 都有 $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ ，则称 $\mathbf{A}$ 为正定矩阵。若 $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ ，则称为半正定矩阵。

正定矩阵在深度学习中有重要应用：

- **Hessian 矩阵**：若函数在某点的 Hessian 矩阵是正定的，则该点是局部极小点。
- **协方差矩阵**：数据集协方差矩阵总是半正定的。

- 牛顿法：利用正定的 Hessian 近似来寻找最优方向。

§ 1.5 特征分解

1.5.1 特征值与特征向量

特征值与特征向量

设 $\mathbf{A} \in \mathbb{R}^{n \times n}$ 为方阵。若非零向量 $\mathbf{v} \in \mathbb{R}^n$ 满足：

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

则称 λ 为矩阵 $\mathbf{A}$ 的特征值， $\mathbf{v}$ 为对应的特征向量。

特征方程 $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ 给出了 n 次特征多项式，其 n 个根（可能重复或为复数）即为特征值。

【例题 1.1】 求矩阵 $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ 的特征值和特征向量。

解：特征方程： $\det(\mathbf{A} - \lambda\mathbf{I}) = \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = (2-\lambda)^2 - 1 = 0$

解得 $\lambda^2 - 4\lambda + 3 = 0$ ，即 $(\lambda - 1)(\lambda - 3) = 0$ ，故 $\lambda_1 = 1$ ， $\lambda_2 = 3$ 。

当 $\lambda_1 = 1$ 时： $(2-1)v_1 + v_2 = 0 \Rightarrow v_1 = -v_2$ ，取 $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ 。

当 $\lambda_2 = 3$ 时： $(2-3)v_1 + v_2 = 0 \Rightarrow v_2 = v_1$ ，取 $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 。

1.5.2 特征分解

若方阵 $\mathbf{A} \in \mathbb{R}^{n \times n}$ 有 n 个线性无关的特征向量 $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ ，对应特征值 $\{\lambda_1, \dots, \lambda_n\}$ ，则 $\mathbf{A}$ 可以被分解为：

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

其中 $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n]$ ， $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ 。

特别地，若 $\mathbf{A}$ 为实对称矩阵，则存在正交矩阵 $\mathbf{Q}$ 使得：

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$$

因为这组特征向量可以构成标准正交基，这是谱定理的特殊形式。

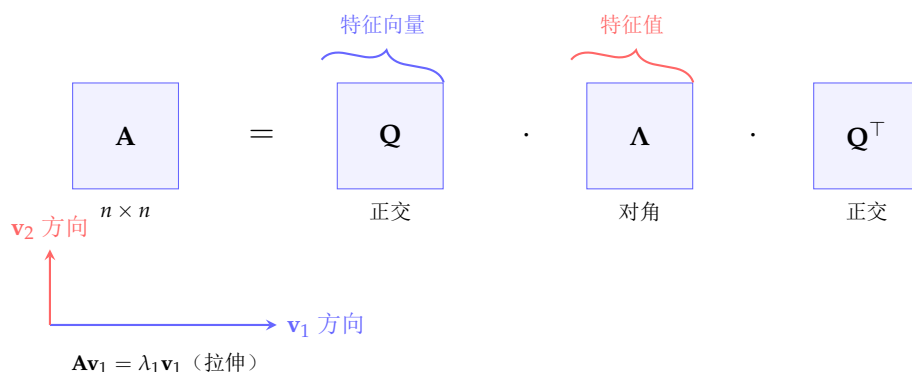


图 1.3: 实对称矩阵的特征分解。 $A = Q\Lambda Q^T$ 表示将 A 的作用分解为：旋转 (Q^T)、缩放 (Λ)、旋转回去 (Q)。

Python 中的特征分解

```

1 import numpy as np
2
3 A = np.array([[2, 1],
4               [1, 2]])
5
6 # 特征值和特征向量
7 eigenvalues, eigenvectors = np.linalg.eig(A)
8
9 print(f"特征值:\n{eigenvalues}")
10 print(f"特征向量:\n{eigenvectors}")
11
12 # 验证: A v = lambda v
13 for i in range(2):
14     v = eigenvectors[:, i]
15     lam = eigenvalues[i]
16     print(f"\n验证特征值 {lam}:")
17     print(f"A v = {A @ v}")
18     print(f"lambda * v = {lam * v}")
19
20 # 实对称矩阵的特征分解重构
21 Q = eigenvectors # 对实对称矩阵, 特征向量自然正交
22 L = np.diag(eigenvalues)
23 A_reconstructed = Q @ L @ Q.T
24 print(f"\n重构矩阵:\n{A_reconstructed}")
25 print(f"重构误差: {np.linalg.norm(A - A_reconstructed)}")

```

1.5.3 特征值与深度学习

特征值分析在深度学习中具有重要应用：

- **主成分分析 (PCA)**：数据协方差矩阵的特征向量对应数据的主方向，特征值表示该方向上的方差大小。
- **谱归一化**：用权重矩阵的最大奇异值（奇异值是特征值的一种推广）来归一化权重，稳定 GAN 训练。

- **梯度下降的收敛性**：损失函数的 Hessian 矩阵的特征值分布影响梯度下降的收敛速度。条件数（最大特征值与最小特征值之比）越大，收敛越慢。
- **权重初始化**：Xavier 和 He 初始化方法基于特征值分析，确保前向传播和反向传播时信号的方差保持稳定。

§ 1.6 奇异值分解

奇异值分解（Singular Value Decomposition, SVD）将特征分解推广到任意矩阵（不限于方阵）。

奇异值分解

对于任意矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ，存在分解：

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$$

其中：

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ 是正交矩阵，列向量称为左奇异向量
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ 是正交矩阵，列向量称为右奇异向量
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ 是对角矩阵，对角线元素 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ 称为奇异值

SVD 的几何解释：任何矩阵 $\mathbf{A}$ 的作用可以分解为三个步骤：

1. $\mathbf{V}^\top$ ：旋转（或反射）输入空间
2. $\mathbf{\Sigma}$ ：沿坐标轴缩放（奇异值为缩放因子）
3. $\mathbf{U}$ ：旋转（或反射）输出空间

1.6.1 低秩近似

SVD 的一个重要应用是矩阵的低秩近似。保留前 r 个最大的奇异值：

$$\mathbf{A} \approx \mathbf{U}_{:,1:r} \mathbf{\Sigma}_{1:r,1:r} (\mathbf{V}_{:,1:r})^\top$$

这是 $\mathbf{A}$ 在 Frobenius 范数意义下秩为 r 的最佳近似（Eckart-Young-Mirsky 定理）。

在深度学习中，低秩近似用于：

- **模型压缩**：对权重矩阵进行低秩分解减少参数。
- **LoRA (Low-Rank Adaptation)**：在微调大语言模型时，用低秩矩阵来近似权重更新。
- **推荐系统**：矩阵分解技术是协同过滤的核心。

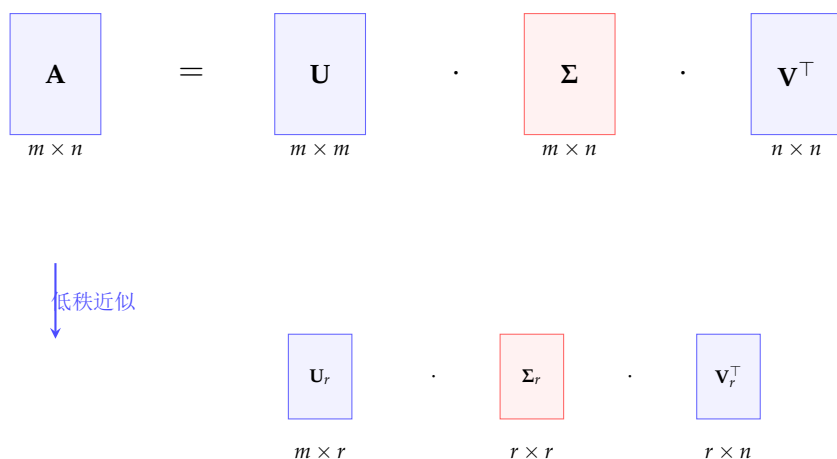


图 1.4: 完整的 SVD 与低秩近似。低秩近似仅保留前 r 个最大奇异值，将存储复杂度从 $O(mn)$ 降低到 $O(r(m+n))$ 。

Python 中的 SVD 与低秩近似

```

1 import numpy as np
2
3 # 创建矩阵
4 np.random.seed(42)
5 A = np.random.randn(6, 4)
6
7 # SVD 分解
8 U, S, Vt = np.linalg.svd(A, full_matrices=True)
9 print(f"奇异值: {S}")
10
11 # 重构验证
12 Sigma = np.zeros((6, 4))
13 np.fill_diagonal(Sigma, S)
14 A_reconstructed = U @ Sigma @ Vt
15 print(f"重构误差: {np.linalg.norm(A - A_reconstructed):.2e}")
16
17 # 低秩近似 (保留前 r 个奇异值)
18 r = 2
19 U_r = U[:, :r]
20 S_r = np.diag(S[:r])
21 Vt_r = Vt[:r, :]
22 A_approx = U_r @ S_r @ Vt_r
23
24 print(f"\n原始矩阵:\n{A}")
25 print(f"\n秩-{r} 近似:\n{A_approx}")
26 print(f"近似误差: {np.linalg.norm(A - A_approx):.4f}")
27
28 # 信息保留比例
29 energy_total = np.sum(S ** 2)
30 energy_approx = np.sum(S[:r] ** 2)
31 print(f"能量保留比例: {energy_approx / energy_total:.2%}")

```

§ 1.7 Moore-Penrose 伪逆

对于非方阵或奇异矩阵，常规的逆矩阵不存在。Moore-Penrose 伪逆（简称伪逆）提供了广义的逆矩阵概念。

Moore-Penrose 伪逆

对于矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ，其伪逆 $\mathbf{A}^+ \in \mathbb{R}^{n \times m}$ 定义为满足以下四个条件的唯一矩阵：

1. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$
2. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{A}^+$
3. $(\mathbf{A}\mathbf{A}^+)^\top = \mathbf{A}\mathbf{A}^+$
4. $(\mathbf{A}^+\mathbf{A})^\top = \mathbf{A}^+\mathbf{A}$

利用 SVD，伪逆可以通过以下公式计算：

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$$

其中 $\mathbf{\Sigma}^+$ 是 $\mathbf{\Sigma}$ 的伪逆：将非零奇异值取倒数后转置。

在实际应用中，当 $\mathbf{A}$ 可逆时， $\mathbf{A}^+ = \mathbf{A}^{-1}$ 。当 $\mathbf{A}$ 列满秩时， $\mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ 。伪逆在求解线性回归的正规方程中起到关键作用。

§ 1.8 行列式

行列式（Determinant）将方阵映射为一个标量，记为 $\det(\mathbf{A})$ 或 $|\mathbf{A}|$ 。

行列式的几何意义： $\det(\mathbf{A})$ 的绝对值表示矩阵 $\mathbf{A}$ 所代表的线性变换对空间体积的缩放因子。若 $\det(\mathbf{A}) = 0$ ，则变换将空间压缩到更低维度（矩阵不可逆）。

行列式的重要性质：

- $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$
- $\det(\mathbf{A}^\top) = \det(\mathbf{A})$
- $\det(\mathbf{A}^{-1}) = 1 / \det(\mathbf{A})$
- 特征值之积： $\det(\mathbf{A}) = \prod_i \lambda_i$

§ 1.9 迹

矩阵的迹（Trace）是主对角线元素之和：

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n A_{ii}$$

迹的重要性质：

- $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$
- $\text{tr}(c\mathbf{A}) = c \text{tr}(\mathbf{A})$
- $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ (循环置换不变性)
- $\text{tr}(\mathbf{A}) = \sum_i \lambda_i$ (特征值之和)

迹在深度学习中的一个重要用途是：标量的导数可以通过迹运算技巧来简化。例如， $\nabla_{\mathbf{A}} \text{tr}(\mathbf{AB}) = \mathbf{B}^\top$ 。

§ 1.10 PCA 与 SVD

主成分分析 (PCA) 是降维的经典方法，其数学基础与 SVD 紧密相连。给定数据中心化后的矩阵 $\mathbf{X} \in \mathbb{R}^{m \times n}$ (m 个样本， n 个特征)，PCA 的目标是找到投影方向使得投影后的方差最大。

PCA 的求解步骤：

1. 对数据去中心化 (减去均值)
2. 计算协方差矩阵 $\mathbf{C} = \frac{1}{m-1} \mathbf{X}^\top \mathbf{X}$
3. 对 $\mathbf{C}$ 进行特征分解，取前 k 个最大特征值对应的特征向量作为主成分

等价地，可以直接对 $\mathbf{X}$ 进行 SVD: $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ ，则 $\mathbf{V}$ 的前 k 列即为前 k 个主成分方向。

Python 中的 PCA 实现

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(42)
5 # 生成二维数据（有相关性的数据）
6 n_samples = 200
7 mean = [3, 4]
8 cov = [[2.0, 1.5],
9         [1.5, 1.0]]
10 X = np.random.multivariate_normal(mean, cov, n_samples)
11
12 # 1. 去中心化
13 X_centered = X - X.mean(axis=0)
14
15 # 2. 协方差矩阵
16 C = (X_centered.T @ X_centered) / (n_samples - 1)
17
18 # 3. 特征分解
19 eigenvalues, eigenvectors = np.linalg.eig(C)
20
21 # 按特征值降序排列
22 idx = eigenvalues.argsort()[::-1]
23 eigenvalues = eigenvalues[idx]
24 eigenvectors = eigenvectors[:, idx]
25
26 print(f"特征值: {eigenvalues}")
27 print(f"主成分方向:\n{eigenvectors}")
28
29 # 4. 投影到第一主成分
30 X_pca = X_centered @ eigenvectors[:, :1]
31 print(f"降维后数据形状: {X_pca.shape}")
32
33 # 5. 使用SVD方法（等价）
34 U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
35 principal_components_svd = Vt[:1, :].T # 第一主成分
36 print(f"SVD方法的主成分方向:\n{principal_components_svd}")
37
38 # 解释方差比例
39 explained_variance_ratio = eigenvalues / eigenvalues.sum()
40 print(f"各主成分解释方差比例: {explained_variance_ratio}")
```

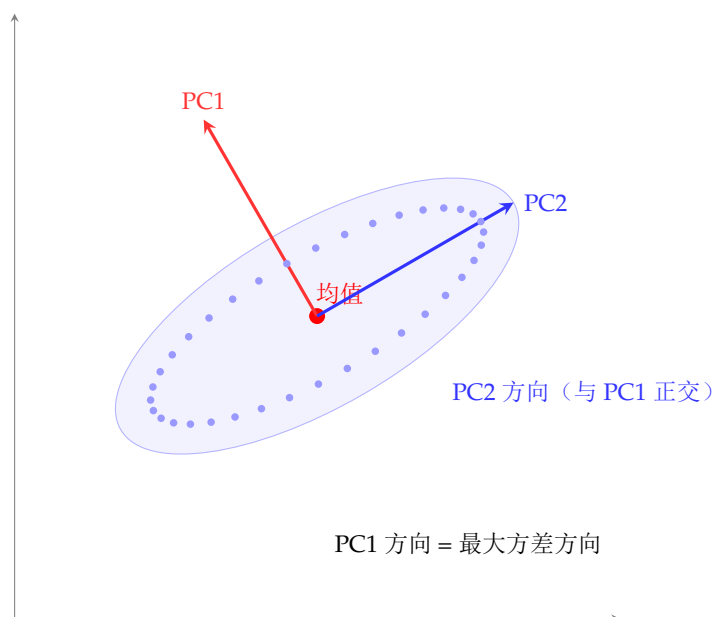


图 1.5: PCA 主成分方向的几何解释。PC1 是数据方差最大的方向，PC2 与 PC1 正交且方差次之。

§ 1.11

向量与矩阵微积分

在后续章节中，我们将频繁使用矩阵微积分。这里给出常用公式。

对于向量和矩阵的导数，有：

- $\nabla_{\mathbf{x}}(\mathbf{a}^T \mathbf{x}) = \mathbf{a}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^T \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^T) \mathbf{x}$
- $\nabla_{\mathbf{x}} \|\mathbf{x}\|_2^2 = 2\mathbf{x}$
- $\nabla_{\mathbf{A}}(\text{tr}(\mathbf{A}\mathbf{B})) = \mathbf{B}^T$
- $\nabla_{\mathbf{A}}(\det(\mathbf{A})) = \det(\mathbf{A})(\mathbf{A}^{-1})^T$

这些公式将在推导梯度下降、正规方程和反向传播时反复出现。

§ 1.12

本章小结

本章介绍了深度学习所需的线性代数基础知识。核心概念包括：

- 标量、向量、矩阵和张量是多维数组的分层抽象
- 矩阵乘法是神经网络前向传播的基本运算
- 范数用于衡量向量和矩阵的大小，在正则化中起关键作用
- 特征分解揭示了矩阵的本质结构，SVD 将其推广到任意矩阵
- PCA 利用特征分解实现数据降维

- SVD 的低秩近似在模型压缩中具有重要应用

牢固掌握这些线性代数知识，将为理解深度学习的数学本质打下坚实的基础。

微积分基础

微积分是理解深度学习优化算法的关键。梯度下降法建立在导数和梯度的概念之上，反向传播算法本质上是链式法则的系统应用。本章系统介绍极限与导数、偏导数与梯度、链式法则、Hessian 矩阵、Taylor 展开、凸优化等核心主题。

§2.1
极限与连续性

2.1.1 极限的定义

函数的极限

设函数 $f(x)$ 在点 x_0 的某个去心邻域内有定义。若存在常数 L ，使得当 x 无限接近 x_0 时， $f(x)$ 无限接近 L ，则称 L 为 $x \rightarrow x_0$ 时 $f(x)$ 的极限，记为：

$$\lim_{x \rightarrow x_0} f(x) = L$$

精确的 ε - δ 定义： $\forall \varepsilon > 0, \exists \delta > 0$ ，当 $0 < |x - x_0| < \delta$ 时，有 $|f(x) - L| < \varepsilon$ 。

2.1.2 连续性

若 $\lim_{x \rightarrow x_0} f(x) = f(x_0)$ ，则称 f 在 x_0 处连续。深度学习中的常见激活函数（ReLU、sigmoid、tanh）在定义域上都是连续的，但 ReLU 在 $x = 0$ 处的导数不连续，这在数学上不影响优化（因为梯度下降使用的是次梯度）。

§2.2
导数

2.2.1 导数的定义

导数

函数 $f(x)$ 在点 x 处的导数定义为：

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

若该极限存在，则称 f 在 x 处可导。

导数的几何意义是函数曲线在该点处切线的斜率，物理意义是瞬时变化率。在深度学习中，导数衡量了损失函数关于参数的“瞬时”变化率，指示了调整参数的方向——沿着负梯度方向移动可以减小损失函数的值。

2.2.2 基本导数公式

$$\frac{d}{dx}(c) = 0 \quad (2.1)$$

$$\frac{d}{dx}(x^n) = nx^{n-1} \quad (2.2)$$

$$\frac{d}{dx}(e^x) = e^x \quad (2.3)$$

$$\frac{d}{dx}(\ln x) = \frac{1}{x} \quad (2.4)$$

$$\frac{d}{dx}(\sin x) = \cos x \quad (2.5)$$

$$\frac{d}{dx}(\cos x) = -\sin x \quad (2.6)$$

$$\frac{d}{dx}(a^x) = a^x \ln a \quad (2.7)$$

2.2.3 导数的运算法则

导数运算法则

设 f 和 g 在 x 处可导，则：

- 线性性： $(af + bg)' = af' + bg'$ （其中 a, b 为常数）
- 乘法法则： $(fg)' = f'g + fg'$
- 除法法则： $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$
- 链式法则： $(f(g(x)))' = f'(g(x)) \cdot g'(x)$

2.2.4 常见激活函数的导数

深度学习中的激活函数及其导数具有重要地位：

- **Sigmoid**: $\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- **Tanh**: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, $\tanh'(x) = 1 - \tanh^2(x)$
- **ReLU**: $\text{ReLU}(x) = \max(0, x)$, $\text{ReLU}'(x) = \begin{cases} 0 & x < 0 \\ 1 & x > 0 \end{cases}$ （在 $x = 0$ 处导数未定义，实践中通常取 0 或使用次梯度）
- **Leaky ReLU**: $f(x) = \max(\alpha x, x)$, α 为一个小的正数（如 0.01）
- **GELU**: 高斯误差线性单元，常用于 Transformer 模型

Python 中的激活函数与导数

```

1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def sigmoid_derivative(x):
7     s = sigmoid(x)
8     return s * (1 - s)
9
10 def relu(x):
11     return np.maximum(0, x)
12
13 def relu_derivative(x):
14     return (x > 0).astype(float)
15
16 def tanh_derivative(x):
17     t = np.tanh(x)
18     return 1 - t ** 2
19
20 # 测试
21 x = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
22 print(f"x: {x}")
23 print(f"sigmoid(x): {sigmoid(x)}")
24 print(f"sigmoid'(x): {sigmoid_derivative(x)}")
25 print(f"ReLU(x): {relu(x)}")
26 print(f"ReLU'(x): {relu_derivative(x)}")

```

§2.3 偏导数与梯度

2.3.1 偏导数

对于多元函数 $f(x_1, x_2, \dots, x_n)$ ，关于 x_i 的偏导数定义为在固定其他变量情况下， f 关于 x_i 的变化率：

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}$$

偏导数的计算规则与一元导数相同，只是将其他变量视为常数。

【例题 2.1】 求 $f(x, y) = x^2y + 3xy^2$ 的偏导数。

$$\frac{\partial f}{\partial x} = 2xy + 3y^2, \quad \frac{\partial f}{\partial y} = x^2 + 6xy$$

2.3.2 梯度

梯度

函数 $f(x_1, x_2, \dots, x_n)$ 的梯度是由所有偏导数组成的向量：

$$\nabla f = \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \dots \quad \frac{\partial f}{\partial x_n} \right]^\top$$

梯度的几何意义：梯度向量 ∇f 指向函数值增长最快的方向，其模长表示该方向上的最大变化率。因此， $-\nabla f$ 指向函数值下降最快的方向——这正是梯度下降算法的核心思想。

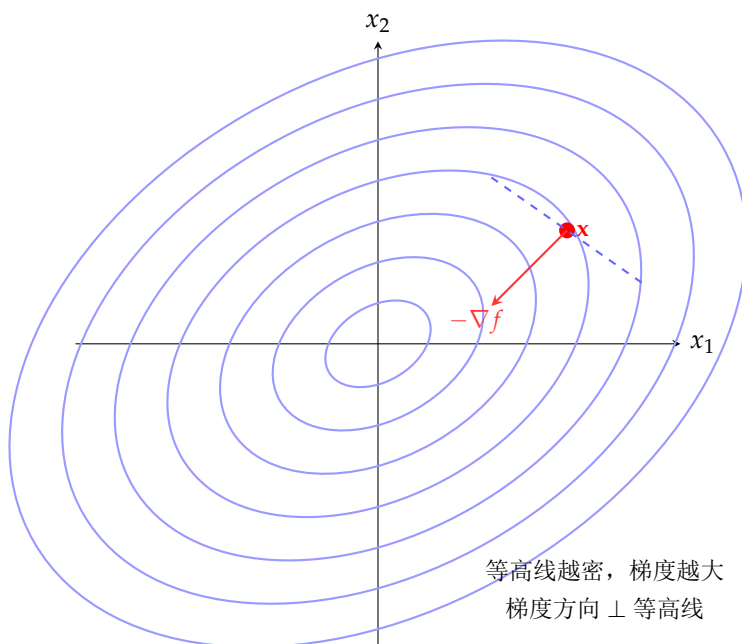


图 2.1: 梯度的几何意义。梯度方向与函数等高线垂直，指向函数值增长最快的方向。负梯度方向指向函数值下降最快的方向。

2.3.3 Jacobian 矩阵

当输出是多维向量时（如神经网络的某一层输出），推广梯度概念得到 Jacobian 矩阵。设 $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，其中 $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), \dots, f_m(\mathbf{x})]^\top$ ，则 Jacobian 矩阵为：

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Jacobian 矩阵的每一行是对应输出分量关于所有输入变量的梯度，每一列是某一输入对所有输出的影响。在反向传播中，Jacobian 矩阵描述了每一层的局部导数。

Python 中的梯度计算（数值微分）

```

1 import numpy as np
2
3 def numerical_gradient(f, x, h=1e-5):
4     """用中心差分法计算 f 在 x 处的梯度"""
5     grad = np.zeros_like(x)
6     it = np.nditer(x, flags=['multi_index'])
7     while not it.finished:
8         idx = it.multi_index
9         x_plus = x.copy()
10        x_minus = x.copy()
11        x_plus[idx] += h
12        x_minus[idx] -= h
13        grad[idx] = (f(x_plus) - f(x_minus)) / (2 * h)
14        it.iternext()
15    return grad
16
17 # 测试函数: f(x, y) = x^2 + y^2
18 def f(x):
19     return x[0]**2 + x[1]**2
20
21 x0 = np.array([3.0, 4.0])
22 grad = numerical_gradient(f, x0)
23 print(f"f(x) = x0^2 + x1^2")
24 print(f"在 x={x0} 处的数值梯度: {grad}")
25 print(f"解析梯度 (2x): {2 * x0}")
26
27 # 测试: f(x, y) = x^2 * y + sin(y)
28 def g(x):
29     return x[0]**2 * x[1] + np.sin(x[1])
30
31 x1 = np.array([2.0, np.pi/4])
32 grad_g = numerical_gradient(g, x1)
33 print(f"\n在 x={x1} 处的梯度: {grad_g}")
34 # 解析: [2xy, x^2 + cos(y)]
35 analytic = np.array([2*2*np.pi/4, 4 + np.cos(np.pi/4)])
36 print(f"解析梯度: {analytic}")

```

§2.4
链式法则

2.4.1 一元函数的链式法则

若 $y = f(u)$ 且 $u = g(x)$, 则复合函数 $y = f(g(x))$ 的导数为:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = f'(g(x)) \cdot g'(x)$$

2.4.2 多元函数的链式法则

对于多元复合函数 $z = f(u, v)$ 其中 $u = g(x, y)$ 、 $v = h(x, y)$ ：

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial z}{\partial v} \frac{\partial v}{\partial x}$$

类似地求 $\frac{\partial z}{\partial y}$ 。这可以推广到任意多个中间变量。

2.4.3 反向传播与链式法则

反向传播（Backpropagation）本质上是链式法则在计算图上的高效实现。考虑一个简单的三层网络：

$$\mathbf{h}^{(1)} = \sigma(\mathbf{W}^{(1)}\mathbf{x}), \quad \mathbf{h}^{(2)} = \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)}), \quad \hat{\mathbf{y}} = \mathbf{W}^{(3)}\mathbf{h}^{(2)}$$

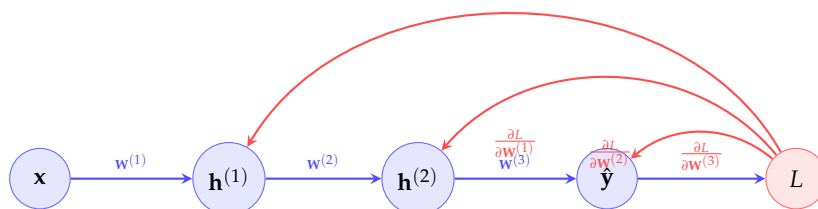
损失函数 $L = \ell(\hat{\mathbf{y}}, \mathbf{y})$ 。梯度计算如下：

$$\frac{\partial L}{\partial \mathbf{W}^{(3)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{W}^{(3)}}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}^{(2)}} \cdot \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{W}^{(2)}}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}^{(2)}} \cdot \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

注意每一步需要乘以上一层的局部梯度（Jacobian 矩阵），这正是“反向”的含义——梯度从输出层逐层传播回输入层。



红色：反向传播路径

蓝色：前向传播路径

图 2.2: 反向传播示意图。前向传播（蓝色）计算每一层的输出，反向传播（红色）从损失函数出发，利用链式法则逐层计算梯度。

Python 中的简单反向传播实现

```

1 import numpy as np
2
3 np.random.seed(42)
4
5 # 简单三层网络
6 n_input = 4; n_hidden1 = 3; n_hidden2 = 2; n_output = 1
7
8 W1 = np.random.randn(n_hidden1, n_input) * 0.1
9 W2 = np.random.randn(n_hidden2, n_hidden1) * 0.1
10 W3 = np.random.randn(n_output, n_hidden2) * 0.1
11
12 x = np.random.randn(n_input, 1)
13 y_true = np.array([[1.0]])
14
15 def sigmoid(z):
16     return 1 / (1 + np.exp(-z))
17
18 # 前向传播
19 z1 = W1 @ x;          a1 = sigmoid(z1)
20 z2 = W2 @ a1;          a2 = sigmoid(z2)
21 z3 = W3 @ a2;          a3 = z3 # 线性输出
22
23 # 损失 (MSE)
24 loss = 0.5 * np.sum((a3 - y_true) ** 2)
25 print(f"前向损失: {loss:.6f}")
26
27 # 反向传播
28 delta3 = a3 - y_true          # dL/da3
29 dW3 = delta3 @ a2.T           # dL/dW3
30
31 delta2 = (W3.T @ delta3) * (a2 * (1 - a2)) # dL/da2 -> dL/dz2
32 dW2 = delta2 @ a1.T           # dL/dW2
33
34 delta1 = (W2.T @ delta2) * (a1 * (1 - a1)) # dL/da1 -> dL/dz1
35 dW1 = delta1 @ x.T            # dL/dW1
36
37 print(f"dL/dW3 形状: {dW3.shape}")
38 print(f"dL/dW2 形状: {dW2.shape}")
39 print(f"dL/dW1 形状: {dW1.shape}")
40
41 # 手动梯度下降一步
42 lr = 0.01
43 W3 -= lr * dW3
44 W2 -= lr * dW2
45 W1 -= lr * dW1
46 print(f"一步更新后的权重大小: {np.sum(np.abs(W1)):.4f}")

```


§ 2.5 Hessian 矩阵

Hessian 矩阵

对于二阶可导的函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，其 Hessian 矩阵 $\mathbf{H}$ 是由所有二阶偏导数组成的 $n \times n$ 矩阵：

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Hessian 矩阵是对称矩阵（当二阶混合偏导数连续时，即 $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$ ）。

Hessian 矩阵在深度学习中的作用：

- 驻点判断：若 $\nabla f(\mathbf{x}^*) = 0$ 且 $\mathbf{H}(\mathbf{x}^*)$ 正定，则 $\mathbf{x}^*$ 是局部极小点。
- 牛顿法：利用 Hessian 信息实现二阶优化： $\mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{H}^{-1} \nabla f(\mathbf{x}_t)$ 。
- 曲率信息：Hessian 的特征值反映损失函数在不同方向上的弯曲程度。最大特征值对应最陡峭的方向，最小特征值对应最平坦的方向。
- 鞍点分析：在高维优化中，Hessian 同时有正负特征值意味着存在鞍点，这是深度学习优化的主要挑战之一。

由于深度学习参数数量巨大，直接计算和存储完整的 Hessian 矩阵不现实，因此实际中常用近似方法（如 BFGS、L-BFGS、Hessian-Free 优化）。

§ 2.6 Taylor 展开

Taylor 展开是将函数在某点附近用多项式逼近的方法。它是理解优化算法收敛性的理论基础。

2.6.1 一元函数的 Taylor 展开

若 f 在 x_0 处 $n+1$ 阶可导，则在 x_0 附近：

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + R_n(x)$$

其中 $R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}$ 为 Lagrange 余项， ξ 在 x 和 x_0 之间。
常用的一阶和二阶 Taylor 展开：

- 一阶： $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$ （线性近似）
- 二阶： $f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2$ （二次近似）

2.6.2 多元函数的 Taylor 展开

对于多元函数 $f(\mathbf{x})$ ，在 $\mathbf{x}_0$ 附近的二阶 Taylor 展开为：

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}(\mathbf{x}_0) (\mathbf{x} - \mathbf{x}_0)$$

这一展开式在分析梯度下降的收敛性时非常重要。例如，用二阶 Taylor 展开分析学习率对收敛性的影响：设 $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$ ，代入上式得：

$$f(\mathbf{x}_{t+1}) \approx f(\mathbf{x}_t) - \eta \|\nabla f(\mathbf{x}_t)\|^2 + \frac{\eta^2}{2} \nabla f(\mathbf{x}_t)^\top \mathbf{H} \nabla f(\mathbf{x}_t)$$

为使损失下降，需要 $\eta < \frac{2\|\nabla f\|^2}{\nabla f^\top \mathbf{H} \nabla f}$ 。

Python 中的 Taylor 展开验证

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.sin(x) + 0.1 * x**2
6
7 def f_prime(x):
8     return np.cos(x) + 0.2 * x
9
10 def f_double_prime(x):
11     return -np.sin(x) + 0.2
12
13 x0 = 1.0 # 展开点
14
15 # 计算Taylor展开
16 x = np.linspace(-2, 4, 200)
17 fx = f(x)
18
19 # 一阶Taylor
20 taylor1 = f(x0) + f_prime(x0) * (x - x0)
21
22 # 二阶Taylor
23 taylor2 = taylor1 + 0.5 * f_double_prime(x0) * (x - x0)**2
24
25 # 三阶Taylor
26 f_triple = lambda x: -np.cos(x)
27 taylor3 = taylor2 + (f_triple(x0) / 6) * (x - x0)**3
28
29 print(f"展开点 x0 = {x0}")
30 print(f"f(x0) = {f(x0):.4f}")
31 print(f"在 x=0.5 处:")
32 print(f"  真实值: {f(0.5):.4f}")
33 print(f"  一阶近似: {f(x0) + f_prime(x0)*(0.5-x0):.4f}")
34 print(f"  二阶近似: {f(x0) + f_prime(x0)*(0.5-x0) + 0.5*f_double_prime(x0)
    *(0.5-x0)**2:.4f}")

```

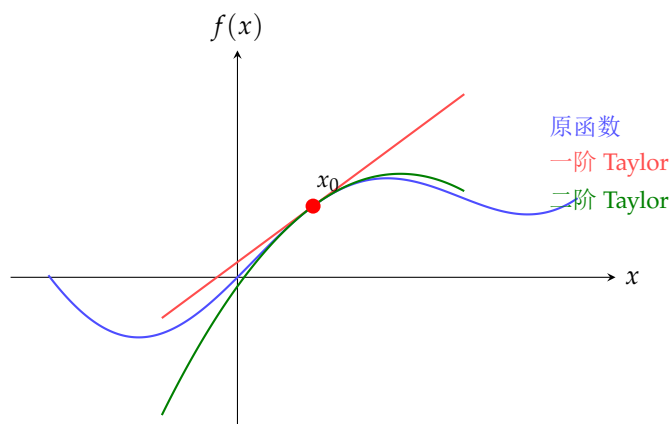


图 2.3: 函数在一阶和二阶 Taylor 展开下的近似。在展开点附近，二阶近似比一阶近似更精确，但远离展开点时近似误差会增大。

§2.7 凸优化基础

2.7.1 凸集与凸函数

凸集

集合 $C \subseteq \mathbb{R}^n$ 是凸集，若对任意 $\mathbf{x}, \mathbf{y} \in C$ 和任意 $\lambda \in [0, 1]$ ，有 $\lambda \mathbf{x} + (1 - \lambda)\mathbf{y} \in C$ 。即连接集合中任意两点的线段完全包含在集合中。

凸函数

定义在凸集 C 上的函数 f 是凸函数，若对任意 $\mathbf{x}, \mathbf{y} \in C$ 和任意 $\lambda \in [0, 1]$ ，有：

$$f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$$

即函数图像在连接任意两点的线段下方。

若不等式严格成立 ($\mathbf{x} \neq \mathbf{y}$ 且 $\lambda \in (0, 1)$ 时)，则称 f 为严格凸函数。

2.7.2 凸函数的判定条件

凸函数的判定

设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 二阶可导，则：

- f 是凸函数当且仅当其 Hessian 矩阵处处半正定： $\mathbf{H}(\mathbf{x}) \succeq 0, \forall \mathbf{x}$
- 若 $\mathbf{H}(\mathbf{x}) \succ 0$ (正定)，则 f 是严格凸函数

对于一元函数，条件简化为 $f''(x) \geq 0$ (处处成立)。

常见的凸函数：

- $f(x) = x^2$: 严格凸
- $f(x) = e^x$: 严格凸

- $f(x) = |x|$: 凸但不严格
- $f(x) = x \log x$ ($x > 0$): 严格凸
- MSE 损失 $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$: 凸
- 带有 L^2 正则化的线性回归损失: 严格凸

2.7.3 凸优化的性质

凸优化问题具有优良的性质:

1. 局部极小即全局极小: 对凸函数, 任何局部极小点都是全局极小点。
2. 驻点即极小点: 若 $\nabla f(\mathbf{x}^*) = 0$, 则 $\mathbf{x}^*$ 是全局极小点。
3. 梯度下降收敛到全局最优: 对于凸函数, 梯度下降可以保证收敛到全局最优解。

然而, 深度学习的损失函数通常是高度非凸的, 存在大量局部极小点和鞍点。但实践表明, SGD 等方法在深度网络中表现良好, 部分原因在于:

- 高维空间中的鞍点比局部极小点更常见
- SGD 的随机噪声有助于逃离鞍点
- 过参数化的网络往往使优化问题变得“更容易”

Python 中的凸函数与非凸函数对比

```

1 import numpy as np
2
3 # 凸函数:  $f(x) = x^2$ 
4 def convex_f(x):
5     return x**2
6
7 # 非凸函数:  $f(x) = \sin(x) + 0.1 * x^2$ 
8 def nonconvex_f(x):
9     return np.sin(3*x) + 0.1 * x**2
10
11 # 验证凸性: 检查 Jensen 不等式
12 def check_convexity(f, x1, x2, n_points=100):
13     violations = 0
14     for lam in np.linspace(0, 1, n_points):
15         mid = lam * x1 + (1 - lam) * x2
16         f_mid = f(mid)
17         bound = lam * f(x1) + (1 - lam) * f(x2)
18         if f_mid > bound + 1e-10:
19             violations += 1
20     return violations
21
22 v1 = check_convexity(convex_f, -2, 3)
23 v2 = check_convexity(nonconvex_f, -2, 3)
24 print(f"f(x)=x^2 违反凸性次数: {v1} (应为0)")
25 print(f"f(x)=sin(3x)+0.1x^2 违反凸性次数: {v2} (预期>0)")

```

§2.8 约束优化与 Lagrange 乘子法

在深度学习中，有时需要在约束条件下优化目标函数。Lagrange 乘子法是处理等式约束优化的基本方法。

Lagrange 乘子法

考虑约束优化问题： $\min_{\mathbf{x}} f(\mathbf{x})$ ，满足 $g(\mathbf{x}) = 0$ 。构造 Lagrange 函数：

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$$

则最优解满足 $\nabla_{\mathbf{x}} \mathcal{L} = 0$ 和 $\nabla_{\lambda} \mathcal{L} = g(\mathbf{x}) = 0$ 。

对于不等式约束 $g(\mathbf{x}) \leq 0$ ，使用 KKT 条件（Karush-Kuhn-Tucker conditions）进行推广。SVM（支持向量机）的对偶推导就是 Lagrange 乘子法的典型应用。

§2.9 本章小结

本章介绍了深度学习所需的微积分基础知识：

- 导数和偏导数描述了函数的变化率，梯度指向函数增长最快的方向
- 链式法则是反向传播算法的数学基础
- Hessian 矩阵提供了二阶曲率信息，用于判断驻点类型
- Taylor 展开将复杂函数局部近似为多项式
- 凸函数保证局部极小即全局极小
- 在深度学习中，梯度下降、牛顿法和各种优化器都建立在微积分理论之上

这些概念将在后续章节中反复出现。下一章将介绍概率论与统计学基础，为理解损失函数的概率解释和贝叶斯方法做准备。

概率与统计基础

概率论与统计学为深度学习提供了描述不确定性的数学语言。许多损失函数（如交叉熵）可以从概率角度推导出来，生成模型直接建模数据的概率分布，而贝叶斯方法则为模型选择提供了统一框架。本章介绍随机变量、常见分布、条件概率与贝叶斯定理、期望与方差、信息论基础，以及最大似然估计等内容。

§3.1

随机变量与概率分布

3.1.1 离散随机变量

离散随机变量与概率质量函数

离散随机变量 X 取有限或可数无限个值。其概率质量函数（PMF）定义为：

$$P(X = x) = p_X(x), \quad \sum_x p_X(x) = 1$$

其中 $p_X(x) \in [0, 1]$ 对于所有 x 。

常见的离散分布：

- Bernoulli 分布**：描述单次二值试验。 $X \in \{0, 1\}$, $P(X = 1) = p$, $P(X = 0) = 1 - p$ 。PMF: $p_X(x) = p^x(1 - p)^{1-x}$ 。
- 二项分布**： n 次独立 Bernoulli 试验中成功的次数。 $X \sim \text{Binomial}(n, p)$, $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$ 。
- 多项分布**：二项分布向多类别的推广。每个类别 k 有概率 p_k , $\sum_k p_k = 1$ 。
- Poisson 分布**：描述单位时间内随机事件发生的次数。 $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, $\lambda > 0$ 。
- 类别分布 (Categorical)**：深度学习中分类问题的输出分布。 $P(X = k) = p_k$, $\sum_{k=1}^K p_k = 1$ 。当 $K = 2$ 时退化为 Bernoulli 分布。

3.1.2 连续随机变量

连续随机变量与概率密度函数

连续随机变量 X 的概率密度函数 (PDF) $f_X(x)$ 满足 $f_X(x) \geq 0$ 且:

$$\int_{-\infty}^{\infty} f_X(x) dx = 1$$

X 落在区间 $[a, b]$ 内的概率为 $P(a \leq X \leq b) = \int_a^b f_X(x) dx$ 。

常见的连续分布:

- 均匀分布: $X \sim U(a, b)$, PDF: $f_X(x) = \frac{1}{b-a}$, $x \in [a, b]$ 。

- 正态分布 (高斯分布): $X \sim \mathcal{N}(\mu, \sigma^2)$, PDF:

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- 多元正态分布: $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, PDF:

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

- 指数分布: 描述等待时间。 $f_X(x) = \lambda e^{-\lambda x}$, $x \geq 0$ 。

- Laplace 分布: $f_X(x) = \frac{1}{2b} \exp(-\frac{|x-\mu|}{b})$, 常用于 L^1 正则化的贝叶斯解释。

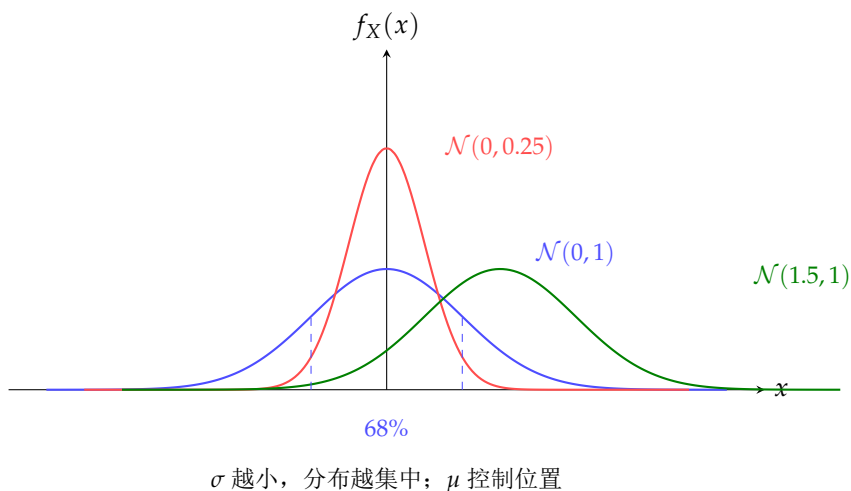


图 3.1: 不同均值和方差的正态分布概率密度函数。 μ 控制分布的中心位置, σ 控制分布的宽度。

Python 中的概率分布

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 # 正态分布
6 mu, sigma = 0, 1
7 x = np.linspace(-4, 4, 200)
8 pdf = stats.norm.pdf(x, mu, sigma)
9
10 # 计算概率
11 print(f"P(-1 < X < 1) = {stats.norm.cdf(1) - stats.norm.cdf(-1):.4f}")
12 print(f"P(-2 < X < 2) = {stats.norm.cdf(2) - stats.norm.cdf(-2):.4f}")
13
14 # 采样
15 samples = np.random.normal(mu, sigma, 10000)
16 print(f"采样均值: {samples.mean():.4f} (理论: 0)")
17 print(f"采样标准差: {samples.std():.4f} (理论: 1)")
18
19 # Bernoulli 分布
20 p = 0.7
21 bernoulli_samples = np.random.binomial(1, p, 10000)
22 print(f"\nBernoulli(p={p}) 样本均值: {bernoulli_samples.mean():.4f}")
23
24 # 多元正态分布
25 mean = [1, 2]
26 cov = [[2.0, 0.8],
27         [0.8, 1.0]]
28 multi_samples = np.random.multivariate_normal(mean, cov, 5000)
29 print(f"\n多元正态样本均值: {multi_samples.mean(axis=0)}")
30 print(f"多元正态样本协方差: \n{np.cov(multi_samples.T)}")

```

§3.2

条件概率与贝叶斯定理

3.2.1 条件概率

条件概率

在事件 B 发生的条件下，事件 A 发生的概率为：

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

条件概率是理解贝叶斯推断、生成模型和概率图模型的基础。直观上，条件概率描述了在获得新信息（ B 发生）后，对事件 A 发生概率的更新。

3.2.2 全概率公式与贝叶斯定理

全概率公式

若 $\{B_1, B_2, \dots, B_n\}$ 是样本空间的一个划分，则：

$$P(A) = \sum_{i=1}^n P(A | B_i)P(B_i)$$

贝叶斯定理

$$P(B | A) = \frac{P(A | B)P(B)}{P(A)}$$

其中：

- $P(B)$ ：先验概率（Prior）
- $P(A | B)$ ：似然（Likelihood）
- $P(B | A)$ ：后验概率（Posterior）
- $P(A)$ ：证据（Evidence），可由全概率公式求得

贝叶斯定理在机器学习中有深远影响：

- 朴素贝叶斯分类器：假设特征条件独立，利用贝叶斯定理进行分类。
- 贝叶斯神经网络：为网络权重建模概率分布而非点估计。
- 变分推断：通过优化逼近后验分布。
- 贝叶斯优化：用于超参数调优，高效探索参数空间。

【例题 3.1】（医疗诊断问题）某种疾病在人群中的发病率为 0.1% ($P(D) = 0.001$)。检测方法的灵敏度为 99% ($P(+ | D) = 0.99$)，假阳性率为 2% ($P(+ | \neg D) = 0.02$)。若某人检测结果为阳性，他真正患病的概率是多少？

由全概率公式：

$$P(+) = P(+ | D)P(D) + P(+ | \neg D)P(\neg D) = 0.99 \times 0.001 + 0.02 \times 0.999 = 0.02097$$

由贝叶斯定理：

$$P(D | +) = \frac{P(+ | D)P(D)}{P(+)} = \frac{0.99 \times 0.001}{0.02097} \approx 0.0472$$

即便检测结果为阳性，真正患病的概率也只有约 4.72%——这体现了先验概率的重要性。

3.2.3 独立性与条件独立

若 $P(A \cap B) = P(A)P(B)$ ，则事件 A 和 B 独立。独立意味着知道 A 是否发生不影响 B 的概率。

在给定 C 条件下，若 $P(A \cap B | C) = P(A | C)P(B | C)$ ，则 A 和 B 关于 C 条件独立。条件独立是朴素贝叶斯和概率图模型中的核心假设。

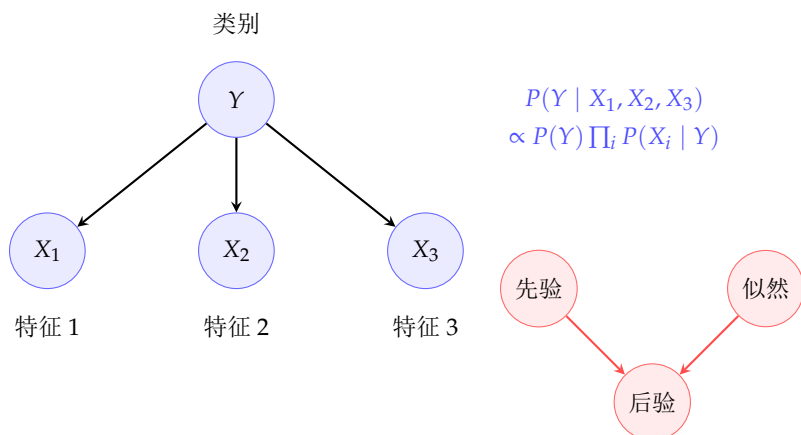


图 3.2: 朴素贝叶斯的概率图模型（左）和贝叶斯推断的基本框架（右）。在给定类别 Y 的条件下，各特征 X_i 相互独立。

§ 3.3 期望、方差与协方差

3.3.1 期望

期望

随机变量 X 的期望（数学期望/均值）是其所有可能值按概率加权平均：

$$\mathbb{E}[X] = \begin{cases} \sum_x x \cdot p_X(x) & \text{离散} \\ \int_{-\infty}^{\infty} x \cdot f_X(x) dx & \text{连续} \end{cases}$$

期望的性质（线性性）：

- $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ (a, b 为常数)
- $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ 当 X, Y 独立时
- $\mathbb{E}[g(X)] = \sum_x g(x)p_X(x)$ 或 $\int g(x)f_X(x)dx$

3.3.2 方差与标准差

方差与标准差

方差衡量随机变量围绕其均值的离散程度：

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

标准差 $\sigma_X = \sqrt{\text{Var}(X)}$ 。

方差的性质：

- $\text{Var}(aX + b) = a^2 \text{Var}(X)$
- 若 X 和 Y 独立，则 $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$

3.3.3 协方差与相关系数

协方差

两个随机变量 X 和 Y 的协方差衡量它们之间的线性相关程度：

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

协方差矩阵 Σ 汇总了多个随机变量间的协方差：

$$\Sigma_{ij} = \text{Cov}(X_i, X_j)$$

相关系数 ρ_{XY} 是标准化后的协方差，取值范围为 $[-1, 1]$ ：

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

$\rho_{XY} = 1$ 表示完全正相关， $\rho_{XY} = -1$ 表示完全负相关， $\rho_{XY} = 0$ 表示无线性相关（但可能存在非线性关系）。

Python 中的期望与协方差

```

1 import numpy as np
2
3 np.random.seed(42)
4 n = 10000
5
6 # 生成相关的两个变量
7 X = np.random.randn(n)
8 Y = 0.7 * X + np.random.randn(n) * np.sqrt(1 - 0.7**2) # 相关系数约0.7
9
10 print(f"E[X] = {X.mean():.4f}")
11 print(f"Var(X) = {X.var():.4f}")
12 print(f"Cov(X,Y) = {np.cov(X, Y)[0, 1]:.4f}")
13 print(f"Corr(X,Y) = {np.corrcoef(X, Y)[0, 1]:.4f}")
14
15 # 协方差矩阵
16 data = np.column_stack([X, Y])
17 cov_matrix = np.cov(data.T)
18 print(f"\n协方差矩阵:\n{cov_matrix}")
19
20 # 验证性质: Var(X+Y) = Var(X) + Var(Y) + 2Cov(X,Y)
21 var_sum_direct = (X + Y).var()
22 var_sum_formula = X.var() + Y.var() + 2 * np.cov(X, Y)[0, 1]
23 print(f"\nVar(X+Y) 直接计算: {var_sum_direct:.4f}")
24 print(f"Var(X+Y) 按公式: {var_sum_formula:.4f}")

```

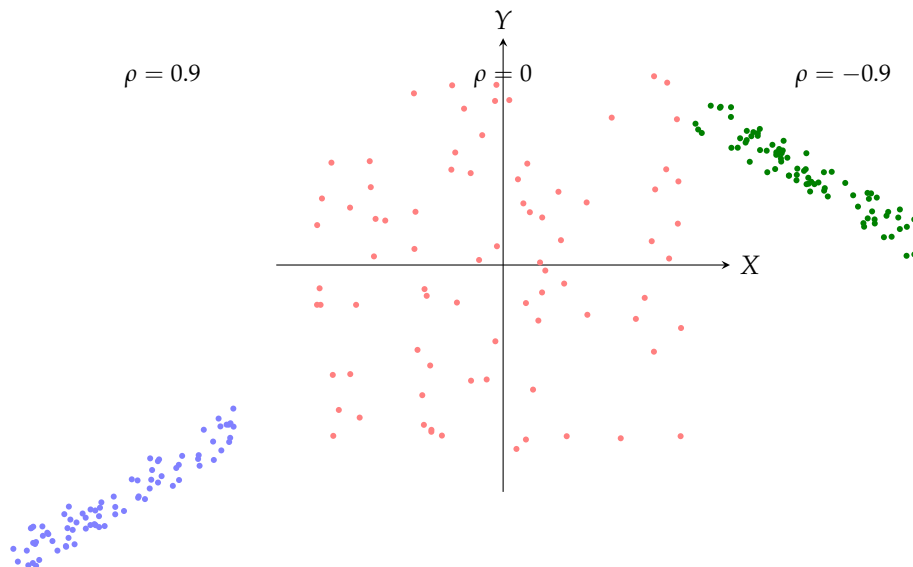


图 3.3: 不同相关系数下的散点图。 $\rho = 0.9$ 时呈现强正相关， $\rho = 0$ 时无明显线性关系， $\rho = -0.9$ 时呈现强负相关。

§ 3.4 信息论基础

信息论由 Claude Shannon 于 1948 年创立，为深度学习中的损失函数设计和模型评估提供了理论基础。

3.4.1 信息量与熵

自信息

事件 $X = x$ 的自信息量定义为：

$$I(x) = -\log P(X = x)$$

通常使用以 2 为底的对数（单位为比特）或以 e 为底的对数（单位为纳特，nat）。概率越小的事件包含的信息量越大。

熵

随机变量 X 的熵（Shannon 熵）是自信息的期望，衡量其不确定性：

$$H(X) = \mathbb{E}_X[I(X)] = -\sum_x P(X = x) \log P(X = x)$$

对于连续随机变量，微分熵为：

$$H(X) = -\int f_X(x) \log f_X(x) dx$$

熵的性质：

- $H(X) \geq 0$ （离散情况），当 X 退化为确定取值时 $H(X) = 0$
- 对于有 K 个可能取值的离散随机变量， $H(X) \leq \log K$ ，当分布均匀时熵最大

- 正态分布在给定方差下具有最大微分熵

3.4.2 KL 散度与交叉熵

KL 散度

KL 散度 (Kullback-Leibler divergence) 衡量分布 Q 相对于分布 P 的差异:

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

KL 散度的重要性质:

- 非负性: $D_{\text{KL}}(P\|Q) \geq 0$, 只在 $P = Q$ 时取等号 (Gibbs 不等式)
- 非对称性: $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$, 因此不是真正的“距离”
- KL 散度衡量了用 Q 来近似 P 时额外需要的编码长度

交叉熵

$$H(P, Q) = - \sum_x P(x) \log Q(x) = H(P) + D_{\text{KL}}(P\|Q)$$

交叉熵 = 真实分布的熵 + KL 散度。当 P 固定时, 最小化交叉熵等价于最小化 KL 散度。因此, 在分类任务中, 最小化交叉熵损失等价于让模型输出分布尽可能接近真实分布——这正是逻辑回归和 softmax 分类器的数学基础。

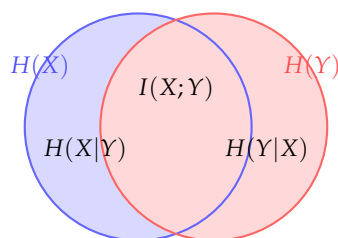
3.4.3 互信息

互信息

两个随机变量 X 和 Y 的互信息衡量它们共享的信息量:

$$I(X; Y) = D_{\text{KL}}(P(X, Y) \| P(X)P(Y)) = \sum_{x, y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)}$$

互信息是非负的, 且 $I(X; Y) = 0$ 当且仅当 X 和 Y 独立。在深度学习中, 互信息用于表示学习中的信息瓶颈理论, 以及 GAN 中的互信息最大化。



$$H(X, Y) = H(X) + H(Y) - I(X; Y)$$

图 3.4: 熵与互信息关系的韦恩图。 $H(X, Y)$ 是联合熵, $I(X; Y)$ 是互信息 (交集), $H(X | Y)$ 和 $H(Y | X)$ 是条件熵。

Python 中的信息论计算

```

1 import numpy as np
2
3 def entropy(probs):
4     """计算离散分布的熵"""
5     probs = np.asarray(probs)
6     probs = probs[probs > 0] # 排除零概率避免log(0)
7     return -np.sum(probs * np.log2(probs))
8
9 def kl_divergence(p, q):
10    """KL散度 D_KL(p||q)"""
11    p = np.asarray(p); q = np.asarray(q)
12    mask = p > 0
13    return np.sum(p[mask] * np.log2(p[mask] / q[mask]))
14
15 def cross_entropy(p, q):
16    """交叉熵 H(p, q)"""
17    p = np.asarray(p); q = np.asarray(q)
18    mask = p > 0
19    return -np.sum(p[mask] * np.log2(q[mask]))
20
21 # 示例
22 p = np.array([0.1, 0.2, 0.7]) # 真实分布
23 q1 = np.array([0.1, 0.2, 0.7]) # 完美预测
24 q2 = np.array([0.3, 0.3, 0.4]) # 不准确预测
25
26 print(f"H(p) = {entropy(p):.4f}")
27 print(f"D_KL(p||q1) = {kl_divergence(p, q1):.6f} (完美, 应为0)")
28 print(f"D_KL(p||q2) = {kl_divergence(p, q2):.4f}")
29 print(f"H(p, q1) = {cross_entropy(p, q1):.4f}")
30 print(f"H(p, q2) = {cross_entropy(p, q2):.4f}")
31
32 # 正态分布的微分熵
33 sigma = 2.0
34 diff_entropy = 0.5 * np.log(2 * np.pi * np.e * sigma**2)
35 print(f"\nN(0,{sigma**2}) 的微分熵: {diff_entropy:.4f}")
36
37 # 互信息示例
38 # P(X,Y) 联合分布
39 joint = np.array([
40     [0.25, 0.10],
41     [0.15, 0.50]
42 ])
43 px = joint.sum(axis=1)
44 py = joint.sum(axis=0)
45 mutual_info = 0
46 for i in range(2):
47     for j in range(2):
48         if joint[i, j] > 0:
49             mutual_info += joint[i, j] * np.log2(
50                 joint[i, j] / (px[i] * py[j])
51             )
52 print(f"互信息 I(X;Y) = {mutual_info:.4f}")

```

§ 3.5 最大似然估计

3.5.1 似然函数

给定独立同分布的观测数据 $\mathcal{D} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}\}$ ，以及参数为 θ 的模型，似然函数为：

$$L(\theta) = P(\mathcal{D} | \theta) = \prod_{i=1}^m p(\mathbf{x}^{(i)} | \theta)$$

对数似然函数通常更方便优化：

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^m \log p(\mathbf{x}^{(i)} | \theta)$$

3.5.2 最大似然估计

最大似然估计（Maximum Likelihood Estimation, MLE）选择使观测数据出现概率最大的参数：

$$\theta_{\text{MLE}} = \arg \max_{\theta} L(\theta) = \arg \max_{\theta} \sum_{i=1}^m \log p(\mathbf{x}^{(i)} | \theta)$$

MLE 具有优良的渐近性质：当样本量趋于无穷时，MLE 是一致的（收敛到真值）和渐近有效的（达到 Cramér-Rao 下界）。

3.5.3 MLE 与损失函数

许多常用的损失函数可以从 MLE 的角度推导：

- **均方误差 (MSE)**：假设目标值 y 服从以模型输出为均值、固定方差的高斯分布，即 $p(y | \mathbf{x}, \theta) = \mathcal{N}(f(\mathbf{x}; \theta), \sigma^2)$ 。其负对数似然正比于 $\sum_i (y_i - f(\mathbf{x}^{(i)}))^2$ ，即 MSE 损失。
- **交叉熵损失**：假设输出服从类别分布（Categorical distribution）， $p(y | \mathbf{x}, \theta) = \prod_{k=1}^K (\hat{y}_k)^{\mathbf{1}_{y=k}}$ 。负对数似然为 $-\sum_i \log \hat{y}_{y_i}$ ，即交叉熵损失。

因此，最小化 MSE 等价于在高斯噪声假设下的 MLE，最小化交叉熵等价于在类别分布假设下的 MLE。

Python 中的 MLE 示例

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 # 从 N(mu=5, sigma=2) 生成数据
5 np.random.seed(42)
6 true_mu, true_sigma = 5.0, 2.0
7 data = np.random.normal(true_mu, true_sigma, 1000)
8
9 # 负对数似然函数
10 def neg_log_likelihood(params):
11     mu, sigma = params[0], params[1]
12     if sigma <= 0:
13         return np.inf
14     n = len(data)
15     # -log L = n*log(sqrt(2*pi)*sigma) + sum((x-mu)^2)/(2*sigma^2)
16     nll = n * np.log(np.sqrt(2 * np.pi) * sigma)
17     nll += np.sum((data - mu) ** 2) / (2 * sigma ** 2)
18     return nll
19
20 # 优化
21 result = minimize(neg_log_likelihood, x0=[0, 1],
22                  bounds=[(None, None), (1e-6, None)])
23 mu_hat, sigma_hat = result.x
24 print(f"真实参数: mu={true_mu}, sigma={true_sigma}")
25 print(f"MLE估计: mu={mu_hat:.4f}, sigma={sigma_hat:.4f}")
26
27 # 验证: 样本均值和样本标准差应为MLE
28 print(f"\n样本均值: {data.mean():.4f}")
29 print(f"样本标准差 (MLE, 除以n): {data.std(ddof=0):.4f}")
30 print(f"样本标准差 (无偏, 除以n-1): {data.std(ddof=1):.4f}")

```

§3.6 最大后验估计

MLE 只考虑似然，忽略了参数的先验信息。最大后验估计（Maximum A Posteriori, MAP）结合先验分布和似然：

$$\theta_{\text{MAP}} = \arg \max_{\theta} P(\theta | \mathcal{D}) = \arg \max_{\theta} P(\mathcal{D} | \theta) P(\theta)$$

取对数：

$$\theta_{\text{MAP}} = \arg \max_{\theta} [\log P(\mathcal{D} | \theta) + \log P(\theta)]$$

MAP 与正则化的关系：

- 高斯先验 $P(\mathbf{w}) \propto \exp(-\frac{\|\mathbf{w}\|^2}{2\sigma^2})$ 对应 L^2 正则化（权重衰减）
- Laplace 先验 $P(\mathbf{w}) \propto \exp(-\frac{\|\mathbf{w}\|_1}{\sigma})$ 对应 L^1 正则化（稀疏化）

这为正则化提供了贝叶斯解释：正则化项实际上是参数先验分布的对数。

§3.7 随机过程初步

3.7.1 马尔可夫链

马尔可夫链是一类重要的随机过程，满足马尔可夫性质：给定当前状态，未来状态与过去状态条件独立。即：

$$P(X_{t+1} = x_{t+1} \mid X_t = x_t, X_{t-1} = x_{t-1}, \dots, X_1 = x_1) = P(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

马尔可夫链在深度学习中的应用包括：

- MCMC (Markov Chain Monte Carlo) 采样方法
- 强化学习中的马尔可夫决策过程 (MDP)
- RNN/Transformer 中的自回归序列建模

§3.8 本章小结

本章介绍了深度学习所需的概率与统计基础知识：

- 随机变量分为离散和连续两类，分别用 PMF 和 PDF 描述
- 正态分布在深度学习中无处不在，从权重初始化到批归一化
- 贝叶斯定理是连接先验、似然和后验的桥梁，为正则化提供了概率解释
- 期望、方差和协方差是对随机变量进行定量分析的基本工具
- 信息论中的熵、KL 散度和交叉熵是理解分类损失函数的关键
- MLE 和 MAP 将参数估计与损失函数优化统一在概率框架下

扎实的概率统计基础有助于理解深度学习的损失函数设计、生成模型和贝叶斯方法。

线性回归与梯度下降

线性回归是最基础也是最经典的机器学习模型之一。尽管形式简单，它包含了监督学习的核心要素：模型假设、损失函数、优化方法和泛化分析。理解线性回归及其优化算法（梯度下降的各种变体）是学习深度神经网络的必要前提。

§4.1
线性回归模型

4.1.1 模型定义

给定输入特征 $\mathbf{x} \in \mathbb{R}^d$ ，线性回归模型假设输出 y 是输入的线性组合：

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

其中 $\mathbf{w} \in \mathbb{R}^d$ 是权重向量， $b \in \mathbb{R}$ 是偏置项。为简化表示，通常将偏置吸收到权重向量中：令 $\tilde{\mathbf{x}} = [\mathbf{x}; 1]$ （追加常数 1）， $\tilde{\mathbf{w}} = [\mathbf{w}; b]$ 。

对于包含 m 个样本的数据集，设 $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$ （每行一个样本，包含偏置列），目标向量 $\mathbf{y} \in \mathbb{R}^m$ ，则预测向量为：

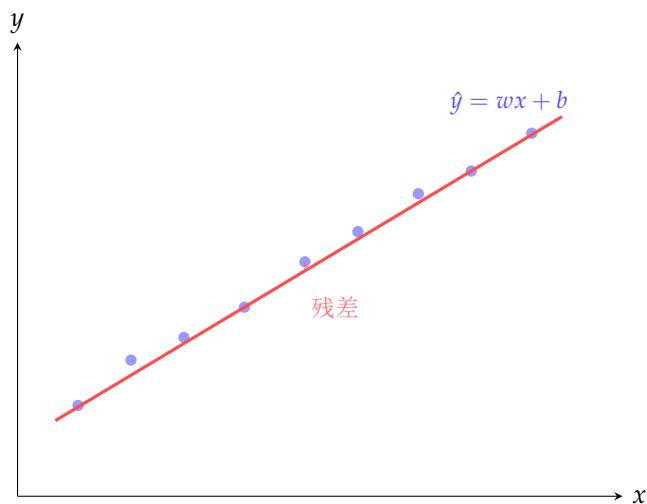
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

4.1.2 损失函数：均方误差

均方误差（Mean Squared Error, MSE）是线性回归最常用的损失函数：

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

系数 $\frac{1}{2}$ 是为了求导时消去指数，简化梯度表达式。



目标：最小化残差平方和

图 4.1: 线性回归的几何解释。红色直线是拟合结果，虚线表示数据点到直线的垂直距离（残差）。

§ 4.2 正规方程

对 $J(\mathbf{w})$ 求导：

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{m} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

令梯度为零得正规方程：

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

当 $\mathbf{X}^T \mathbf{X}$ 可逆时，解析解为：

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

正规方程的计算复杂度为 $O(d^3 + d^2m)$ ，当特征数 d 很大时矩阵求逆不可行。对于深度学习，必须使用迭代方法（梯度下降）。

Python 中的正规方程

```

1 import numpy as np
2
3 np.random.seed(42)
4 m, d = 100, 2
5 X = np.random.randn(m, d)
6 true_w = np.array([3.0, 2.0])
7 true_b = 5.0
8 y = X @ true_w + true_b + 0.5 * np.random.randn(m)
9
10 X_aug = np.column_stack([X, np.ones(m)])
11
12 w_normal = np.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)
13 print(f"正规方程解: w={w_normal[:2]}, b={w_normal[2]:.4f}")
14
15 w_pinv = np.linalg.pinv(X_aug) @ y
16 print(f"伪逆法解: w={w_pinv[:2]}, b={w_pinv[2]:.4f}")
17 print(f"真实值: w={true_w}, b={true_b}")

```

§4.3
梯度下降

4.3.1 批量梯度下降

梯度下降的核心思想：每一步沿负梯度方向移动一小步：

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_t)$$

其中 $\eta > 0$ 是学习率。对于 MSE 损失：

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

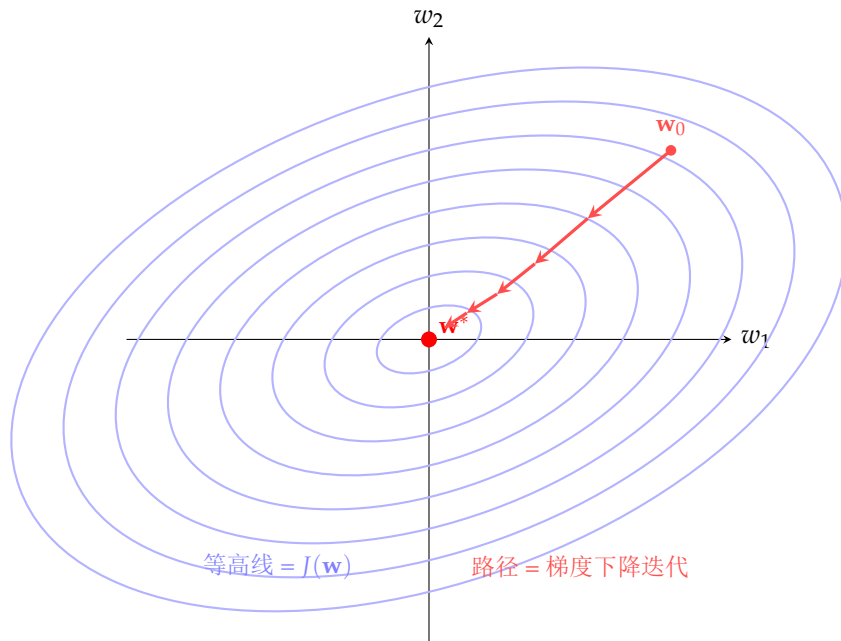


图 4.2: 梯度下降的优化轨迹。每一步沿负梯度方向移动，最终收敛到最小值。

批量梯度下降算法

输入: 训练数据 $\mathbf{X} \in \mathbb{R}^{m \times d}$, 标签 $\mathbf{y} \in \mathbb{R}^m$, 学习率 η , 最大迭代次数 T

初始化: $\mathbf{w}_0$ (通常为零向量)

循环: 对 $t = 0, 1, \dots, T-1$:

- (1). 计算梯度: $\mathbf{g}_t = \frac{1}{m} \mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y})$
- (2). 更新参数: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t$
- (3). 若 $\|\mathbf{g}_t\|_2 < \varepsilon$, 则停止

输出: $\mathbf{w}_T$

4.3.2 随机梯度下降

随机梯度下降 (SGD) 每次更新只使用一个随机样本:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_t; \mathbf{x}^{(i)}, y^{(i)})$$

其中 $\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = (\hat{y}_i - y_i) \mathbf{x}^{(i)}$ 。SGD 的随机噪声有助于逃离鞍点和局部极小点。

4.3.3 小批量随机梯度下降

小批量 SGD 每次使用 B 个样本:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\mathbf{w}} J(\mathbf{w}_t; \mathbf{x}^{(i)}, y^{(i)})$$

其中 $\mathcal{B}_t$ 是第 t 次迭代的随机小批量。小批量 SGD 兼顾了计算效率和梯度估计的方差。

Python 中的梯度下降实现

```

1 import numpy as np
2
3 np.random.seed(42)
4 m, d = 1000, 5
5 X = np.random.randn(m, d)
6 true_w = np.array([1.5, -2.0, 3.0, -0.5, 2.0])
7 y = X @ true_w + 3.0 + 0.5 * np.random.randn(m)
8 X_aug = np.column_stack([X, np.ones(m)])
9
10 def mse_loss(w, X, y):
11     pred = X @ w
12     return 0.5 * np.mean((pred - y) ** 2)
13
14 def mse_gradient(w, X, y):
15     pred = X @ w
16     return X.T @ (pred - y) / len(y)
17
18 # 批量梯度下降
19 w_bgd = np.zeros(d + 1)
20 lr = 0.01
21 for t in range(500):
22     grad = mse_gradient(w_bgd, X_aug, y)
23     w_bgd -= lr * grad
24 print(f"Batch GD - 最终损失: {mse_loss(w_bgd, X_aug, y):.6f}")
25
26 # 小批量SGD
27 w_sgd = np.zeros(d + 1)
28 batch_size = 32
29 for t in range(500):
30     idx = np.random.choice(m, batch_size, replace=False)
31     Xb, yb = X_aug[idx], y[idx]
32     grad = mse_gradient(w_sgd, Xb, yb)
33     w_sgd -= lr * grad
34 print(f"Mini-batch SGD - 最终损失: {mse_loss(w_sgd, X_aug, y):.6f}")

```

§4.4

动量与自适应优化器

4.4.1 带动量的 SGD

动量方法通过累积历史梯度加速收敛：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla J(\mathbf{w}_t) \quad (4.1)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_t \quad (4.2)$$

其中 $\beta \in [0, 1)$ 是动量系数（通常 0.9）。动量在梯度方向一致时加速，在梯度剧烈变化时保持稳定。

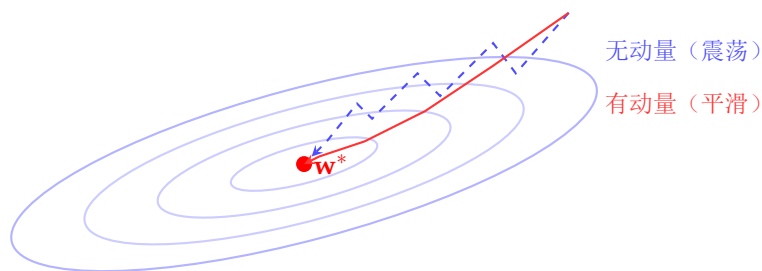


图 4.3: 动量方法在峡谷形损失函数上的效果对比。有动量（红色）的路径明显比无动量（蓝色虚线）更平滑高效。

4.4.2 Nesterov 动量

Nesterov 动量先根据当前速度向前看一步，再在该前瞻点上计算梯度：

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \nabla J(\mathbf{w}_t - \beta \mathbf{v}_{t-1})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_t$$

这种前瞻策略使得 Nesterov 动量在接近最优解时能提前减速。

4.4.3 AdaGrad

AdaGrad 为每个参数自适应调整学习率：

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \nabla J(\mathbf{w}_t) \odot \nabla J(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \varepsilon}} \odot \nabla J(\mathbf{w}_t)$$

AdaGrad 的问题： $\mathbf{s}_t$ 单调递增，学习率持续下降。

4.4.4 RMSProp

RMSProp 用指数移动平均代替累计和：

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \nabla J(\mathbf{w}_t) \odot \nabla J(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \varepsilon}} \odot \nabla J(\mathbf{w}_t)$$

4.4.5 Adam

Adam 结合了动量和 RMSProp，是目前最常用的优化器：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla J(\mathbf{w}_t) \quad (4.3)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \nabla J(\mathbf{w}_t)^2 \quad (4.4)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad (4.5)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (4.6)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} \quad (4.7)$$

默认超参数: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ 。

Python 中的优化器实现

```

1 import numpy as np
2
3 class Momentum:
4     def __init__(self, lr=0.01, beta=0.9):
5         self.lr, self.beta = lr, beta
6         self.v = None
7
8     def update(self, w, grad):
9         if self.v is None:
10             self.v = np.zeros_like(w)
11         self.v = self.beta * self.v + self.lr * grad
12         return w - self.v
13
14 class Adam:
15     def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
16         self.lr, self.beta1, self.beta2, self.eps = lr, beta1, beta2, eps
17         self.m, self.v, self.t = None, None, 0
18
19     def update(self, w, grad):
20         if self.m is None:
21             self.m = np.zeros_like(w)
22             self.v = np.zeros_like(w)
23         self.t += 1
24         self.m = self.beta1 * self.m + (1 - self.beta1) * grad
25         self.v = self.beta2 * self.v + (1 - self.beta2) * (grad ** 2)
26         m_hat = self.m / (1 - self.beta1 ** self.t)
27         v_hat = self.v / (1 - self.beta2 ** self.t)
28         return w - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
29
30 # 测试
31 np.random.seed(0)
32 X = np.random.randn(200, 3)
33 y = X @ np.array([2.0, -1.0, 0.5]) + 1.0
34
35 for name, opt in [('SGD', None), ('Momentum', Momentum(lr=0.05)),
36                  ('Adam', Adam(lr=0.05))]:
37     w_opt = np.zeros(3)
38     for i in range(200):
39         grad = X.T @ (X @ w_opt - y) / len(y)
40         if opt is None:
41             w_opt -= 0.05 * grad
42         else:
43             w_opt = opt.update(w_opt, grad)
44     loss = 0.5 * np.mean((X @ w_opt - y) ** 2)
45     print(f"{name:10s} - 损失: {loss:.6f}, w: {w_opt.round(3)}")

```

§4.5 学习率调度

- 阶梯衰减：每隔固定 epochs 将学习率乘以衰减因子（如 $\times 0.1$ ）
- 指数衰减： $\eta_t = \eta_0 \cdot \gamma^t$
- 余弦退火： $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{t}{T}\pi))$
- 热身（Warmup）：训练初期线性增加学习率，对大型 Transformer 训练至关重要

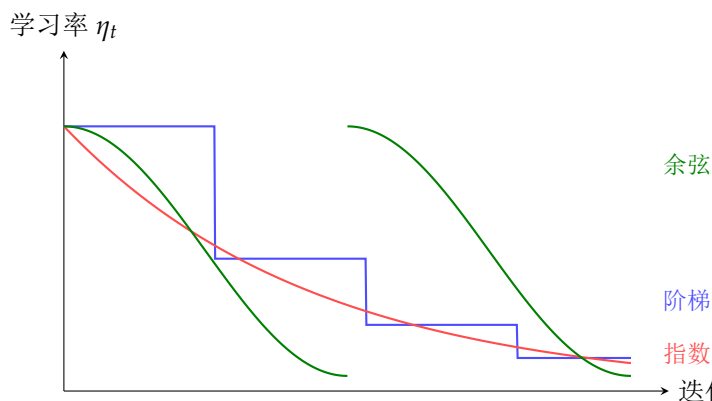


图 4.4: 不同的学习率调度策略对比。阶梯衰减（蓝）分段下降，指数衰减（红）平滑递减，余弦退火（绿）在周期内平滑变化。

§4.6 正则化

4.6.1 L^2 正则化（Ridge 回归/权重衰减）

在 MSE 损失中加入权重向量的 L^2 范数惩罚：

$$J(\mathbf{w}) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

正则化参数 $\lambda > 0$ 控制惩罚强度。梯度：

$$\nabla_{\mathbf{w}} J = \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$$

Ridge 回归的解析解：

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + m\lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

加入 $\lambda \mathbf{I}$ 后矩阵变得可逆（即使原矩阵不可逆），解决了多重共线性问题。 L^2 正则化倾向于让所有权重都接近零但不为零。

4.6.2 L^1 正则化（Lasso 回归）

$$J(\mathbf{w}) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

L^1 正则化倾向于产生稀疏解——许多权重变为精确的零，从而实现特征选择。其梯度涉及次梯度（因为绝对值在原点不可导）：

$$\frac{\partial \|\mathbf{w}\|_1}{\partial w_j} = \text{sign}(w_j)$$

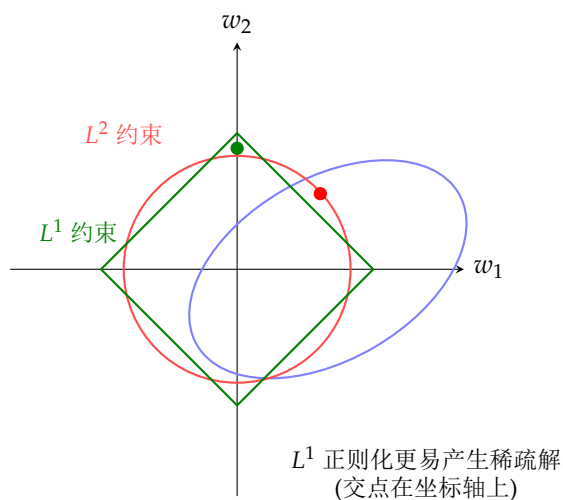


图 4.5: L^1 与 L^2 正则化的几何解释。MSE 的等高线（椭圆）和正则化约束区域（菱形/ L^1 ，圆/ L^2 ）相交于最优解。 L^1 正则化因菱形尖点在坐标轴上，容易产生稀疏解。

4.6.3 弹性网络

弹性网络结合 L^1 和 L^2 正则化：

$$J(\mathbf{w}) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_1 \|\mathbf{w}\|_1 + \frac{\lambda_2}{2} \|\mathbf{w}\|_2^2$$

它兼具 L^1 的稀疏性和 L^2 的稳定性，在特征高度相关时表现优于纯 Lasso。

Python 中的正则化对比

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression, Ridge, Lasso
3
4 np.random.seed(42)
5 m, d = 50, 100 # d >> m: 高维情况
6 X = np.random.randn(m, d)
7 true_w = np.zeros(d)
8 true_w[:5] = [4, 3, -2, 5, 1] # 只有5个非零特征
9 y = X @ true_w + 0.3 * np.random.randn(m)
10
11 lr = LinearRegression()
12 ridge = Ridge(alpha=1.0)
13 lasso = Lasso(alpha=0.5)
14
15 for name, model in [('无正则', lr), ('L2(Ridge)', ridge), ('L1(Lasso)', lasso)]:
16     model.fit(X, y)
17     w_pred = model.coef_
18     print(f"{name}: 非零特征数 = {np.sum(np.abs(w_pred) > 1e-4)}")
19     print(f"   w[:5] = {w_pred[:5].round(2)}")

```

§4.7
多项式回归

线性回归可以扩展到多项式回归：将原始特征 $\mathbf{x}$ 映射到更高维的多项式特征空间。例如，一元二次回归：

$$\hat{y} = w_0 + w_1x + w_2x^2$$

设 $\phi(\mathbf{x}) = [1, x, x^2]^\top$ ，则 $\hat{y} = \mathbf{w}^\top \phi(\mathbf{x})$ ，仍是一个关于 $\mathbf{w}$ 的线性模型。多项式回归可以拟合非线性关系，但阶数过高会过拟合。

§4.8
偏差-方差权衡

模型的泛化误差可以分解为三个部分：

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{(\text{Bias}[\hat{f}])^2}_{\text{偏差}^2} + \underbrace{\text{Var}[\hat{f}]}_{\text{方差}} + \underbrace{\sigma^2}_{\text{不可约误差}}$$

其中：

- **偏差**：模型预测的期望与真实值的差异。高偏差意味着欠拟合（模型太简单）。
- **方差**：对不同训练集，模型预测的变化程度。高方差意味着过拟合（模型太复杂）。

- **不可约误差**：数据本身的噪声，无法通过任何模型消除。

正则化参数 λ 控制偏差-方差权衡： λ 越大，模型越简单（高偏差，低方差）； λ 越小，模型越复杂（低偏差，高方差）。

§4.9 本章小结

本章介绍了线性回归与梯度下降的核心内容：

- 线性回归是最基础的监督学习模型，MSE 是其标准损失函数
- 正规方程提供了解析解，但特征数大时不适用
- 梯度下降系列算法（SGD、动量、AdaGrad、RMSProp、Adam）是深度学习优化的基石
- 学习率调度策略对训练效果有显著影响
- 正则化（ L^1 、 L^2 、弹性网络）控制模型复杂度，防止过拟合
- 偏差-方差权衡揭示了模型复杂度和泛化性能之间的根本矛盾

逻辑回归与分类

分类问题是机器学习中与回归并列的另一大类监督学习任务。逻辑回归 (Logistic Regression) 是分类问题中最基础且应用广泛的模型之一。尽管名称中含有“回归”，它实际上是一个分类模型。本章从逻辑回归的二分类出发，逐步推广到多分类的 softmax 回归，并介绍分类模型的评估指标。

§5.1 逻辑回归模型

5.1.1 从线性回归到逻辑回归

线性回归输出实数值，不能直接用于分类。逻辑回归在线性组合 $\mathbf{w}^\top \mathbf{x} + b$ 的基础上，使用 sigmoid 函数将输出映射到 $(0, 1)$ 区间：

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

其中 $\sigma(z) = \frac{1}{1+e^{-z}}$ 是 sigmoid 函数（也称为 logistic 函数），具有以下性质：

- $\sigma(z) \in (0, 1)$ ，可以将输出解释为概率
- $\sigma(0) = 0.5$
- $\lim_{z \rightarrow \infty} \sigma(z) = 1$, $\lim_{z \rightarrow -\infty} \sigma(z) = 0$
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ （便于计算梯度）
- $\sigma(-z) = 1 - \sigma(z)$

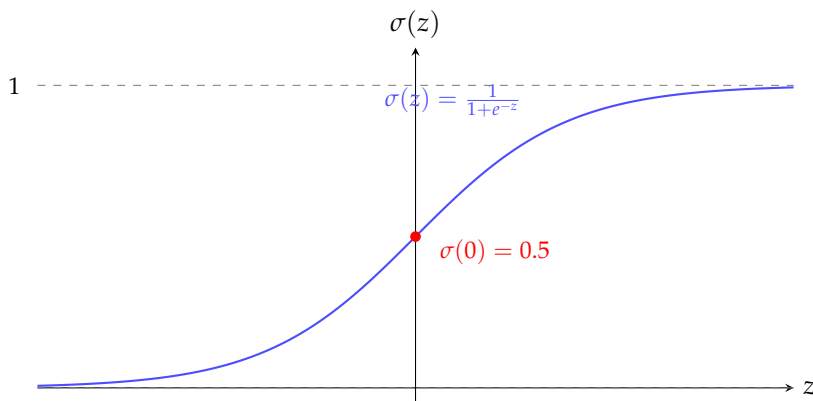


图 5.1: Sigmoid 函数的图像。 $z = 0$ 时输出 0.5， $z \rightarrow \infty$ 时输出趋近于 1， $z \rightarrow -\infty$ 时输出趋近于 0。

5.1.2 决策边界

将 $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ 的输出解释为 $P(y = 1 | \mathbf{x})$ （即样本属于正类的概率）。分类决策规则为：

若 $\hat{y} \geq 0.5$ ，则预测为正类；否则预测为负类

由于 $\sigma(z) \geq 0.5 \iff z \geq 0$ ，决策规则简化为：

若 $\mathbf{w}^\top \mathbf{x} + b \geq 0$ ，则预测为正类

因此，决策边界是超平面 $\mathbf{w}^\top \mathbf{x} + b = 0$ ，逻辑回归本质上是线性分类器。

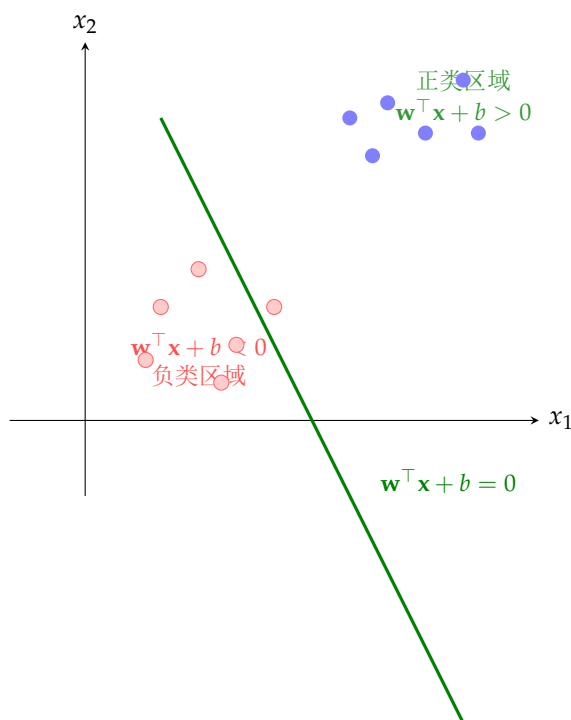


图 5.2: 逻辑回归的线性决策边界。决策边界是超平面 $\mathbf{w}^\top \mathbf{x} + b = 0$ ，将特征空间划分为正类区域和负类区域。

§ 5.2 交叉熵损失

5.2.1 概率解释

设 $y \in \{0, 1\}$ 为二分类标签。逻辑回归模型的输出可解释为条件概率：

$$P(y = 1 | \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}), \quad P(y = 0 | \mathbf{x}; \mathbf{w}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x})$$

这等价于 Bernoulli 分布：

$$P(y | \mathbf{x}; \mathbf{w}) = \hat{y}^y \cdot (1 - \hat{y})^{1-y}$$

其中 $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$ 。

5.2.2 最大似然推导

给定独立同分布的数据 $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ ，似然函数为：

$$L(\mathbf{w}) = \prod_{i=1}^m \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

取负对数似然得到交叉熵损失（Binary Cross-Entropy, BCE）：

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

交叉熵损失的直观理解：当 $y_i = 1$ 时，损失为 $-\log \hat{y}_i$ 。若模型输出 $\hat{y}_i \approx 1$ ，损失接近 0；若 $\hat{y}_i \approx 0$ ，损失趋于 $+\infty$ （严重惩罚错误的自信预测）。

交叉熵损失的梯度：

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \mathbf{x}^{(i)} = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y})$$

这个梯度形式与线性回归的梯度 $\frac{1}{m} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$ 惊人地相似——唯一的区别在于预测值的计算方式（sigmoid vs 线性）。

Python 中的逻辑回归从零实现

```
1 import numpy as np
2
3 np.random.seed(42)
4 m, d = 200, 3
5 X = np.random.randn(m, d)
6 true_w = np.array([2.0, -1.0, 1.5])
7 logits = X @ true_w
8 probs = 1 / (1 + np.exp(-logits))
9 y = (np.random.random(m) < probs).astype(float)
10
11 # Sigmoid函数
12 def sigmoid(z):
13     return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
14
15 # 交叉熵损失
16 def bce_loss(w, X, y):
17     z = X @ w; y_hat = sigmoid(z)
18     eps = 1e-15
19     return -np.mean(y * np.log(y_hat + eps) + (1 - y) * np.log(1 - y_hat + eps))
20
21 # 交叉熵梯度
22 def bce_gradient(w, X, y):
23     z = X @ w; y_hat = sigmoid(z)
24     return X.T @ (y_hat - y) / len(y)
25
26 # 梯度下降
27 w = np.zeros(d)
28 lr = 0.1
29 for t in range(1000):
30     grad = bce_gradient(w, X, y)
31     w -= lr * grad
32     if t % 200 == 0:
33         print(f"Epoch {t:4d}, Loss: {bce_loss(w, X, y):.6f}")
34
35 print(f"\n真实w: {true_w}")
36 print(f"学习w: {w.round(3)}")
37
38 # 准确率
39 y_pred = (sigmoid(X @ w) >= 0.5).astype(float)
40 accuracy = np.mean(y_pred == y)
41 print(f"训练准确率: {accuracy:.2%}")
```

§5.3 多分类：Softmax 回归

5.3.1 模型定义

对于 K 类分类问题 ($K > 2$)，Softmax 回归（也称为多项逻辑回归）将逻辑回归推广到多类别。模型为每个类别学习一个权重向量 $\mathbf{w}_k \in \mathbb{R}^d$ ，输入 $\mathbf{x}$ 属于第 k 类的概率为：

$$P(y = k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

其中 $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K] \in \mathbb{R}^{d \times K}$ 是权重矩阵。Softmax 函数 $\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ 将 K 维实数向量 $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$ 映射为概率分布。

Softmax 的性质：

- 输出为概率分布： $\sum_{k=1}^K \hat{y}_k = 1$, $\hat{y}_k \in [0, 1]$
- 平移不变性： $\text{softmax}(\mathbf{z} + \mathbf{c}) = \text{softmax}(\mathbf{z})$ ($\mathbf{c}$ 所有分量相等)
- 数值稳定性：实践中减去 $\max(z_k)$ 防止指数溢出
- 当 $K = 2$ 时，Softmax 回归等价于逻辑回归（参数过参数化）

5.3.2 交叉熵损失

对于多分类问题，使用分类交叉熵损失（Categorical Cross-Entropy）。设真实标签的 one-hot 编码为 $\mathbf{y}^{(i)} \in \{0, 1\}^K$ ，预测概率为 $\hat{\mathbf{y}}^{(i)}$ ：

$$J(\mathbf{W}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$$

由于 $\mathbf{y}^{(i)}$ 是 one-hot 向量，上式可简化为：

$$J(\mathbf{W}) = -\frac{1}{m} \sum_{i=1}^m \log \hat{y}_{y^{(i)}}^{(i)}$$

其中 $y^{(i)}$ 是第 i 个样本的真实类别索引。这就是深度学习框架中常用的交叉熵损失。

Python 中的 Softmax 回归实现

```

1 import numpy as np
2
3 np.random.seed(42)
4 m, d, K = 500, 4, 3 # 500样本, 4特征, 3类别
5
6 X = np.random.randn(m, d)
7 W_true = np.random.randn(d, K)
8 logits_true = X @ W_true
9 y = np.argmax(logits_true + 0.3 * np.random.randn(m, K), axis=1)
10
11 def softmax(z):
12     z_max = np.max(z, axis=1, keepdims=True)
13     exp_z = np.exp(z - z_max)
14     return exp_z / np.sum(exp_z, axis=1, keepdims=True)
15
16 def cross_entropy_loss(W, X, y, K):
17     z = X @ W; probs = softmax(z)
18     m = len(y)
19     eps = 1e-15
20     log_likelihood = -np.log(probs[np.arange(m), y] + eps)
21     return np.mean(log_likelihood)
22
23 def cross_entropy_gradient(W, X, y, K):
24     z = X @ W; probs = softmax(z)
25     m = len(y)
26     y_onehot = np.eye(K)[y]
27     return X.T @ (probs - y_onehot) / m
28
29 # 训练
30 W = np.random.randn(d, K) * 0.1
31 lr = 0.5
32 for t in range(500):
33     grad = cross_entropy_gradient(W, X, y, K)
34     W -= lr * grad
35     if t % 100 == 0:
36         loss = cross_entropy_loss(W, X, y, K)
37         acc = np.mean(np.argmax(X @ W, axis=1) == y)
38         print(f"Epoch {t:3d}, Loss: {loss:.4f}, Acc: {acc:.2%}")
39
40 print(f"\n最终准确率: {np.mean(np.argmax(X @ W, axis=1) == y):.2%}")

```

§5.4 分类评估指标

5.4.1 混淆矩阵

混淆矩阵是分类模型预测结果的 $K \times K$ 总结：

对于二分类：

- **TP (True Positive)**: 正类样本被正确预测为正类
- **TN (True Negative)**: 负类样本被正确预测为负类
- **FP (False Positive)**: 负类样本被错误预测为正类（假阳性）
- **FN (False Negative)**: 正类样本被错误预测为负类（假阴性）

	预测为正	预测为负
实际为正	TP 真正例	FN 假阴性
实际为负	FP 假阳性	TN 真阴性

图 5.3: 二分类混淆矩阵。对角线上是正确分类的样本（TP 和 TN），反对角线上是错误分类的样本（FP 和 FN）。

5.4.2 准确率、精确率、召回率与 F1 分数

准确率是最直观的指标：

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

在类别不平衡时（如正类仅占 1%），准确率不再可靠——总是预测负类即可获得 99% 准确率。

精确率 (Precision) 衡量预测为正的样本中有多少是真正的正类：

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

召回率 (Recall) 衡量真正的正类中有多少被正确识别：

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1 分数 是精确率和召回率的调和平均：

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 分数与调和平均

调和平均赋予较小值更高的权重：若 Precision 为 0.1，Recall 为 0.9，则算术平均为 0.5，但 F1 仅为 0.18。这确保 F1 只有在精确率和召回率都高时才高。

5.4.3 ROC 曲线与 AUC

逻辑回归输出的概率可以设置不同的阈值来调整分类决策。ROC 曲线（Receiver Operating Characteristic curve）描述了在不同阈值下，真阳性率（ $TPR=Recall$ ）和假阳性率（ $FPR=FP/(FP+TN)$ ）的权衡关系。

- **AUC (Area Under Curve)**: ROC 曲线下的面积，取值范围 $[0,1]$ 。AUC 不依赖于阈值选择，是衡量模型排序能力的指标。
- $AUC = 1$: 完美分类器
- $AUC = 0.5$: 随机猜测
- $AUC < 0.5$: 比随机猜测还差

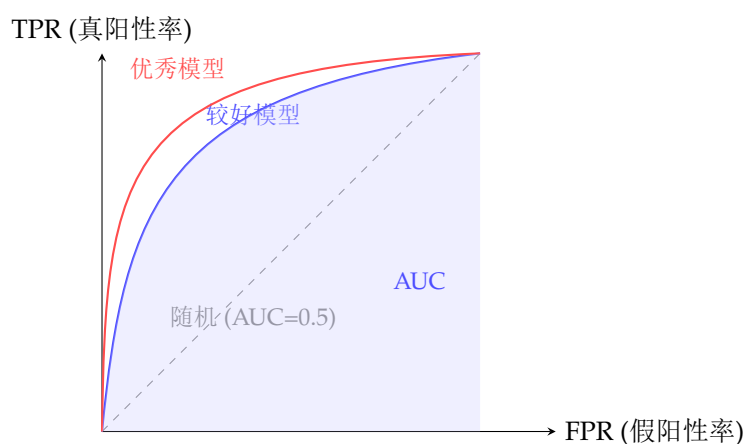


图 5.4: ROC 曲线示意图。曲线越靠近左上角，模型性能越好。AUC 是曲线下的面积。

§ 5.5

多分类的评估指标

对于多分类问题，可以将评估指标推广：

- **宏平均 (Macro-averaging)**: 对每个类别独立计算指标，然后取平均。每个类别权重相等，适合关注小类。
- **微平均 (Micro-averaging)**: 合并所有类别的 TP/FP/FN/TN，然后计算指标。每个样本权重相等，适合关注大类。
- **加权平均 (Weighted-averaging)**: 宏平均但按每个类别的样本数加权。

§ 5.6

类别不平衡问题

在实际应用中，类别不平衡（如欺诈检测中欺诈交易远少于正常交易）是常见挑战：

- **重采样**: 过采样少数类（如 SMOTE 算法合成新样本）或欠采样多数类

- **类别权重**：在损失函数中为少数类赋予更高权重
- **Focal Loss**： $L = -\alpha(1 - \hat{y})^\gamma y \log \hat{y}$ ，减少易分类样本的权重，使模型专注于难分类样本
- **阈值移动**：调整分类决策阈值来平衡精确率和召回率

Python 中的分类评估指标

```

1 import numpy as np
2 from sklearn.metrics import (confusion_matrix, accuracy_score,
3     precision_score, recall_score, f1_score,
4     roc_auc_score, classification_report)
5
6 np.random.seed(42)
7 n = 500
8 # 模拟预测：正类概率
9 y_true = (np.random.random(n) > 0.7).astype(int)
10 y_score = y_true * 0.7 + (1 - y_true) * 0.3 + 0.2 * np.random.randn(n)
11 y_score = np.clip(y_score, 0.01, 0.99)
12 y_pred = (y_score > 0.5).astype(int)
13
14 print("混淆矩阵:")
15 print(confusion_matrix(y_true, y_pred))
16
17 print(f"\n准确率: {accuracy_score(y_true, y_pred):.3f}")
18 print(f"精确率: {precision_score(y_true, y_pred):.3f}")
19 print(f"召回率: {recall_score(y_true, y_pred):.3f}")
20 print(f"F1分数: {f1_score(y_true, y_pred):.3f}")
21 print(f"AUC-ROC: {roc_auc_score(y_true, y_score):.3f}")
22
23 print("\n详细报告:")
24 print(classification_report(y_true, y_pred,
25     target_names=['负类', '正类']))

```

§5.7

逻辑回归与深度学习

逻辑回归可以视为单层神经网络（不含隐藏层）使用 sigmoid 激活函数。它在深度学习中扮演两个重要角色：

- **最后一层（二分类）**：在神经网络末端使用 sigmoid 激活 + BCE 损失进行二分类。
- **最后一层（多分类）**：在神经网络末端使用 softmax 激活 + 交叉熵损失进行多分类。
- **基础构建块**：理解逻辑回归的损失函数和梯度推导，有助于理解更复杂网络的反向传播。

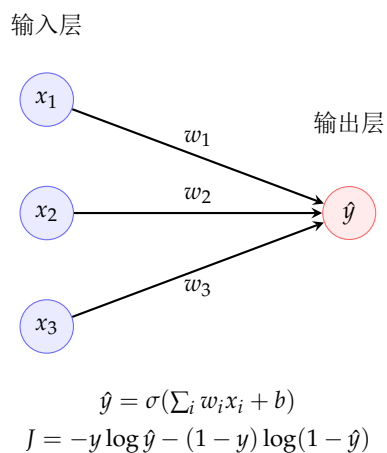


图 5.5: 逻辑回归作为单层神经网络。输入特征通过加权求和后经 sigmoid 激活，输出分类概率。

§ 5.8 本章小结

本章介绍了逻辑回归与分类的核心内容：

- 逻辑回归使用 sigmoid 函数将线性输出映射为概率
- 交叉熵损失从最大似然估计自然导出，是分类任务的标准损失
- Softmax 回归将逻辑回归推广到多分类
- 混淆矩阵、精确率、召回率、F1 和 ROC/AUC 是分类模型的核心评估指标
- 类别不平衡问题需要特殊处理方法
- 逻辑回归是神经网络的基本构建块

PyTorch 入门

PyTorch 是目前最流行的深度学习框架之一，以其动态计算图、Pythonic 的编程风格和强大的 GPU 加速能力而受到研究者广泛喜爱。本章从零开始介绍 PyTorch 的核心概念：张量操作、自动求导、神经网络模块、优化器、数据加载，以及完整的训练流程。

§6.1 张量基础

6.1.1 创建张量

张量（Tensor）是 PyTorch 中的核心数据结构，类似于 NumPy 的 ndarray，但额外支持 GPU 加速和自动微分。

张量创建与基本操作

```
1 import torch
2
3 # 创建张量
4 a = torch.tensor([1, 2, 3])
5 b = torch.zeros(2, 3)
6 c = torch.ones(3, 4)
7 d = torch.randn(3, 3)      # 标准正态分布
8 e = torch.arange(0, 10, 2) # 0到10，步长2
9 f = torch.eye(3)           # 3x3单位矩阵
10
11 print(f"a: {a}, 形状: {a.shape}")
12 print(f"b:\n{b}")
13 print(f"d:\n{d}")
14
15 # 从NumPy转换
16 import numpy as np
17 np_arr = np.array([[1, 2], [3, 4]])
18 torch_arr = torch.from_numpy(np_arr)
19 back_to_np = torch_arr.numpy()
```


6.1.2 张量属性与运算

张量属性与运算

```
1 import torch
2
3 x = torch.randn(2, 3, 4)
4 print(f"形状: {x.shape}")
5 print(f"维度数: {x.ndim}")
6 print(f"元素总数: {x.numel()}")
7 print(f"数据类型: {x.dtype}")
8 print(f"所在设备: {x.device}")
9
10 # 基本运算
11 a = torch.tensor([1., 2., 3.])
12 b = torch.tensor([4., 5., 6.])
13 print(f"加法: {a + b}")
14 print(f"乘法(逐元素): {a * b}")
15 print(f"点积: {torch.dot(a, b)}")
16
17 # 矩阵运算
18 A = torch.randn(3, 4)
19 B = torch.randn(4, 2)
20 C = A @ B # 矩阵乘法
21 print(f"矩阵乘法结果形状: {C.shape}")
22
23 # 维度操作
24 x = torch.randn(2, 3, 4)
25 print(f"reshape: {x.reshape(6, 4).shape}")
26 print(f"转置: {x.transpose(0, 2).shape}")
27 print(f"permute: {x.permute(2, 1, 0).shape}")
28 print(f"unsqueeze: {x.unsqueeze(0).shape}") # 增加维度
29 print(f"squeeze: {x.unsqueeze(0).squeeze(0).shape}")
30
31 # 拼接
32 a = torch.randn(2, 3); b = torch.randn(2, 3)
33 print(f"cat(dim=0): {torch.cat([a, b], dim=0).shape}")
34 print(f"cat(dim=1): {torch.cat([a, b], dim=1).shape}")
35 print(f"stack: {torch.stack([a, b], dim=0).shape}")
```

6.1.3 索引与切片

张量的索引和切片与 NumPy 类似，支持高级索引。关键概念：视图（**view**）与副本（**copy**）——大多数切片操作返回视图（共享底层数据），使用 `.clone()` 创建独立副本。

张量索引与切片

```
1 import torch
2
3 x = torch.arange(24).reshape(2, 3, 4)
4 print(f"张量形状: {x.shape}")
5
6 # 基本索引
7 print(f"x[0]:\n{x[0]}")          # 第一个3x4矩阵
8 print(f"x[0, 1]: {x[0, 1]}")    # 第一矩阵的第二行
9 print(f"x[:, :2, :]\n{x[:, :2, :]}") # 所有批次的前两行
10
11 # 花式索引
12 indices = torch.tensor([0, 2])
13 print(f"x[:, :, [0,2]]:\n{x[:, :, indices]}")
14
15 # 布尔索引
16 mask = x > 10
17 print(f"大于10的元素: {x[mask]}")
18
19 # view与副本
20 y = x.view(6, 4) # 视图（共享数据）
21 z = x.clone()    # 独立副本
22 y[0, 0] = 999
23 print(f"修改view后原始张量x[0,0,0]: {x[0,0,0]}") # 已被修改！
```

§6.2
GPU 加速

PyTorch 的一大优势是无缝的 GPU 支持。张量可以在 CPU 和 GPU 之间轻松移动：

GPU 操作

```
1 import torch
2
3 # 检查GPU可用性
4 print(f"CUDA可用: {torch.cuda.is_available()}")
5 if torch.cuda.is_available():
6     print(f"GPU数量: {torch.cuda.device_count()}")
7     print(f"当前GPU: {torch.cuda.get_device_name(0)}")
8
9 # 在指定设备上创建张量
10 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11
12 x_cpu = torch.randn(1000, 1000)
13 x_gpu = x_cpu.to(device)      # 移动到GPU
14 x_back = x_gpu.cpu()         # 移回CPU
15
16 # 直接在目标设备上创建
17 y = torch.randn(1000, 1000, device=device)
18
19 # GPU加速矩阵乘法
20 import time
21 A = torch.randn(5000, 5000, device=device)
22 B = torch.randn(5000, 5000, device=device)
23
24 if device.type == 'cuda':
25     torch.cuda.synchronize()
26 t0 = time.time()
27 C = A @ B
28 if device.type == 'cuda':
29     torch.cuda.synchronize()
30 print(f"GPU 矩阵乘法时间: {time.time()-t0:.3f}s")
31
32 # 内存管理
33 print(f"GPU内存分配: {torch.cuda.memory_allocated()/1e9:.2f} GB")
34 print(f"GPU内存缓存: {torch.cuda.memory_reserved()/1e9:.2f} GB")
35 torch.cuda.empty_cache()     # 清理缓存
```

§ 6.3
自动求导

自动求导（Autograd）是 PyTorch 的核心功能，它自动计算计算图中任意参数的梯度。

6.3.1 计算图

PyTorch 在每次前向传播时动态构建计算图。设置 `requires_grad=True` 的张量会记录其上的所有操作，调用 `.backward()` 后自动计算梯度。

自动求导基础

```
1 import torch
2
3 # 需要梯度的张量
4 x = torch.tensor([2.0, 3.0], requires_grad=True)
5 w = torch.tensor([1.0, -1.0], requires_grad=True)
6 b = torch.tensor([0.5], requires_grad=True)
7
8 # 前向计算:  $y = w \cdot x + b$ 
9 y = torch.dot(w, x) + b
10
11 # 反向传播
12 y.backward()
13
14 print(f"y = {y.item()}")
15 print(f"dy/dw = {w.grad}") # 应为 x = [2, 3]
16 print(f"dy/dx = {x.grad}") # 应为 w = [1, -1]
17 print(f"dy/db = {b.grad}") # 应为 1
18
19 # 多次反向传播需要 retain_graph=True
20 x = torch.tensor(3.0, requires_grad=True)
21 y = x ** 2
22 y.backward(retain_graph=True)
23 print(f"dy/dx = {x.grad}") #  $2 \cdot x = 6$ 
24
25 # 高阶导数
26 x = torch.tensor(2.0, requires_grad=True)
27 y = x ** 3
28 grad1 = torch.autograd.grad(y, x, create_graph=True)[0]
29 print(f"一阶导数 ( $3x^2$  at  $x=2$ ): {grad1.item()}") # 12
30 grad2 = torch.autograd.grad(grad1, x)[0]
31 print(f"二阶导数 ( $6x$  at  $x=2$ ): {grad2.item()}") # 12
```

6.3.2 梯度在神经网络中的应用

使用 autograd 训练简单模型

```
1 import torch
2
3 torch.manual_seed(42)
4 m, d = 200, 5
5 X = torch.randn(m, d)
6 true_w = torch.tensor([2.0, -1.0, 3.0, -0.5, 1.5])
7 y = X @ true_w + 2.0 + 0.3 * torch.randn(m)
8
9 w = torch.randn(d, requires_grad=True)
10 b = torch.zeros(1, requires_grad=True)
11 lr = 0.05
12
13 for epoch in range(500):
14     # 前向
15     y_pred = X @ w + b
16     loss = 0.5 * torch.mean((y_pred - y) ** 2)
17
18     # 反向
19     loss.backward()
20
21     # 参数更新（手动，之后会介绍优化器）
22     with torch.no_grad():
23         w -= lr * w.grad
24         b -= lr * b.grad
25         w.grad.zero_()
26         b.grad.zero_()
27
28     if epoch % 100 == 0:
29         print(f"Epoch {epoch:3d}, Loss: {loss.item():.6f}")
30
31 print(f"\n真实w: {true_w}")
32 print(f"学习w: {w.data}")
33 print(f"学习b: {b.item():.4f}")
```

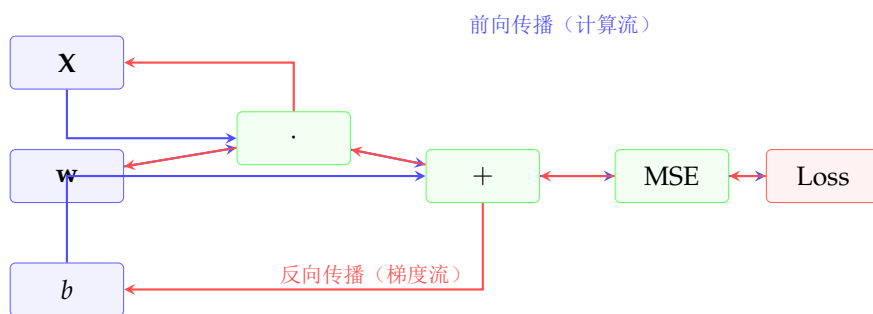


图 6.1: PyTorch 自动求导的计算图。蓝色实线是前向传播，红色虚线是反向传播的梯度流。每个节点记录局部导数，通过链式法则自动合成完整梯度。

§ 6.4

nn.Module: 构建神经网络

`torch.nn.Module` 是 PyTorch 中构建神经网络的基类。它封装了参数、前向传播逻辑以及参数管理。

定义神经网络模块

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleNet(nn.Module):
5     def __init__(self, input_dim, hidden_dim, output_dim):
6         super().__init__()
7         self.fc1 = nn.Linear(input_dim, hidden_dim)
8         self.relu = nn.ReLU()
9         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
10        self.fc3 = nn.Linear(hidden_dim, output_dim)
11
12    def forward(self, x):
13        x = self.fc1(x)
14        x = self.relu(x)
15        x = self.fc2(x)
16        x = self.relu(x)
17        x = self.fc3(x)
18        return x
19
20 # 创建模型
21 model = SimpleNet(input_dim=10, hidden_dim=32, output_dim=3)
22 print(model)
23
24 # 查看参数
25 for name, param in model.named_parameters():
26     print(f"{name}: {param.shape}")
27
28 # 总参数量
29 total_params = sum(p.numel() for p in model.parameters())
30 trainable_params = sum(p.numel() for p in model.parameters()
31                          if p.requires_grad)
32 print(f"\n总参数: {total_params}, 可训练: {trainable_params}")
```

6.4.1 常用层

常用神经网络层

```
1 import torch.nn as nn
2
3 # 线性层（全连接层）
4 fc = nn.Linear(in_features=128, out_features=64)
5
6 # 激活函数
7 relu = nn.ReLU()
8 gelu = nn.GELU()
9 sigmoid = nn.Sigmoid()
10 tanh = nn.Tanh()
11 leaky_relu = nn.LeakyReLU(negative_slope=0.01)
12
13 # 卷积层
14 conv1d = nn.Conv1d(in_channels=3, out_channels=16, kernel_size=3)
15 conv2d = nn.Conv2d(in_channels=3, out_channels=32,
16                    kernel_size=3, padding=1, stride=2)
17
18 # 池化层
19 maxpool = nn.MaxPool2d(kernel_size=2)
20 avgpool = nn.AvgPool2d(kernel_size=2)
21 adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))
22
23 # 归一化层
24 batch_norm = nn.BatchNorm1d(64)
25 layer_norm = nn.LayerNorm(128)
26
27 # 正则化
28 dropout = nn.Dropout(p=0.5)
29
30 # 循环层
31 lstm = nn.LSTM(input_size=64, hidden_size=128,
32                num_layers=2, batch_first=True)
33 gru = nn.GRU(input_size=64, hidden_size=128, batch_first=True)
34
35 # 嵌入层
36 embedding = nn.Embedding(num_embeddings=10000, embedding_dim=256)
37
38 # 打印参数量
39 print(f"Conv2d 参数量: {sum(p.numel() for p in conv2d.parameters())}")
40 print(f"LSTM 参数量: {sum(p.numel() for p in lstm.parameters())}")
```

6.4.2 Sequential 容器

对于简单的顺序模型，可以使用 `nn.Sequential` 简化代码：

Sequential 容器

```
1 import torch.nn as nn
2
3 # 方式1: 按顺序传入层
4 model = nn.Sequential(
5     nn.Linear(28*28, 512),
6     nn.ReLU(),
7     nn.Dropout(0.2),
8     nn.Linear(512, 256),
9     nn.ReLU(),
10    nn.Dropout(0.2),
11    nn.Linear(256, 10)
12 )
13
14 # 方式2: 使用OrderedDict命名各层
15 from collections import OrderedDict
16 model_named = nn.Sequential(OrderedDict([
17     ('fc1', nn.Linear(28*28, 512)),
18     ('relu1', nn.ReLU()),
19     ('fc2', nn.Linear(512, 256)),
20     ('relu2', nn.ReLU()),
21     ('fc3', nn.Linear(256, 10))
22 ]))
23
24 print(model)
25 x = torch.randn(4, 28*28) # batch of 4
26 out = model(x)
27 print(f"输出形状: {out.shape}") # (4, 10)
```

§6.5

损失函数与优化器

PyTorch 提供了丰富的损失函数和优化器：

损失函数

```
1 import torch
2 import torch.nn as nn
3
4 # 回归损失
5 mse_loss = nn.MSELoss()
6 l1_loss = nn.L1Loss()
7 huber_loss = nn.SmoothL1Loss()
8
9 # 二分类损失
10 bce_loss = nn.BCELoss() # 需要sigmoid后的输出
11 bce_logits = nn.BCEWithLogitsLoss() # 包含sigmoid (更稳定)
12
13 # 多分类损失
14 ce_loss = nn.CrossEntropyLoss() # 包含log_softmax
15
16 # 测试
17 # 回归示例
18 y_pred = torch.randn(32, 1)
19 y_true = torch.randn(32, 1)
20 print(f"MSE: {mse_loss(y_pred, y_true):.4f}")
21
22 # 分类示例
23 logits = torch.randn(32, 10) # 原始分数 (未归一化)
24 labels = torch.randint(0, 10, (32,))
25 print(f"交叉熵: {ce_loss(logits, labels):.4f}")
```

优化器

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 model = nn.Linear(10, 3)
6
7 # SGD
8 sgd = optim.SGD(model.parameters(), lr=0.01, momentum=0.9,
9                 weight_decay=1e-4, nesterov=True)
10
11 # Adam
12 adam = optim.Adam(model.parameters(), lr=0.001,
13                   betas=(0.9, 0.999), weight_decay=1e-4)
14
15 # AdamW (Adam + 解耦权重衰减, 推荐使用)
16 adamw = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
17
18 # RMSprop
19 rmsprop = optim.RMSprop(model.parameters(), lr=0.001, alpha=0.99)
20
21 # 学习率调度
22 optimizer = optim.AdamW(model.parameters(), lr=0.01)
23
24 # 阶梯衰减
25 scheduler_step = optim.lr_scheduler.StepLR(
26     optimizer, step_size=30, gamma=0.1)
27
28 # 余弦退火
29 scheduler_cos = optim.lr_scheduler.CosineAnnealingLR(
30     optimizer, T_max=100, eta_min=1e-6)
31
32 # OneCycleLR
33 scheduler_1cycle = optim.lr_scheduler.OneCycleLR(
34     optimizer, max_lr=0.01, total_steps=1000)
35
36 # 训练中使用
37 for epoch in range(10):
38     # ... 训练代码 ...
39     optimizer.step()
40     scheduler_cos.step() # 每个epoch更新
41     print(f"Epoch {epoch}: lr = {scheduler_cos.get_last_lr()[0]:.6f}")
```

§6.6
数据加载

PyTorch 的 `torch.utils.data` 模块提供了标准的数据加载方式：

Dataset 与 DataLoader

```

1 import torch
2 from torch.utils.data import Dataset, DataLoader, random_split
3 import numpy as np
4
5 # 自定义Dataset
6 class MyDataset(Dataset):
7     def __init__(self, n_samples=1000, n_features=10):
8         self.X = torch.randn(n_samples, n_features)
9         self.y = (self.X.sum(dim=1) > 0).long()
10
11     def __len__(self):
12         return len(self.X)
13
14     def __getitem__(self, idx):
15         return self.X[idx], self.y[idx]
16
17 # 创建数据集
18 dataset = MyDataset(n_samples=1000)
19
20 # 划分训练/验证集
21 train_size = int(0.8 * len(dataset))
22 val_size = len(dataset) - train_size
23 train_dataset, val_dataset = random_split(
24     dataset, [train_size, val_size])
25
26 # DataLoader
27 train_loader = DataLoader(train_dataset, batch_size=32,
28                           shuffle=True, num_workers=0,
29                           drop_last=True)
30 val_loader = DataLoader(val_dataset, batch_size=32,
31                         shuffle=False, num_workers=0)
32
33 # 使用
34 for batch_idx, (data, target) in enumerate(train_loader):
35     print(f"Batch {batch_idx}: data {data.shape}, target {target.shape}")
36     if batch_idx >= 2:
37         break

```

6.6.1 内置数据集

PyTorch 的 torchvision 提供了常用图像数据集：

使用 torchvision 加载 MNIST

```
1 import torch
2 from torch.utils.data import DataLoader
3 from torchvision import datasets, transforms
4
5 # 数据预处理
6 transform = transforms.Compose([
7     transforms.ToTensor(), # 转为张量并归一化到[0,1]
8     transforms.Normalize((0.1307,), (0.3081,)) # MNIST均值/标准差
9 ])
10
11 # 加载数据集
12 train_dataset = datasets.MNIST(
13     root='./data', train=True, download=True,
14     transform=transform)
15 test_dataset = datasets.MNIST(
16     root='./data', train=False, download=True,
17     transform=transform)
18
19 train_loader = DataLoader(train_dataset, batch_size=64,
20                           shuffle=True)
21 test_loader = DataLoader(test_dataset, batch_size=1000,
22                          shuffle=False)
23
24 # 查看数据
25 images, labels = next(iter(train_loader))
26 print(f"图像形状: {images.shape}") # (64, 1, 28, 28)
27 print(f"标签形状: {labels.shape}") # (64,)
28 print(f"标签值范围: [{labels.min()}, {labels.max()}]")
```

§6.7
训练循环

一个完整的训练循环需要包含：训练模式设置、梯度清零、前向传播、损失计算、反向传播、参数更新和评估。

完整的训练流程

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader, random_split
5
6 # 准备数据
7 class ToyDataset(torch.utils.data.Dataset):
8     def __init__(self, n=3000, d=20, n_classes=5):
9         self.X = torch.randn(n, d)
10        self.y = torch.randint(0, n_classes, (n,))
11
12        def __len__(self): return len(self.X)
13        def __getitem__(self, idx): return self.X[idx], self.y[idx]
14
15    dataset = ToyDataset()
16    train_ds, val_ds = random_split(dataset, [2400, 600])
17    train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
18    val_loader = DataLoader(val_ds, batch_size=64, shuffle=False)
19
20 # 创建模型
21 class MLP(nn.Module):
22     def __init__(self, in_dim, hidden, n_classes):
23         super().__init__()
24         self.net = nn.Sequential(
25             nn.Linear(in_dim, hidden),
26             nn.ReLU(),
27             nn.Dropout(0.3),
28             nn.Linear(hidden, hidden // 2),
29             nn.ReLU(),
30             nn.Dropout(0.3),
31             nn.Linear(hidden // 2, n_classes)
32         )
33
34     def forward(self, x):
35         return self.net(x)
36
37    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
38    model = MLP(in_dim=20, hidden=128, n_classes=5).to(device)
39    criterion = nn.CrossEntropyLoss()
40    optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=1e-4)
41    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
42
43 # 训练循环
44 def train_epoch(model, loader, criterion, optimizer, device):
45     model.train()
46     total_loss, correct, total = 0.0, 0, 0
47
48     for data, target in loader:
49         data, target = data.to(device), target.to(device)
50
51         optimizer.zero_grad()
52         output = model(data)
53         loss = criterion(output, target)
54         loss.backward()

```

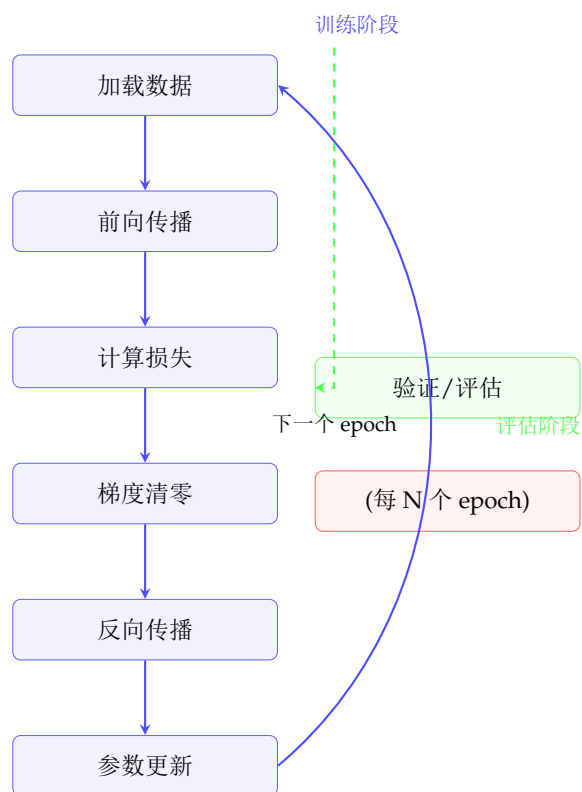


图 6.2: 完整的 PyTorch 训练循环流程。每个 epoch 包含训练阶段（数据加载、前向反向、参数更新）和周期性验证评估。

§6.8 模型保存与加载

PyTorch 提供两种模型保存方式：

模型保存与加载

```
1 import torch
2 import torch.nn as nn
3
4 model = nn.Sequential(
5     nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 3))
6
7 # 方式1: 保存整个模型 (不推荐, 耦合度高)
8 torch.save(model, 'full_model.pt')
9 loaded_model = torch.load('full_model.pt')
10
11 # 方式2: 只保存参数 (推荐)
12 torch.save(model.state_dict(), 'model_params.pt')
13
14 # 加载参数
15 new_model = nn.Sequential(
16     nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 3))
17 new_model.load_state_dict(torch.load('model_params.pt'))
18
19 # 保存检查点 (checkpoint)
20 checkpoint = {
21     'epoch': 50,
22     'model_state_dict': model.state_dict(),
23     'optimizer_state_dict': optimizer.state_dict(),
24     'loss': 0.0234,
25     'best_acc': 0.95,
26 }
27 torch.save(checkpoint, 'checkpoint.pt')
28
29 # 恢复训练
30 checkpoint = torch.load('checkpoint.pt')
31 model.load_state_dict(checkpoint['model_state_dict'])
32 optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
33 start_epoch = checkpoint['epoch']
```


§6.9

完整实例：MNIST 手写数字识别

MNIST 分类完整实现

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader
5 from torchvision import datasets, transforms
6
7 # 超参数
8 batch_size = 128
9 epochs = 10
10 lr = 0.001
11
12 # 数据准备
13 transform = transforms.Compose([
14     transforms.ToTensor(),
15     transforms.Normalize((0.1307,), (0.3081,))
16 ])
17
18 train_dataset = datasets.MNIST(
19     './data', train=True, download=True, transform=transform)
20 test_dataset = datasets.MNIST(
21     './data', train=False, transform=transform)
22
23 train_loader = DataLoader(train_dataset, batch_size=batch_size,
24                             shuffle=True)
25 test_loader = DataLoader(test_dataset, batch_size=batch_size,
26                             shuffle=False)
27
28 # CNN 模型
29 class CNN(nn.Module):
30     def __init__(self):
31         super().__init__()
32         self.conv1 = nn.Conv2d(1, 32, 3, padding=1)
33         self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
34         self.pool = nn.MaxPool2d(2, 2)
35         self.fc1 = nn.Linear(64 * 7 * 7, 128)
36         self.fc2 = nn.Linear(128, 10)
37         self.dropout = nn.Dropout(0.25)
38         self.relu = nn.ReLU()
39
40     def forward(self, x):
41         x = self.pool(self.relu(self.conv1(x)))
42         x = self.pool(self.relu(self.conv2(x)))
43         x = x.view(x.size(0), -1) # 展平
44         x = self.relu(self.fc1(x))
45         x = self.dropout(x)
46         x = self.fc2(x)
47         return x
48
49 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
50 model = CNN().to(device)
51 criterion = nn.CrossEntropyLoss()

```

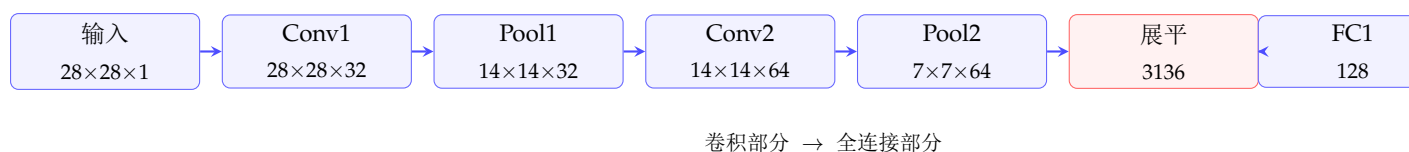


图 6.3: MNIST CNN 模型结构。输入 28x28 灰度图，经过两层卷积 + 池化提取特征，展平后通过两层全连接层输出 10 类概率。

§ 6.10 本章小结

本章介绍了 PyTorch 的核心使用方法：

- 张量是 PyTorch 的基础数据结构，支持 GPU 加速和自动微分
- 自动求导（Autograd）自动计算梯度，是实现反向传播的基础
- `nn.Module` 封装了神经网络层和参数管理
- PyTorch 提供了丰富的损失函数和优化器（SGD、Adam、AdamW 等）
- `Dataset` 和 `DataLoader` 规范了数据加载和批处理流程
- 完整的训练循环包括：训练模式设置、梯度清零、前向、损失计算、反向、参数更新
- 模型保存推荐使用 `state_dict` 方式

掌握这些 PyTorch 基础知识后，读者可以继续学习更高级的主题如卷积神经网络、循环神经网络、Transformer、生成模型等。

神经网络——深度学习的核心

本章是全书的枢纽章节。我们将从感知机出发，逐层揭示多层神经网络的数学原理：前向传播如何逐层变换表示，反向传播如何高效计算梯度，以及激活函数、损失函数、正则化等技术如何共同塑造现代深度学习。你将分别用 NumPy 和 PyTorch 亲手实现一个能识别手写数字的多分类神经网络。

§7.1

从感知机到多层神经网络

7.1.1 神经元——最基本的计算单元

神经网络的基本组成单元是人工神经元（artificial neuron）。它接收一组输入信号，加权求和后经激活函数变换输出：

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b, \quad a = g(z)$$

其中 $\mathbf{w} \in \mathbb{R}^n$ 为权重向量， b 为偏置， $g(\cdot)$ 为激活函数。

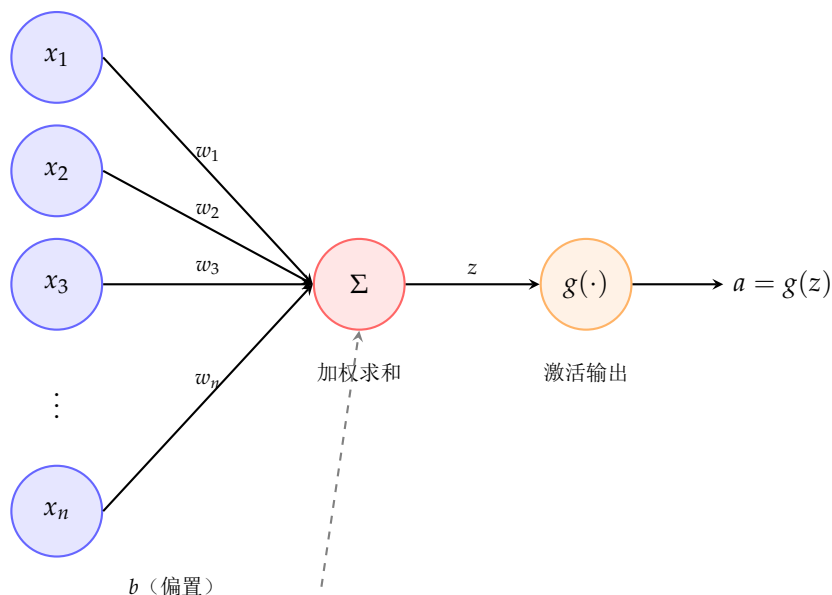


图 7.1: 人工神经元的结构。输入信号 $x_1, \dots, x_n$ 经权重 w_i 加权求和后加偏置 b ，再经激活函数 $g(\cdot)$ 变换产生输出 a

模型参数中，权重 w_i 表示输入 x_i 对输出的影响程度，偏置 b 允许神经元在没有输入时也有非零输出

——相当于线性模型中的截距项。

单个神经元等价于线性模型（若 g 为恒等映射）或广义线性模型。单神经元的表达能力有限，只能解决线性可分问题。

7.1.2 单层网络与异或问题

将多个神经元并排放置，构成**单层神经网络**（也称单隐藏层）：

$$\mathbf{a}^{[1]} = g(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}), \quad \hat{\mathbf{y}} = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

其中 $\mathbf{W}^{[1]} \in \mathbb{R}^{h \times n_x}$ 为第一层权重（ h 个神经元，每个 n_x 维输入）， $\mathbf{W}^{[2]} \in \mathbb{R}^{n_y \times h}$ 为输出层权重。

单隐藏层的网络已经具备**万能逼近能力**——只要有足够多的隐藏神经元，它可以逼近任意连续函数。然而，单层网络无法高效解决 XOR（异或）问题：XOR 是非线性可分的，需要至少两层隐藏层才能用紧致的分段线性边界成功分类。

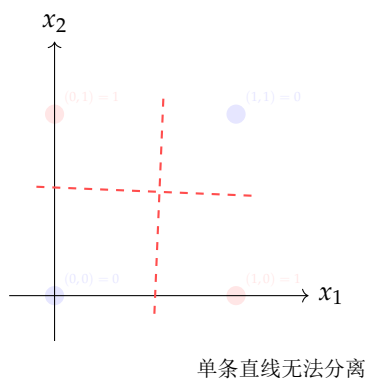


图 7.2: XOR 问题的几何直观。蓝色点输出 0，红色点输出 1。单条直线（单层感知机）无法分开这两类点

7.1.3 多层网络与表示学习

多层神经网络通过堆叠多个隐藏层，逐层提取越来越抽象的特征表示：

$$\mathbf{a}^{[l]} = g(\mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}), \quad l = 1, 2, \dots, L$$

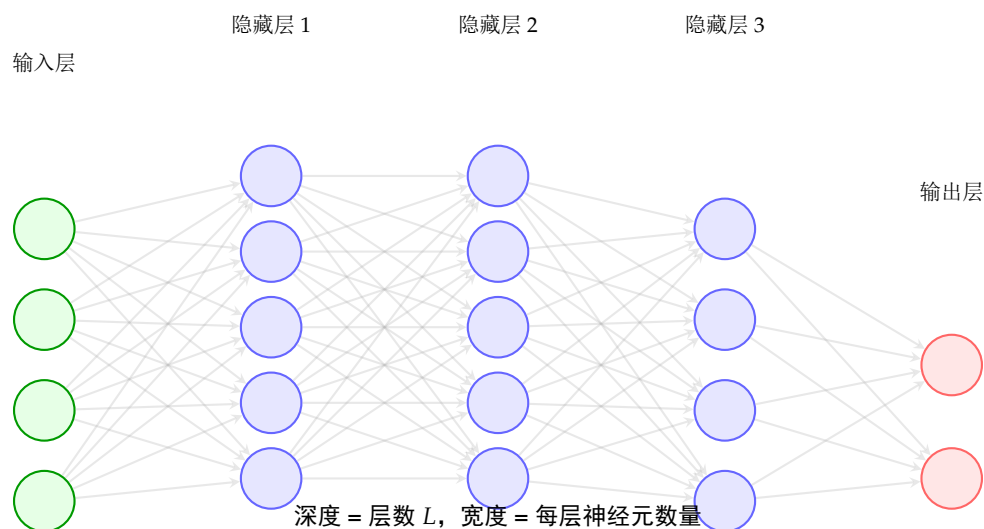


图 7.3: 具有三个隐藏层的深度神经网络。浅层学习简单的局部特征，深层将这些特征组合为更复杂的抽象表示

深度学习的核心优势在于层次化表示学习：

- 浅层神经元学习低级特征（边缘、纹理、音调）
- 中层神经元将低级特征组合为中级特征（形状、部件、音素）
- 深层神经元构建高级语义概念（物体、人脸、词语含义）

§7.2

激活函数——神经网络的灵魂

没有激活函数，多层网络等价于单层线性模型——线性变换的复合仍然是线性变换。激活函数引入非线性，赋予网络表达能力。

7.2.1 Sigmoid 与 Tanh

早期神经网络中最常用的两种激活函数：

$$\begin{aligned}\sigma(z) &= \frac{1}{1 + e^{-z}}, \quad \sigma(z) \in (0, 1) \\ \sigma'(z) &= \sigma(z)(1 - \sigma(z))\end{aligned}\tag{7.1}$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh(z) \in (-1, 1), \quad \tanh'(z) = 1 - \tanh^2(z)$$

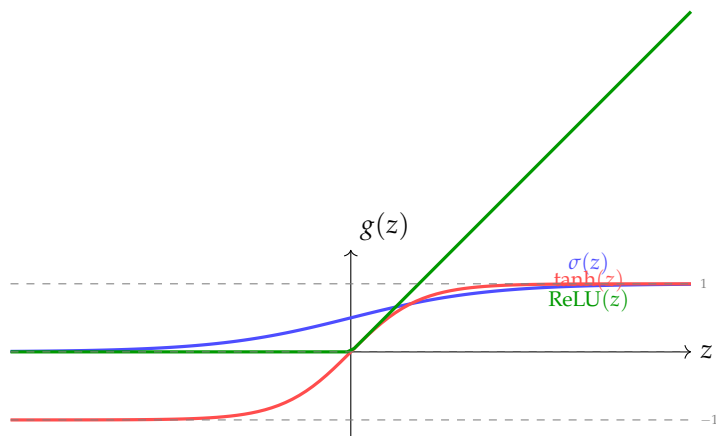


图 7.4: 三种常见激活函数的曲线对比。Sigmoid 将输出压缩到 $(0,1)$ ，Tanh 压缩到 $(-1,1)$ ，ReLU 在正半轴保持线性

7.2.2 ReLU 及其变体

ReLU (Rectified Linear Unit) 是现代神经网络中使用最广泛的激活函数：

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

ReLU 的优点：解决梯度消失问题（正半区导数为 1）、计算高效（只需比较和取 max）、具有稀疏激活特性（负半区输出为 0）。

ReLU 的缺点：“死亡 ReLU”问题——如果大梯度导致权重更新使得某个神经元对所有输入都输出负值，该神经元的梯度恒为 0，永远无法恢复。

Leaky ReLU 解决死亡神经元问题：

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha \text{ 通常取 } 0.01$$

GELU (Gaussian Error Linear Unit) 是 BERT 和 GPT 中使用的激活函数，将 ReLU 的硬阈值平滑化：

$$\text{GELU}(z) = z \cdot \Phi(z) \approx z \cdot \sigma(1.702z)$$

其中 $\Phi(z)$ 是标准正态分布的累积分布函数。GELU 的平滑性使其在 Transformer 架构中表现优于 ReLU。

7.2.3 Softmax——多分类的输出层

对于 K 类分类问题，**Softmax** 函数将原始得分 (logits) 映射为概率分布：

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K$$

Softmax 的性质：输出非负且和为 1；保持输入的大小顺序（单调性）；对平移不敏感： $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$ 。

激活函数的 PyTorch 实现对比

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import matplotlib.pyplot as plt
5
6 z = torch.linspace(-5, 5, 1000)
7
8 activations = {
9     'Sigmoid': F.sigmoid(z),
10    'Tanh': F.tanh(z),
11    'ReLU': F.relu(z),
12    'LeakyReLU': F.leaky_relu(z, negative_slope=0.01),
13    'GELU': F.gelu(z),
14    'Softplus': F.softplus(z),      # 平滑版ReLU:  $\log(1+e^z)$ 
15    'Swish': z * F.sigmoid(z),     # 自门控激活
16 }
17
18 fig, axes = plt.subplots(2, 4, figsize=(14, 7))
19 for ax, (name, y) in zip(axes.flat, activations.items()):
20     ax.plot(z.numpy(), y.numpy(), 'b-', lw=2)
21     ax.set_title(name); ax.grid(True, alpha=0.3)
22     ax.set_xlabel('z'); ax.set_ylabel('g(z)')
23
24 # 在PyTorch中作为层使用
25 relu_layer = nn.ReLU()
26 gelu_layer = nn.GELU()
27 leaky_layer = nn.LeakyReLU(0.01)

```

§7.3

前向传播——信息的流动

7.3.1 前向传播的矩阵形式

给定一个 L 层网络，前向传播将输入 $\mathbf{x} \in \mathbb{R}^{n_x}$ 逐层变换为最终输出 $\hat{\mathbf{y}}$:

$$\begin{aligned}
 \mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}, & \mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
 \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}, & \mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
 &\vdots \\
 \mathbf{z}^{[L]} &= \mathbf{W}^{[L]}\mathbf{a}^{[L-1]} + \mathbf{b}^{[L]}, & \hat{\mathbf{y}} = \mathbf{a}^{[L]} &= g^{[L]}(\mathbf{z}^{[L]})
 \end{aligned} \tag{7.2}$$

其中 $\mathbf{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$, $\mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$, $\mathbf{a}^{[l]} \in \mathbb{R}^{n_l}$ 。

在前向传播中，网络从数据中提取特征的过程可以理解为由逐层的坐标变换：每一层将前一层输出的特征空间进行线性变换（旋转、缩放、投影）后再经非线性扭曲，逐步构造出最适合当前任务的特征表示。

7.3.2 向量化——批量前向传播

在训练和推理中，我们通常一次处理多个样本（mini-batch）。将 m 个样本堆叠为矩阵 $\mathbf{X} \in \mathbb{R}^{n_x \times m}$ ，前向传播的向量化形式为：

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}\mathbf{X} + \mathbf{b}^{[1]}, \quad \mathbf{A}^{[1]} = g^{[1]}(\mathbf{Z}^{[1]})$$

其中 $\mathbf{b}^{[1]}$ 通过广播（broadcasting）加到每一列。这种向量化写法利用底层 BLAS 库的优化矩阵乘法，比显式的 for 循环快数十到数百倍。

§7.4 损失函数——定义优化目标

7.4.1 均方误差（MSE）

回归问题最常用的损失函数：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^m \|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|_2^2$$

MSE 对异常值敏感（平方放大大误差），但在高斯噪声假设下等价于最大似然估计。

7.4.2 交叉熵损失

分类问题的标准选择。对于二分类（ $y \in \{0, 1\}$ ）：

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

对于多分类（ K 类），使用分类交叉熵：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$$

其中 $\mathbf{y}^{(i)}$ 是 one-hot 向量， $\hat{\mathbf{y}}^{(i)}$ 是 Softmax 输出的概率分布。

交叉熵来自信息论： $H(p, q) = -\sum_x p(x) \log q(x)$ 衡量真实分布 p 与预测分布 q 之间的差异。最小化交叉熵等价于最大化正确类别的对数似然。交叉熵配合 Softmax 在反向传播时具有优美的梯度形式：

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_k} = \hat{y}_k - y_k$$

这一简洁的梯度形式是交叉熵损失被广泛使用的重要原因。

常见损失函数的 PyTorch 实现

```

1 import torch
2 import torch.nn as nn
3
4 # MSE (回归)
5 mse_loss = nn.MSELoss()
6 y_pred = torch.randn(32, 1)
7 y_true = torch.randn(32, 1)
8 loss_mse = mse_loss(y_pred, y_true)
9
10 # 二分类交叉熵 (BCEWithLogits 包含 Sigmoid, 数值更稳定)
11 bce_loss = nn.BCEWithLogitsLoss()
12 logits = torch.randn(32, 1) # 原始得分, 未经 Sigmoid
13 labels = torch.randint(0, 2, (32, 1)).float()
14 loss_bce = bce_loss(logits, labels)
15
16 # 多分类交叉熵 (CrossEntropyLoss 包含 Softmax)
17 ce_loss = nn.CrossEntropyLoss()
18 logits = torch.randn(32, 10) # 原始得分, (batch, classes)
19 labels = torch.randint(0, 10, (32,))
20 loss_ce = ce_loss(logits, labels)
21
22 # 手动计算 (理解原理)
23 def cross_entropy_manual(logits, labels, n_classes=10):
24     probs = torch.softmax(logits, dim=1)
25     labels_onehot = torch.zeros_like(logits)
26     labels_onehot.scatter_(1, labels.unsqueeze(1), 1)
27     return -(labels_onehot * torch.log(probs + 1e-9)).sum(dim=1).mean()
28
29 loss_manual = cross_entropy_manual(logits, labels)
30 print(f"PyTorch: {loss_ce.item():.6f}")
31 print(f"Manual: {loss_manual.item():.6f}")
32 print(f"Match: {torch.allclose(loss_ce, loss_manual, atol=1e-5)}")

```

§7.5

反向传播——学习的引擎

7.5.1 链式法则

反向传播 (Backpropagation) 是计算梯度的核心算法。它基于微积分的链式法则：对于复合函数 $f(g(x))$ ，有

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

在神经网络中，损失 $\mathcal{L}$ 是参数 $\{\mathbf{W}^{[l]}, \mathbf{b}^{[l]}\}_{l=1}^L$ 的复合函数。链式法则允许我们从输出层开始，逐层反向计算梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[l]}} \cdot \frac{\partial \mathbf{z}^{[l]}}{\partial \mathbf{W}^{[l]}}$$

7.5.2 反向传播的逐步推导

考虑一个 L 层网络（第 l 层激活函数为 $g^{[l]}$ ），损失函数为 $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ 。反向传播从输出层开始：

步骤 1：初始化。 计算输出层的误差：

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{[L]}} \odot g^{[L]'}(\mathbf{z}^{[L]})$$

对于 Softmax + 交叉熵的组合， $\delta^{[L]} = \hat{\mathbf{y}} - \mathbf{y}$ 。

步骤 2：反向迭代。 对于 $l = L-1, L-2, \dots, 1$ ：

$$\delta^{[l]} = \left(\mathbf{W}^{[l+1]\top} \delta^{[l+1]} \right) \odot g^{[l]'}(\mathbf{z}^{[l]})$$

步骤 3：计算梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} \mathbf{a}^{[l-1]\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$$

步骤 4：参数更新。 使用梯度下降更新：

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}, \quad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}}$$

7.5.3 反向传播的计算复杂度

反向传播的巧妙之处在于：它通过复用中间计算结果，使得梯度计算的时间复杂度与前向传播相同（都是 $O(\sum_l n_l n_{l-1})$ ），而非简单地对每个参数独立计算梯度那样的 $O((\sum_l n_l)^2)$ 。这是深度学习得以扩展到亿万级别参数的内在原因。

反向传播的计算效率

对于一个 L 层的全连接网络，前向传播和反向传播的计算复杂度均正比于网络中所有突触连接的数量。这一性质使得训练极深网络在计算上是可行的。

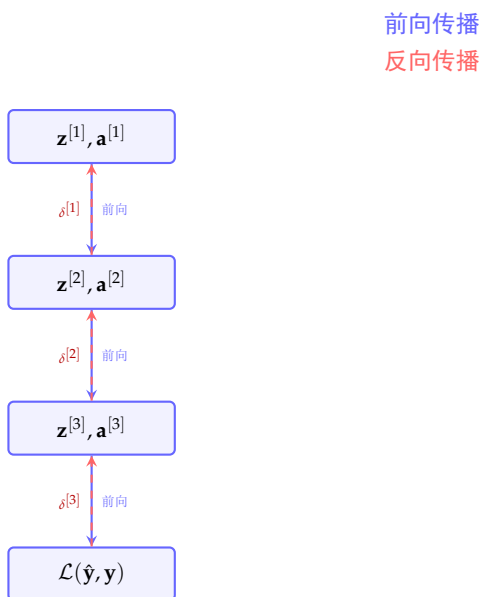


图 7.5: 前向传播存储中间激活值，反向传播利用这些缓存值计算各层参数的梯度

7.5.4 局部误差的直观解释

反向传播中的 $\delta_j^{[l]}$ 表示改变第 l 层的第 j 个神经元的线性输出 $z_j^{[l]}$ 对最终损失的影响程度。它可以拆解为两个因子的乘积：

$$\delta_j^{[l]} = \underbrace{\left(\sum_k w_{kj}^{[l+1]} \delta_k^{[l+1]} \right)}_{\text{来自第 } l+1 \text{ 层的反馈}} \times \underbrace{g^{[l]'}(z_j^{[l]})}_{\text{本层激活函数的梯度}}$$

这个形式揭示了反向传播的本质：每个神经元通过与后一层神经元的连接权重，接收来自后层的“误差反馈”，再乘以自身激活函数的导数，得到本层输出的误差信号。

§ 7.6

梯度消失与梯度爆炸

7.6.1 问题成因

深度网络训练中的核心困难。以 Sigmoid 激活函数为例， $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 在 z 较大或较小时导数趋近于 0。在反向传播中，梯度需要逐层乘以激活函数的导数：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} = \delta^{[1]} \mathbf{x}^\top, \quad \delta^{[1]} = \mathbf{W}^{[2]\top} \sigma'(\mathbf{z}^{[2]}) \odot \mathbf{W}^{[3]\top} \sigma'(\mathbf{z}^{[3]}) \odot \dots$$

经过多层的连乘，梯度呈指数级衰减（梯度消失）或指数级增长（梯度爆炸）。

梯度消失与梯度爆炸

如果 $\forall l, \|\mathbf{W}^{[l]}\| \cdot \|g^{[l]'}\| < 1$ ，梯度在反向传播中指数衰减，早期层的参数几乎停止更新。反之，如果乘积远大于 1，梯度指数增长，参数更新量极大，训练发散。

7.6.2 解决方案

- **ReLU 激活函数**：正半区梯度恒为 1，根本解决 Sigmoid/Tanh 的饱和问题
- **权重初始化**：Xavier 初始化（保持前向和反向方差稳定）、He 初始化（适配 ReLU）
- **Batch Normalization**：将每层输入的分布标准化，使激活值落在激活函数的非饱和区
- **残差连接（ResNet）**：提供梯度的“高速公路”，使梯度可以直接流到浅层
- **梯度裁剪**：限制梯度的范数，防止梯度爆炸

§ 7.7

正则化——防止过拟合

7.7.1 L^1 与 L^2 正则化

在损失函数中加入参数范数的惩罚项，限制模型复杂度：

$$\mathcal{L}_{\text{reg}} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) + \frac{\lambda}{2} \|\mathbf{W}\|_2^2 \quad (L^2 \text{ 正则化, Ridge, 梯度衰减})$$

$$\mathcal{L}_{\text{reg}} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \|\mathbf{W}\|_1 \quad (L^1 \text{ 正则化, Lasso, 稀疏化})$$

L^2 正则化（权重衰减）：所有权重向零方向收缩，防止任何单个权重过大。 L^1 正则化：产生稀疏解，自动特征选择。

7.7.2 Dropout

Dropout 是深度神经网络中最高效的正则化技术之一。在训练时，每个神经元以概率 p （通常 $p = 0.5$ ）被随机丢弃：

$$a_j^{[l]} = \begin{cases} 0 & \text{以概率 } p \\ a_j^{[l]} / (1 - p) & \text{以概率 } 1 - p \end{cases}$$

除以 $1 - p$ （inverted dropout）使得训练时的期望激活值不变，推理时无需任何缩放。

Dropout 的直观解释：相当于训练大量子网络的集成。每次迭代随机丢弃不同神经元，迫使网络学习冗余的、鲁棒的表示，而非依赖某个特定神经元的输出。

Dropout 在 PyTorch 中的应用

```

1 import torch.nn as nn
2
3 class MLPWithDropout(nn.Module):
4     def __init__(self, in_dim, hidden_dim, out_dim, dropout_p=0.5):
5         super().__init__()
6         self.net = nn.Sequential(
7             nn.Linear(in_dim, hidden_dim),
8             nn.ReLU(),
9             nn.Dropout(p=dropout_p),          # 训练时随机丢 ?      nn.
10             nn.Linear(hidden_dim, hidden_dim),
11             nn.ReLU(),
12             nn.Dropout(p=dropout_p),
13             nn.Linear(hidden_dim, out_dim)
14         )
15
16     def forward(self, x):
17         return self.net(x)
18
19 model = MLPWithDropout(784, 256, 10, dropout_p=0.3)
20
21 # 训练模式：Dropout 生效
22 model.train()
23 out_train = model(torch.randn(4, 784)) # 每次前向，不同神经元被丢 ?
24
25 # 评估模式：Dropout 关闭（所有神经元参与，无缩 ?
26 model.eval()
27 out_eval = model(torch.randn(4, 784))

```

7.7.3 Batch Normalization 的正则化效应

BN 除了加速训练，还有轻微的正则化效果：mini-batch 的均值和方差作为噪声源，类似于 Dropout 的随机扰动。不过这种正则化效果很弱，通常不能单独替代 Dropout 或权重衰减。

§7.8 用 NumPy 从零实现神经网络

为深入理解神经网络的本质，我们用纯 NumPy 实现一个用于 MNIST 二分类（数字 0 vs 非 0）的两层网络。

NumPy 实现的完整两层神经网络

```

1 import numpy as np
2
3 def sigmoid(z):
4     return 1 / (1 + np.exp(-z))
5
6 def relu(z):
7     return np.maximum(0, z)
8
9 def initialize_parameters(n_x, n_h, n_y, seed=42):
10     """He 初始化"""
11     np.random.seed(seed)
12     W1 = np.random.randn(n_h, n_x) * np.sqrt(2.0 / n_x)
13     b1 = np.zeros((n_h, 1))
14     W2 = np.random.randn(n_y, n_h) * np.sqrt(2.0 / n_h)
15     b2 = np.zeros((n_y, 1))
16     return {'W1': W1, 'b1': b1, 'W2': W2, 'b2': b2}
17
18 def forward_propagation(X, params):
19     W1, b1 = params['W1'], params['b1']
20     W2, b2 = params['W2'], params['b2']
21
22     Z1 = W1 @ X + b1          # 线性组合
23     A1 = relu(Z1)             # ReLU 激活
24     Z2 = W2 @ A1 + b2         # 输出层
25     A2 = sigmoid(Z2)         # 二分类用 Sigmoid
26
27     cache = (Z1, A1, Z2, A2)
28     return A2, cache
29
30 def compute_loss(A2, Y):
31     m = Y.shape[1]
32     # 二分类交叉熵 ?    loss = -(Y * np.log(A2 + 1e-8)
33     #                   + (1 - Y) * np.log(1 - A2 + 1e-8))
34     return np.mean(loss)
35
36 def backward_propagation(X, Y, params, cache):
37     m = X.shape[1]
38     W2 = params['W2']
39     Z1, A1, Z2, A2 = cache
40
41     # 输出层梯 ?    dZ2 = A2 - Y                                # (n_y, m)
42     dW2 = dZ2 @ A1.T / m                                         # (n_y, n_h)
43     db2 = np.sum(dZ2, axis=1, keepdims=True) / m
44
45     # 隐藏层梯 ?    dA1 = W2.T @ dZ2                            # (n_h, m)
46     dZ1 = dA1 * (Z1 > 0).astype(float)    # ReLU 导数
47     dW1 = dZ1 @ X.T / m                                         # (n_h, n_x)
48     db1 = np.sum(dZ1, axis=1, keepdims=True) / m
49
50     grads = {'dW1': dW1, 'db1': db1, 'dW2': dW2, 'db2': db2}
51     return grads
52
53 def update_parameters(params, grads, lr):
54     for key in params:

```

7.8.1 代码核心要点说明

- **He 初始化**: 权重从 $\mathcal{N}(0, \sqrt{2/n_{\text{in}}})$ 采样, 偏置初始化为零。这保证 ReLU 激活在前向传播中层间方差基本稳定。
- **ReLU 的梯度**: $dZ_1 = dA_1 \odot (Z_1 > 0)$ 。利用 `'(Z1 > 0).astype(float)'` 高效计算 ReLU 在每点的导数 (正为 1, 负为 0, 0 处为 0)。
- **交叉熵的简洁梯度**: $dZ_2 = A_2 - Y$ 。这是交叉熵损失配合 Sigmoid 的数学性质——避免了除零风险且计算量极小。
- **向量化**: 所有运算均通过矩阵乘法实现, 避免 Python 循环。

§7.9

用 PyTorch 实现多分类神经网络

7.9.1 模块化构建

PyTorch 多分类神经网络 (MNIST)

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader
5 from torchvision import datasets, transforms
6
7 # ===== 数据准备 =====
8 transform = transforms.Compose([
9     transforms.ToTensor(), # [0,255] -> [0,1]
10    transforms.Normalize((0.1307,), (0.3081,)) # MNIST 经验统计
11 ])
12
13 train_data = datasets.MNIST(
14     root='./data', train=True,
15     download=True, transform=transform
16 )
17 test_data = datasets.MNIST(
18     root='./data', train=False,
19     download=True, transform=transform
20 )
21
22 train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
23 test_loader = DataLoader(test_data, batch_size=1000, shuffle=False)
24
25 # ===== 定义模型 =====
26 class FeedForwardNet(nn.Module):
27     def __init__(self, in_dim=784, hidden_dims=[256, 128],
28                 out_dim=10, dropout_p=0.3):
29         super().__init__()
30         layers = []
31         prev_dim = in_dim
32         for h_dim in hidden_dims:
33             layers.append(nn.Linear(prev_dim, h_dim))
34             layers.append(nn.ReLU())
35             layers.append(nn.BatchNorm1d(h_dim))
36             layers.append(nn.Dropout(dropout_p))
37             prev_dim = h_dim
38         layers.append(nn.Linear(prev_dim, out_dim))
39         self.net = nn.Sequential(*layers)
40
41     def forward(self, x):
42         x = x.view(x.size(0), -1) # 展平: (batch,1,28,28) -> (batch,784)
43         return self.net(x)
44
45 # ===== 训练配置 =====
46 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
47 model = FeedForwardNet().to(device)
48 criterion = nn.CrossEntropyLoss()
49 optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)

```

7.9.2 超参数的影响

超参数	过小	适中	过大
学习率	收敛极慢	稳定收敛	振荡/发散
隐藏层大小	欠拟合	良好泛化	过拟合
网络深度	表达能力不足	层次化表示	梯度消失
Dropout 率	弱正则化	有效防过拟合	欠拟合
权重衰减	弱正则化	良好约束	过低拟合

§7.10

参数初始化策略

Xavier/Glorot 初始化

对于使用 Sigmoid 或 Tanh 激活的网络，权重应从以下分布采样以保持前向和反向传播的方差稳定：

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

或正态分布： $W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$ 。

He/Kaiming 初始化

对于 ReLU 激活函数及其变体，推荐：

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

这是 PyTorch 中 `nn.Linear` 的默认初始化（通过 `kaiming_uniform_` 实现）。

注. PyTorch 大多数内置层（Linear、Conv2d 等）在创建时自动应用 He 初始化，通常无需手动设置。但自定义层或从随机分布初始化时仍需注意选择正确的初始化策略。

§7.11

本章小结

本章从单个神经元出发，系统地构建了多层神经网络的理论体系。关键回顾：

- 前向传播将输入逐层变换为高阶抽象表示，本质是层次化的坐标映射
- 反向传播以与前向传播相同的计算复杂度高效求解所有参数的梯度
- ReLU 系列激活函数解决了深层网络的梯度消失问题
- 交叉熵损失配合 Softmax 具有简洁的梯度形式，是分类任务的标准选择
- Dropout 通过随机丢弃神经元实现高效的隐式集成学习
- 权重初始化和 Batch Normalization 是训练深层网络的关键技术

优化深度神经网络

如果说网络结构决定了模型的“能力上限”，那么优化算法则决定了我们能否到达这个上限。本章从随机梯度下降出发，深入动量法、自适应学习率方法（Adam/AdamW），以及学习率调度、超参数搜索等实用技术，帮助你驯服深层网络的训练过程。

§ 8.1

优化问题的数学描述

8.1.1 经验风险最小化

深度学习中的优化问题可以统一表述为经验风险最小化（Empirical Risk Minimization）：

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$$

其中 $\theta \in \mathbb{R}^d$ 是模型的所有参数（通常 d 可达数十亿）， $\ell(\cdot, \cdot)$ 是样本损失， $\mathcal{L}$ 是整个训练集的平均损失。

这个优化问题的特点使其与经典的凸优化有本质不同：

- **非凸性**——损失曲面存在大量局部极小值、鞍点和平台区域
- **高维度**——参数量可达百万甚至万亿级别
- **数据规模大**——百万级样本，不可能每步计算全梯度
- **目标泛化**——训练损失最低的解不一定是测试损失最低的

8.1.2 非凸优化的挑战——鞍点而非局部极小值

在高维空间中，纯局部极小值（所有方向的曲率都为正）极为罕见；更常见的是**鞍点**（saddle point）——某些方向向上弯曲，其他方向向下弯身。在高维优化中，逃离鞍点是比逃离局部极小值更核心的问题。

注 神经网络的损失曲面高度非凸，但实践表明：大多数局部极小值具有相似的泛化性能，而梯度下降算法擅长逃离鞍点但可能卡在平坦的“高原”区域。因此，现代优化器设计的核心目标是加速逃离鞍点和平坦区域。

§ 8.2 梯度下降法及其变体

8.2.1 批量梯度下降

批量梯度下降（Batch Gradient Descent, BGD）使用整个训练集计算梯度，每次参数更新方向是最准确的：

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t) = \boldsymbol{\theta}_t - \frac{\eta}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

BGD 的梯度估计无偏且方差为零，但每步更新都需要遍历全部数据，计算代价高昂，现代深度学习训练几乎不使用。

8.2.2 随机梯度下降

随机梯度下降（Stochastic Gradient Descent, SGD）每步只用一个随机样本来估计梯度：

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

单个样本的梯度方差极大，导致参数更新方向剧烈振荡。但这一噪声在实际中可能是有益的——它帮助算法跳出局部极小值和鞍点。

8.2.3 小批量随机梯度下降

小批量 SGD（Mini-batch SGD）是实际使用中最标准的方法：每步使用一个小批量（mini-batch） $\mathcal{B}$ 的样本：

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

批大小 $|\mathcal{B}|$ 是重要的超参数：

- 批大小太小——梯度噪声过大，训练不稳定，GPU 利用率低
- 批大小适中——噪声适度，收敛快，对泛化有利（噪声有正则化效果）
- 批大小太大——梯度精确但每步计算量大，泛化性能可能下降

在实际中，批大小常取 32、64、128 或 256，并受 GPU 显存限制。

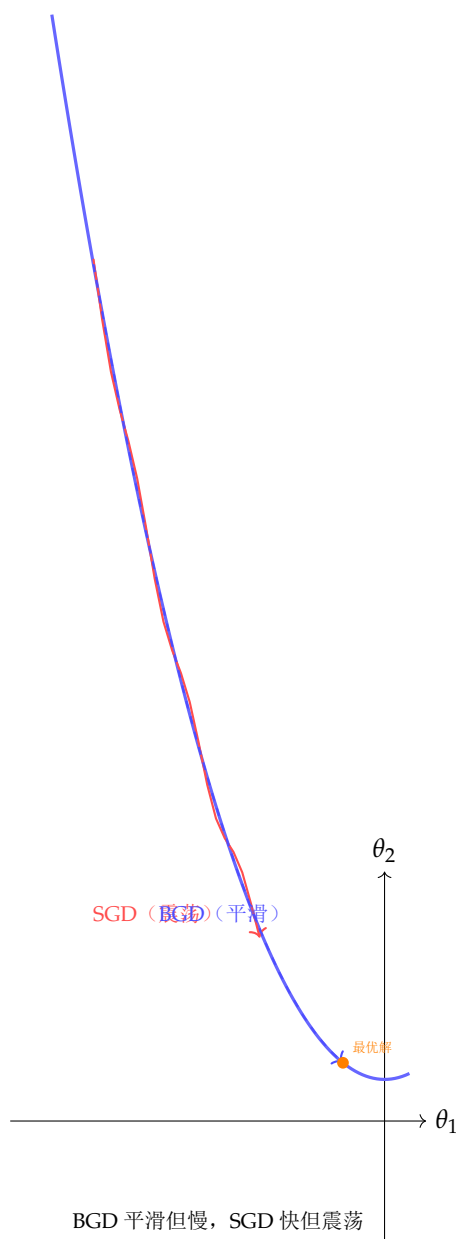


图 8.1: BGD 与 SGD 在损失曲面上的优化轨迹对比。BGD 路径平滑但每步计算代价大，SGD 路径振荡但收敛更快（总计算量更小）

§ 8.3

动量法——让梯度具有“惯性”

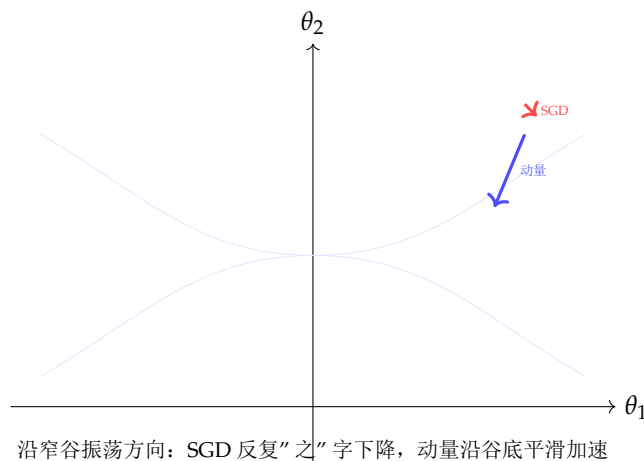
8.3.1 经典动量

动量法（Momentum）在 SGD 的基础上引入速度变量 $\mathbf{v}$ ，使得参数更新不仅取决于当前梯度，还取决于历史上的更新方向：

$$\begin{aligned}\mathbf{v}_t &= \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t) \\ \theta_{t+1} &= \theta_t - \eta \mathbf{v}_t\end{aligned}\tag{8.1}$$

其中 $\beta \in [0, 1)$ 是动量系数（通常取 0.9）。 $\mathbf{v}_t$ 可以视为梯度的指数加权移动平均（EMA）。
动量的两大好处：

- **加速收敛**——在梯度方向一致的维度（如平缓谷底）持续加速
- **抑制振荡**——在梯度方向频繁翻转的维度（如窄谷两侧），正负梯度相互抵消，减小震荡



沿窄谷振荡方向：SGD 反复“之”字下降，动量沿谷底平滑加速

图 8.2: 在窄谷状的损失曲面上，SGD 在峡谷两侧反复振荡下降缓慢，动量法利用历史方向的惯性沿谷底快速推进

8.3.2 Nesterov 加速梯度

Nesterov 加速梯度（NAG）是动量法的改进，其关键思想是：先按累积动量“前瞻”一步，再在“前瞻点”计算梯度进行修正：

$$\begin{aligned}\mathbf{v}_t &= \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t - \eta \beta \mathbf{v}_{t-1}) \\ \theta_{t+1} &= \theta_t - \eta \mathbf{v}_t\end{aligned}\tag{8.2}$$

NAG 可以理解为带有“前瞻”的动量法——在惯性方向上先走一步，看看前方曲率变化，再决定实际更新方向和步长。这使得 NAG 在凸优化中具有更好的理论收敛保证。

§ 8.4 自适应学习率方法

动量法在所有参数维度上使用相同的全局学习率。然而，在深度网络中，不同层、不同参数可能需要不同的学习率。自适应方法为每个参数独立调整学习率。

8.4.1 AdaGrad

AdaGrad 累积历史梯度的平方和，使频繁更新的参数学习率衰减更快：

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \nabla \mathcal{L}(\theta_t)^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{G}_t + \varepsilon}} \odot \nabla \mathcal{L}(\theta_t)$$

其中所有运算为逐元素操作。 $\mathbf{G}_t$ 单调递增，导致学习率在训练后期衰减至几乎为零，这使得 AdaGrad 对非平稳目标（如深度网络的非凸优化）不够友好。

8.4.2 RMSProp

RMSProp 通过将累积改为指数移动平均，解决了 AdaGrad 学习率单调衰减的问题：

$$\mathbf{G}_t = \beta \mathbf{G}_{t-1} + (1 - \beta) (\nabla \mathcal{L}(\boldsymbol{\theta}_t))^2$$

其中 β 通常取 0.9。RMSProp 使用近期梯度的二阶矩估计来调整各维度的学习率。

8.4.3 Adam——当前的事实标准

Adam（Adaptive Moment Estimation）同时结合了动量（一阶矩）和 RMSProp（二阶矩）：

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla \mathcal{L}(\boldsymbol{\theta}_t) \quad (\text{一阶矩, 动量}) \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla \mathcal{L}(\boldsymbol{\theta}_t))^2 \quad (\text{二阶矩, 自适应学习率}) \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad (\text{偏差修正}) \\ \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (\text{偏差修正}) \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} \end{aligned} \tag{8.3}$$

默认超参数： $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ 。

偏差修正的必要性：由于 $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$ ，在训练初期，EMA 估计会严重偏向零。偏差修正项 $1/(1 - \beta^t)$ 在初期放大估计值， t 增大后趋近于 1，自然地消除初始偏差。

Adam 的收敛性质

Adam 在凸优化中具有 $O(1/\sqrt{T})$ 的遗憾界（regret bound），与 SGD 相同。在非凸随机优化中，Adam 可以保证收敛到平稳点。但实际上，Adam 在 Transformer 等现代架构上经常优于 SGD+momentum。

8.4.4 AdamW——解耦权重衰减

标准 Adam 中，权重衰减（ L^2 正则化）与自适应学习率耦合，导致大梯度参数的正则化效果减弱。AdamW 将权重衰减从梯度更新中解耦：

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} - \eta \lambda \boldsymbol{\theta}_{t-1}$$

最后的 $\eta \lambda \boldsymbol{\theta}_{t-1}$ 项是解耦的权重衰减（decoupled weight decay），其效果等价于在损失中加 $(\lambda/2) \|\boldsymbol{\theta}\|^2$ 但不干扰自适应学习率。AdamW 已成为 Transformer 训练的标准优化器。

PyTorch 中各种优化器的使用与对比

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # 构建测试用模型 model = nn.Sequential(
6     nn.Linear(100, 200), nn.ReLU(),
7     nn.Linear(200, 200), nn.ReLU(),
8     nn.Linear(200, 10)
9 )
10
11 # ===== 各种优化器 =====
12 # SGD + 动量
13 opt_sgd = optim.SGD(model.parameters(), lr=0.01,
14                     momentum=0.9, weight_decay=1e-4,
15                     nesterov=True) # Nesterov 加速
16
17 # Adam
18 opt_adam = optim.Adam(model.parameters(), lr=1e-3,
19                      betas=(0.9, 0.999), eps=1e-8,
20                      weight_decay=0) # 注意: Adam 的 weight_decay 是 L2 正则化
21
22 # AdamW (推荐!)
23 opt_adamw = optim.AdamW(model.parameters(), lr=1e-3,
24                        betas=(0.9, 0.999), eps=1e-8,
25                        weight_decay=0.01) # 解耦的权重衰减? # RMSProp
26 opt_rmsprop = optim.RMSprop(model.parameters(), lr=1e-3,
27                             alpha=0.99, eps=1e-8,
28                             momentum=0.9)
29
30 # ===== 遍历所有优化器进行训练 =====
31 optimizers = {
32     'SGD+Nesterov': opt_sgd,
33     'Adam': opt_adam,
34     'AdamW': opt_adamw,
35     'RMSProp+Momentum': opt_rmsprop,
36 }
37
38 def train_with_optimizer(name, optimizer, X, y, n_steps=500):
39     criterion = nn.CrossEntropyLoss()
40     losses = []
41     for step in range(n_steps):
42         optimizer.zero_grad()
43         output = model(X)
44         loss = criterion(output, y)
45         loss.backward()
46         optimizer.step()
47         losses.append(loss.item())
48     return losses
49
50 # 对比不同优化器的收敛曲线? X = torch.randn(128, 100)
51 y = torch.randint(0, 10, (128,)) 104
52 results = {}
53 for name, opt in optimizers.items():
54     model_copy = nn.Sequential(

```

§ 8.5 学习率调度策略

固定的学习率在训练后期可能过大（导致在最优解附近振荡）或过小（导致收敛缓慢）。学习率调度（Learning Rate Schedule）在训练过程中动态调整学习率。

8.5.1 常用调度策略

- 阶梯衰减（Step Decay）：每固定步数将学习率乘以衰减因子 γ （如每 30 个 epoch 将学习率减半）
- 指数衰减： $\eta_t = \eta_0 \cdot \gamma^t$
- 余弦退火（Cosine Annealing）： $\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 + \cos(\frac{t}{T}\pi))$
- 预热（Warmup）：在训练的前若干步将学习率从 0 线性增加到目标值，随后再衰减。对 Transformer 模型训练至关重要
- ReduceLROnPlateau：当验证集指标不再改善时自动降低学习率

PyTorch 学习率调度器示例

```

1 import torch.optim as optim
2 import torch.optim.lr_scheduler as lr_scheduler
3
4 model = nn.Linear(100, 10)
5 optimizer = optim.AdamW(model.parameters(), lr=1e-3)
6
7 # 1. 阶梯衰减 scheduler_step = lr_scheduler.StepLR(optimizer,
8     step_size=30, gamma=0.1)
9
10 # 2. 余弦退火 (带预热)
11 scheduler_cos = lr_scheduler.CosineAnnealingLR(
12     optimizer, T_max=100, eta_min=1e-6)
13
14 # 3. ReduceLROnPlateau
15 scheduler_plateau = lr_scheduler.ReduceLROnPlateau(
16     optimizer, mode='min', factor=0.5, patience=10)
17
18 # 4. OneCycleLR (最先进的通用调度器)
19 scheduler_1cycle = lr_scheduler.OneCycleLR(
20     optimizer, max_lr=1e-2,
21     steps_per_epoch=len(train_loader),
22     epochs=50, pct_start=0.3) # 前30%时间预热
23
24 # 5. 预热 + 余弦退火 (Transformers 标准)
25 def warmup_cosine(optimizer, warmup_steps, total_steps):
26     """自定义 warmup-cosine 组合调度"""
27     def lr_lambda(current_step):
28         if current_step < warmup_steps:
29             return float(current_step) / float(max(1, warmup_steps))
30         progress = float(current_step - warmup_steps) / \
31             float(max(1, total_steps - warmup_steps))
32         return max(0.0, 0.5 * (1.0 + math.cos(math.pi * progress)))
33     return lr_scheduler.LambdaLR(optimizer, lr_lambda)
34
35 # 使用
36 scheduler_warmup_cos = warmup_cosine(optimizer,
37     warmup_steps=1000, total_steps=10000)
38
39 # 训练循环中
40 for epoch in range(100):
41     train(...) # 训练一个epoch
42     scheduler_step.step() # epoch级调度
43
44     # 对 batch 级调度:
45     # for batch_idx, (X, y) in enumerate(loader):
46     #     train_step(...)
47     #     scheduler_1cycle.step() # 每batch更新LR

```

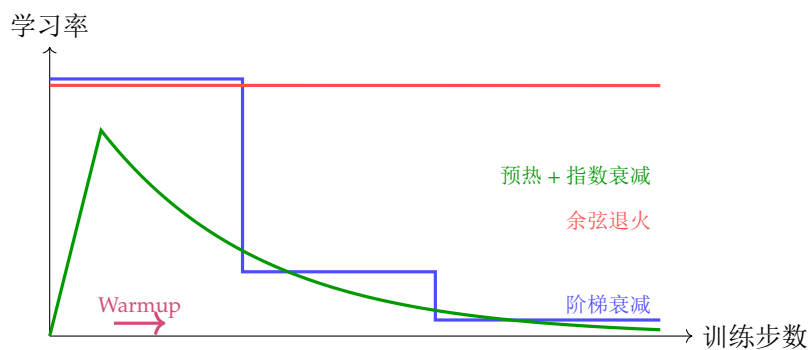


图 8.3: 三种常用学习率调度策略的曲线对比。阶梯衰减简单粗放，余弦退火平滑逐渐，Warmup+ 衰减对 Transformer 至关重要

§8.6 梯度裁剪

在深度网络（特别是 RNN 和 Transformer）训练中，梯度爆炸是一个常见问题。**梯度裁剪**通过限制梯度的最大范数来防止参数更新过大：

$$\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \|\mathbf{g}\|_2 \leq c \\ c \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|_2} & \|\mathbf{g}\|_2 > c \end{cases}$$

即：如果梯度的 L^2 范数超过阈值 c ，则将其缩放到范数恰好为 c 。这一简单技术极大提高了 RNN 训练的稳定性。

梯度裁剪与其他训练技巧

```

1 import torch
2
3 # 梯度裁剪（通常在loss.backward()之后，optimizer.step()之前）
4 loss.backward()
5 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
6 optimizer.step()
7
8 # 也可按值裁剪
9 torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5)
10
11 # ===== 梯度监测 =====
12 def monitor_gradients(model):
13     """ 监控各层梯度范数（帮助诊断训练问题） """
14     total_norm = 0
15     for name, p in model.named_parameters():
16         if p.grad is not None:
17             param_norm = p.grad.data.norm(2)
18             total_norm += param_norm.item() ** 2
19             if param_norm > 10: # 梯度异常大
20                 print(f"WARNING: Large grad in {name}: {param_norm:.2f}")
21     return total_norm ** 0.5
22
23 # ===== 混合精度训练（节省显存，加速训练） =====
24 from torch.cuda.amp import autocast, GradScaler
25
26 scaler = GradScaler()
27
28 for X, y in loader:
29     optimizer.zero_grad()
30
31     with autocast(): # 自动混合精度前向
32         output = model(X)
33         loss = criterion(output, y)
34
35     scaler.scale(loss).backward() # 缩放loss防止梯度下溢？ scaler.unscale_(
36         optimizer) # 反缩？ torch.nn.utils.clip_grad_norm_(model.
37         parameters(), 1.0)
38     scaler.step(optimizer) # 更新参数
39     scaler.update() # 更新缩放？

```

§8.7

二阶优化方法简介

一阶方法（SGD、Adam）只使用**梯度**（一阶导数）信息。二阶方法使用**Hessian**矩阵（二阶导数）来捕捉损失曲面的曲率信息。

8.7.1 牛顿法

牛顿法的更新规则考虑了曲率的倒数：

$$\theta_{t+1} = \theta_t - \mathbf{H}^{-1} \nabla \mathcal{L}(\theta_t)$$

其中 $\mathbf{H} = \nabla_{\theta}^2 \mathcal{L}(\theta)$ 是 Hessian 矩阵。牛顿法在凸函数上具有二次收敛速度，但：

- 计算 $\mathbf{H}^{-1}$ 需要 $O(d^3)$ 时间（ d 为参数数量）
- 存储 $d \times d$ 的 Hessian 矩阵需要 $O(d^2)$ 内存
- 在非凸函数上可能收敛到鞍点而非局部极小值

8.7.2 L-BFGS——实用的拟牛顿法

L-BFGS（Limited-memory BFGS）通过维持有限历史中的梯度差异来近似 Hessian 的逆，避免了显式存储和求逆。在参数较少的任务（如风格迁移、小规模优化）中，L-BFGS 往往比 Adam 表现更好。

在深度学习中，二阶方法主要用于：

- 小规模、对精度要求高的优化任务
- 超参数调优的内层循环
- 加固已收敛模型的微调

§ 8.8 超参数调优

8.8.1 网格搜索 vs 随机搜索 vs 贝叶斯优化

- **网格搜索**：穷举所有超参数组合。当超参数维度较高时，组合数指数爆炸
- **随机搜索**：在超参数空间中随机采样。研究表明，在同样计算预算下，随机搜索找到的最优解通常优于网格搜索
- **贝叶斯优化**：使用高斯过程或 TPE（Tree-structured Parzen Estimator）建模超参数到验证性能的映射，在每次试验后更新模型，智能地选择下一组超参数

8.8.2 实用超参数调优建议

1. **学习率是最重要的超参数**——先花 80% 的精力找到合适的学习率
2. **使用学习率范围测试（LR Range Test）**：从极小学习率开始，每个 batch 逐步增大，观察损失的变化趋势，选择损失下降最快区域的 LR
3. **批大小**通常受 GPU 显存约束，在可行范围内优先选择较大的值
4. **权重衰减**通常取 10^{-4} 到 10^{-2} 之间，配合 AdamW 使用
5. **Dropout 率**：全连接层 0.5，卷积层 0.1-0.3，Transformer 中通常 0.1
6. **隐藏层数/宽度**：从较小的网络开始，逐步增大直到过拟合，再引入正则化

Optuna 贝叶斯超参数调优

```
1 import optuna # pip install optuna
2
3 def objective(trial):
4     # 采样超参数
5     lr = trial.suggest_float('lr', 1e-5, 1e-1, log=True)
6     n_layers = trial.suggest_int('n_layers', 1, 5)
7     hidden_dim = trial.suggest_int('hidden_dim', 32, 512, step=32)
8     dropout_p = trial.suggest_float('dropout_p', 0.0, 0.5)
9     batch_size = trial.suggest_categorical(
10         'batch_size', [16, 32, 64, 128])
11
12     # 构建并训练模型
13     model = build_model(n_layers, hidden_dim, dropout_p)
14     val_acc = train_and_evaluate(model, lr, batch_size)
15
16     return val_acc # Optuna 最大化目标
17
18 # 创建研究并优化
19 study = optuna.create_study(
20     direction='maximize',
21     pruner=optuna.pruners.MedianPruner()
22 )
23 study.optimize(objective, n_trials=50)
24
25 print(f"Best params: {study.best_params}")
26 print(f"Best value: {study.best_value:.4f}")
27
28 # 可视化
29 optuna.visualization.plot_param_importances(study)
30 optuna.visualization.plot_optimization_history(study)
31 optuna.visualization.plot_contour(study,
32     params=['lr', 'hidden_dim'])
```

§8.9

训练稳定性与调试技巧

8.9.1 梯度与激活值监控

深层网络训练中最常见的问题是梯度消失/爆炸和激活值异常：

训练监控与调试代码

```

1 import torch
2 import numpy as np
3
4 class Monitor:
5     """插入模型各层，记录激活值和梯度的统计"""
6     def __init__(self, model):
7         self.model = model
8         self.activations = {}
9         self.gradients = {}
10        self._register_hooks()
11
12    def _register_hooks(self):
13        def forward_hook(name):
14            def hook(module, inp, out):
15                self.activations[name] = {
16                    'mean': out.data.mean().item(),
17                    'std': out.data.std().item(),
18                    'max': out.data.max().item(),
19                    'frac_zero': (out.data == 0).float().mean().item()
20                }
21            return hook
22
23        def backward_hook(name):
24            def hook(module, grad_in, grad_out):
25                if grad_out[0] is not None:
26                    self.gradients[name] = {
27                        'mean': grad_out[0].data.mean().item(),
28                        'std': grad_out[0].data.std().item(),
29                        'norm': grad_out[0].data.norm(2).item()
30                    }
31            return hook
32
33        for name, module in self.model.named_modules():
34            if isinstance(module, (nn.Linear, nn.Conv2d)):
35                module.register_forward_hook(forward_hook(name))
36                module.register_backward_hook(backward_hook(name))
37
38    def report(self):
39        print("\n=== 激活值统计 ===")
40        for name, stats in self.activations.items():
41            flag = "!!!" if abs(stats['mean']) > 10 else ""
42            print(f" {name:30s}: mean={stats['mean']:.4f}, "
43                  f"std={stats['std']:.4f}, zero={stats['frac_zero']:.2%} {flag}")
44
45        print("\n=== 梯度统计 ===")
46        for name, stats in self.gradients.items():
47            flag = "!!!" if stats['norm'] > 100 else ""
48            print(f" {name:30s}: mean={stats['mean']:.6f}, "
49                  f"norm={stats['norm']:.4f} {flag}")
50
51    # ===== 梯度检查（验证自定义反向传播的正确性） =====
52    from torch.autograd import gradcheck
53

```


8.9.2 常见训练问题诊断表

症状	可能原因	解决方案
Loss 不下降 Loss 为 NaN 训练 Loss 远低于验证 Loss	学习率过小/过大 梯度爆炸；学习率过大 过拟合	调整学习率；做 LR Range Test 梯度裁剪；降低学习率 增加正则化（Dropout, 权重衰减）
验证 Loss 远低于训练 Loss Loss 下降后突然跳高 GPU 利用率低	数据泄漏；Dropout 干扰 学习率过大 batch size 过小；数据加载慢	检查数据处理；调整 Dropout 率 降低学习率；使用学习率调度 增大 batch size；增加 num_workers

§8.10 本章小结

优化是深度学习训练的核心实践环节。本章覆盖了从基础 SGD 到现代自适应优化器的完整演进路径：

- 动量法通过累积历史梯度方向，加速窄谷中的收敛
- Adam 结合动量和自适应学习率，是当前最广泛使用的默认优化器
- AdamW 将权重衰减与自适应学习率解耦，进一步提升泛化性能
- 学习率调度（特别是 Warmup + 余弦退火）对训练大模型至关重要
- 梯度裁剪是防止 RNN/Transformer 训练崩溃的简单而有效的技术
- 超参数调优应从学习率开始，逐步扩展到其他参数

卷积神经网络——让机器看懂世界

卷积神经网络 (CNN) 是计算机视觉领域的基石技术，它在图像分类、目标检测、图像分割等任务上的性能已超越人类水平。本章从卷积的数学定义出发，深入理解 CNN 的每一个核心组件——卷积、池化、通道交互——并分别用 NumPy 从零实现卷积和 PyTorch 构建完整的 CNN 训练管线。本章包含大量 Python 代码示例和可视化，是全书代码最密集的一章。

§9.1
视觉问题的本质

9.1.1 全连接网络的失效

考虑一张 $224 \times 224 \times 3$ 的 RGB 图像 (ImageNet 标准尺寸)。如果将其展平为 150528 维的向量并输入一个全连接网络，第一隐藏层假设有 1000 个神经元：

$$\text{第一层权重参数量} = 150528 \times 1000 \approx 1.5 \times 10^8$$

仅第一层就有 1.5 亿个参数。这导致三个致命问题：

1. 计算不可行——每次前向传播需要 1.5 亿次乘法
2. 严重过拟合——参数量远超样本量
3. 忽略空间结构——将图像展平为向量丢失了像素之间的空间关系

9.1.2 图像的三个归纳偏置

CNN 的设计基于对图像数据的三个核心归纳偏置 (inductive biases)：

- 局部性 (Locality) —— 图像的语义信息存在于邻近像素构成的局部区域中，远处的像素通常与当前像素无关
- 平移等变性 (Translation Equivariance) —— 无论一只猫出现在图像的左上角还是右下角，用于检测猫的滤波器应该在整张图上都有效
- 层次化组合 (Hierarchical Composition) —— 边缘组成纹理，纹理组成部件，部件组成物体

全连接

卷积（局部）

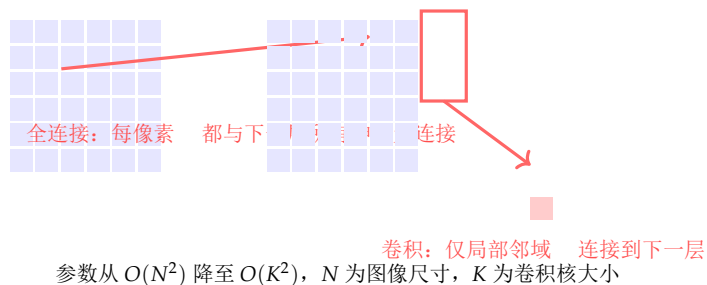


图 9.1: 全连接（左）将每个像素独立连接到所有隐藏神经元，参数爆炸且丢失空间结构。卷积（右）仅处理局部邻域，参数共享，保留空间信息

§ 9.2

卷积运算的数学定义

9.2.1 一维卷积

给定两个函数 f 和 g ，它们的卷积定义为：

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

在离散域中（深度学习均为离散情况）：

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] g[n - m]$$

在深度学习中，我们使用的是**互相关**（cross-correlation），但习惯上仍称为“卷积”。两者的区别仅在于卷积核是否翻转 180 度——由于卷积核是可学习的参数，翻转与否不影响最终学习结果。

卷积与互相关

深度学习中实际使用的“卷积”操作是**互相关**： $(f \star k)[i] = \sum_m f[i + m] k[m]$ 。它与数学卷积的区别是 k 不翻转。因为 k 由梯度下降学习得到，翻转与否无关紧要。

9.2.2 二维卷积——图像处理的核心

对于输入图像 $\mathbf{X} \in \mathbb{R}^{H \times W}$ 和卷积核 $\mathbf{K} \in \mathbb{R}^{k_h \times k_w}$ ，二维卷积（互相关）的输出为：

$$\mathbf{Y}[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \mathbf{X}[i + m, j + n] \cdot \mathbf{K}[m, n]$$

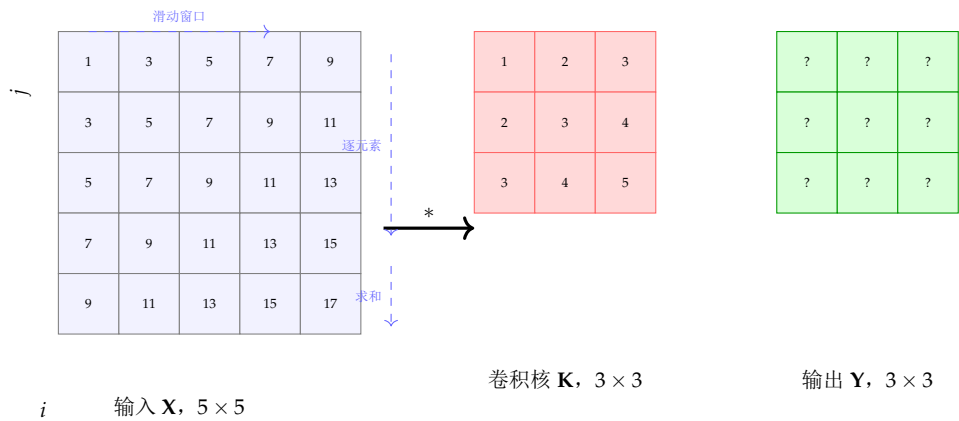


图 9.2: 二维卷积的可视化。3 × 3 卷积核在 5 × 5 输入上以步长 1 滑动，每个位置计算核与局部区域的内积，得到 3 × 3 的特征图

9.2.3 多通道卷积

真实图像具有多个通道（如 RGB 三通道）。对于输入 $\mathbf{X} \in \mathbb{R}^{C_{in} \times H \times W}$ 和卷积核 $\mathbf{K} \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$ ，多通道卷积为：

$$\mathbf{Y}[o, i, j] = \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \mathbf{X}[c, i+m, j+n] \cdot \mathbf{K}[o, c, m, n] + \mathbf{b}[o]$$

其中 o 是输出通道的索引。直观理解：每个输出通道的卷积核是一组（ C_{in} 个） $k_h \times k_w$ 的滤波器，分别作用于输入的每个通道后求和，再加上该输出通道的偏置。

9.2.4 步长与填充

步长（stride）控制卷积核在输入上的滑动步距：

$$H_{out} = \left\lfloor \frac{H_{in} - k_h}{s} + 1 \right\rfloor$$

填充（padding）在输入的四周添加额外的行/列（通常填 0），以控制输出尺寸：

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k_h}{s} + 1 \right\rfloor$$

当 $s = 1$ 且 $p = (k - 1)/2$ 时（kernel_size 为奇数），输出尺寸与输入相同——称为“same” padding。这是现代 CNN 设计中的标准配置（如 ResNet 中的 3 × 3 卷积）。

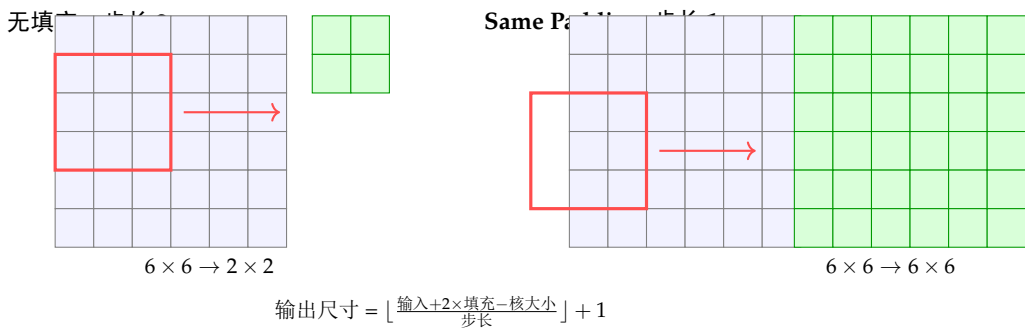


图 9.3: 步长和填充对输出尺寸的影响。Same Padding 使输入输出尺寸保持一致

§ 9.3

用 NumPy 从零实现二维卷积

理解卷积的最直接方式是亲手实现它。以下代码不使用任何深度学习框架，纯 NumPy 实现多通道二维卷积：

NumPy 实现完整的 conv2d

```

1 import numpy as np
2
3 def conv2d_numpy(x, kernel, bias=None, stride=1, padding=0):
4     """
5     纯NumPy二维卷积（互相关）实现。
6
7     参数：
8         x: 输入，shape (C_in, H, W)
9         kernel: 卷积核，shape (C_out, C_in, kH, kW)
10        bias: 偏置，shape (C_out,) 或 None
11        stride: 步长 (int)
12        padding: 填充 (int)
13
14    返回：
15        output: shape (C_out, H_out, W_out)
16    """
17    C_in, H, W = x.shape
18    C_out, C_in_k, kH, kW = kernel.shape
19    assert C_in == C_in_k, "Input channels must match kernel channels"
20
21    # 添加零填充
22    if padding > 0:
23        x_padded = np.pad(x, ((0,0), (padding,padding),
24                               (padding,padding)),
25                           mode='constant', constant_values=0)
26    else:
27        x_padded = x
28
29    # 计算输出尺寸
30    H_out = (H + 2 * padding - kH) // stride + 1
31    W_out = (W + 2 * padding - kW) // stride + 1
32    output = np.zeros((C_out, H_out, W_out))
33
34    # 卷积循环（理解原理用，生产环境用im2col或FFT）
35    for o in range(C_out):
36        for i in range(H_out):
37            for j in range(W_out):
38                h_start = i * stride
39                w_start = j * stride
40                # 提取局部区？
41                patch = x_padded[:,
42                                h_start:h_start + kH,
43                                w_start:w_start + kW] # (C_in, kH, kW)
44                # 内积 + 偏置
45                output[o, i, j] = np.sum(
46                    patch * kernel[o]
47                ) + (bias[o] if bias is not None else 0)
48
49    return output
50
51 # ===== 测试：边缘检测 =====
52 # 创建测试图像（6x6渐变）
53 x = np.arange(36, dtype=np.float32).reshape(1, 6, 6)
54 print("输入图像 (1, 6, 6):\n", x[0])

```

9.3.1 卷积的矩阵乘法实现——`im2col`

上述三层嵌套循环便于理解但效率极低。现代卷积实现使用 `im2col` 技巧将卷积转化为高效的矩阵乘法：

im2col —— 将卷积转化为矩阵乘法

```

1 import numpy as np
2
3 def im2col(x, kH, kW, stride=1, padding=0):
4     """
5     将图像展开为列矩阵，每个局部窗口变为一个列向量。
6
7     参数：
8         x: (C_in, H, W)
9     返回：
10        cols: (C_in * kH * kW, H_out * W_out)
11    """
12    C_in, H, W = x.shape
13
14    if padding > 0:
15        x = np.pad(x, ((0,0), (padding,padding),
16                        (padding,padding)),
17                    mode='constant')
18
19    H_out = (H + 2*padding - kH) // stride + 1
20    W_out = (W + 2*padding - kW) // stride + 1
21
22    cols = np.zeros((C_in * kH * kW, H_out * W_out))
23
24    col_idx = 0
25    for i in range(0, H_out * stride, stride):
26        for j in range(0, W_out * stride, stride):
27            patch = x[:, i:i+kH, j:j+kW]          # (C_in, kH, kW)
28            cols[:, col_idx] = patch.reshape(-1)
29            col_idx += 1
30
31    return cols, H_out, W_out
32
33 def conv2d_im2col(x, kernel, bias=None, stride=1, padding=0):
34     """
35     使用 im2col 的快速卷？ """
36    C_in, H, W = x.shape
37    C_out, C_in_k, kH, kW = kernel.shape
38
39    # 将卷积核展平为 (C_out, C_in*kH*kW)
40    kernel_flat = kernel.reshape(C_out, -1)
41
42    # im2col
43    cols, H_out, W_out = im2col(x, kH, kW, stride, padding)
44
45    # 矩阵乘法: (C_out, C_in*kH*kW) @ (C_in*kH*kW, H_out*W_out)
46    out_flat = kernel_flat @ cols                # (C_out, H_out*W_out)
47
48    output = out_flat.reshape(C_out, H_out, W_out)
49    if bias is not None:
50        output += bias.reshape(-1, 1, 1)
51    return output
52
53 # ===== 性能对比 =====
54 import time

```


§ 9.4 池化层——降采样与平移不变性

池化（Pooling）层在特征图的每个局部窗口内执行聚合操作，降低空间分辨率的同时保留重要信息。

9.4.1 最大池化与平均池化

$$\text{MaxPool: } Y[i, j] = \max_{m, n \in \text{window}} X[i \cdot s + m, j \cdot s + n]$$

$$\text{AvgPool: } Y[i, j] = \frac{1}{k^2} \sum_{m, n \in \text{window}} X[i \cdot s + m, j \cdot s + n]$$

池化的作用：

- **降采样**——将特征图的空间尺寸缩小，减少后续层的计算量
- **增大感受野**——同样的卷积核在更小的特征图上覆盖相对更大的输入范围
- **局部平移不变性**——小的平移不会改变池化输出（MaxPool 对小扰动鲁棒）
- **特征压缩**——保留最显著的特征响应，丢弃冗余信息

NumPy 实现最大池化

```

1 import numpy as np
2
3 def maxpool2d_numpy(x, kernel_size=2, stride=None):
4     """
5     纯NumPy最大池化
6
7     参数:
8         x: (C, H, W)
9         kernel_size: 池化窗口大小
10        stride: 步长 (默认 = kernel_size)
11    """
12    if stride is None:
13        stride = kernel_size
14
15    C, H, W = x.shape
16    H_out = (H - kernel_size) // stride + 1
17    W_out = (W - kernel_size) // stride + 1
18
19    output = np.zeros((C, H_out, W_out))
20
21    for c in range(C):
22        for i in range(H_out):
23            for j in range(W_out):
24                h_start = i * stride
25                w_start = j * stride
26                window = x[c,
27                           h_start:h_start + kernel_size,
28                           w_start:w_start + kernel_size]
29                output[c, i, j] = window.max()
30
31    return output
32
33 # 测试
34 feat_map = np.array([[
35     [1, 3, 2, 4],
36     [5, 6, 8, 7],
37     [3, 2, 1, 0],
38     [9, 5, 4, 2]
39 ]], dtype=np.float32)
40
41 print("输入特征 ? (1,4,4):\n", feat_map[0])
42 print("\nMaxPool (2x2, stride=2):\n",
43       maxpool2d_numpy(feat_map, 2, 2)[0])
44
45 # PyTorch 验证
46 import torch
47 import torch.nn.functional as F
48 x_t = torch.tensor(feat_map).unsqueeze(0)
49 out_torch = F.max_pool2d(x_t, kernel_size=2, stride=2)
50 print("\nPyTorch 输出:\n", out_torch.squeeze().numpy())

```

§ 9.5

PyTorch CNN 完整训练管线

下面的代码展示了一个完整的 CNN 分类器——从数据加载到训练、评估和推理——用于 CIFAR-10 图像分类。

完整的 PyTorch CNN 训练管线 (CIFAR-10)

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 from torch.utils.data import DataLoader
6 import torchvision
7 import torchvision.transforms as transforms
8 import matplotlib.pyplot as plt
9 import numpy as np
10 from tqdm import tqdm
11
12 # ===== 数据增强与标准化 =====
13 train_transform = transforms.Compose([
14     transforms.RandomCrop(32, padding=4),      # 随机裁 ?      transforms.
15     transforms.RandomHorizontalFlip(),         # 随机水平翻转
16     transforms.ColorJitter(0.1, 0.1, 0.1),     # 颜色抖动
17     transforms.ToTensor(),
18     transforms.Normalize(
19         (0.4914, 0.4822, 0.4465),              # CIFAR-10 均值
20         (0.2470, 0.2435, 0.2616)              # CIFAR-10 标准 ?      )
21 ])
22
23 test_transform = transforms.Compose([
24     transforms.ToTensor(),
25     transforms.Normalize(
26         (0.4914, 0.4822, 0.4465),
27         (0.2470, 0.2435, 0.2616)
28     )
29 ])
30
31 train_data = torchvision.datasets.CIFAR10(
32     root='./data', train=True,
33     download=True, transform=train_transform
34 )
35
36 test_data = torchvision.datasets.CIFAR10(
37     root='./data', train=False,
38     download=True, transform=test_transform
39 )
40
41 train_loader = DataLoader(train_data, batch_size=128,
42                             shuffle=True, num_workers=2,
43                             pin_memory=True)
44
45 test_loader = DataLoader(test_data, batch_size=100,
46                             shuffle=False, num_workers=2)
47
48 # CIFAR-10 类别
49 classes = ('plane', 'car', 'bird', 'cat', 'deer',
50            'dog', 'frog', 'horse', 'ship', 'truck')
51
52 # ===== CNN 模型定义 =====
53 class CIFAR10CNN(nn.Module):
54     123
55     def __init__(self, num_classes=10):
56         super().__init__()
57         # Block 1: 3 -> 32 通道

```

9.5.1 感受野的堆叠增长

一个重要的设计原则：两个 3×3 卷积堆叠的感受野等价于一个 5×5 卷积，但参数量更少（ $2 \times 3^2 < 5^2$ ），且中间有非线性激活。这正是 VGG 网络的设计哲学。

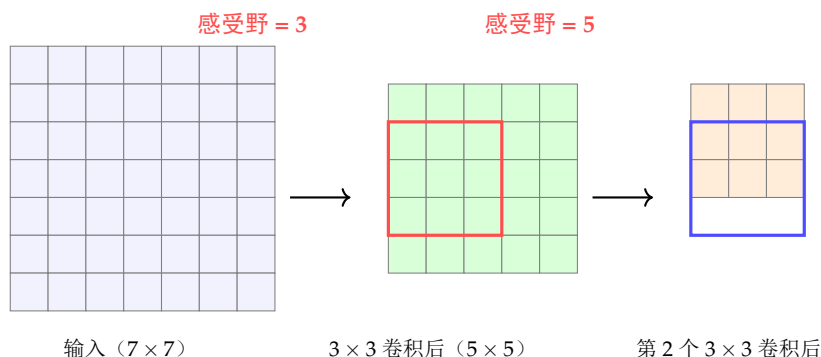


图 9.4: 两个 3×3 卷积堆叠的感受野增长。第一个卷积的感受野为 3，第二个卷积作用在第一个的特征图上，感受野扩展为 5。参数量为 $2 \times 3^2 C^2$ ，而单个 5×5 卷积需要 $5^2 C^2$ 个参数

§ 9.6

转置卷积——从低分辨率到高分辨率

9.6.1 为什么需要上采样

在图像分割和图像生成任务中，我们需要将低分辨率的特征图上采样回原始分辨率。转置卷积 (Transpose Convolution，也称反卷积) 是实现上采样的主要方法之一。

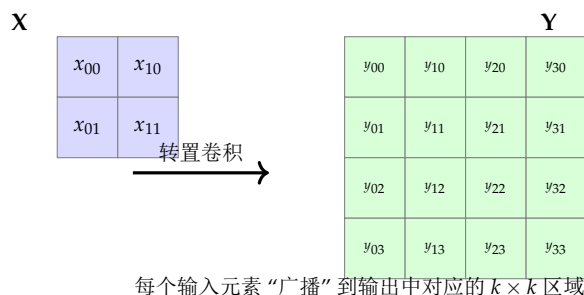


图 9.5: 转置卷积的概念。 2×2 输入经 3×3 转置卷积核上采样到 4×4 (stride=2, padding=1)

9.6.2 转置卷积的数学原理

转置卷积是标准卷积的“反向”操作：标准卷积将输入展平为列向量 $\mathbf{x}$ ，卷积核展为行矩阵 $\mathbf{C}$ ，输出 $\mathbf{y} = \mathbf{C}\mathbf{x}$ 。转置卷积使用 $\mathbf{C}^T$ 作为变换矩阵，将低分辨率特征图上采样：

$$\mathbf{y}_{\text{trans}} = \mathbf{C}^T \mathbf{x}_{\text{low}}$$

PyTorch 转置卷积示例

```

1 import torch
2 import torch.nn as nn
3
4 # 标准卷积：下采 ? conv = nn.Conv2d(3, 64, kernel_size=3, stride=2, padding=1)
5 x = torch.randn(1, 3, 64, 64)
6 out = conv(x)
7 print(f"标准卷积: {x.shape} -> {out.shape}") # (1,3,64,64) -> (1,64,32,32)
8
9 # 转置卷积：上采 ? tconv = nn.ConvTranspose2d(64, 3, kernel_size=4,
10                                           stride=2, padding=1)
11 x_up = tconv(out)
12 print(f"转置卷积: {out.shape} -> {x_up.shape}") # (1,64,32,32) -> (1,3,64,64)
13
14 # U-Net 风格的上采样模块
15 class UpBlock(nn.Module):
16     def __init__(self, in_channels, out_channels):
17         super().__init__()
18         self.up = nn.ConvTranspose2d(
19             in_channels, out_channels,
20             kernel_size=2, stride=2
21         )
22         self.conv = nn.Sequential(
23             nn.Conv2d(out_channels * 2, out_channels, 3,
24                     padding=1),
25             nn.BatchNorm2d(out_channels),
26             nn.ReLU(inplace=True)
27         )
28
29     def forward(self, x, skip):
30         x = self.up(x)
31         x = torch.cat([x, skip], dim=1) # 跳跃连接
32         return self.conv(x)

```

§9.7

特征可视化——理解 CNN 看到了什么

9.7.1 卷积核可视化

第一层卷积核通常学习类似 Gabor 滤波器的边缘和颜色检测器：

卷积核可视化

```

1 import torch
2 import torch.nn as nn
3 import torchvision
4 import matplotlib.pyplot as plt
5 import numpy as np
6
7 def visualize_kernels(model, layer_name='conv1'):
8     """可视化指定卷积层的所有卷积核"""
9     layer = dict(model.named_modules())[layer_name]
10    kernels = layer.weight.data.cpu().numpy()
11
12    # kernels shape: (C_out, C_in, kH, kW)
13    C_out, C_in, kH, kW = kernels.shape
14
15    # 归一化到 [0,1]
16    kernels = (kernels - kernels.min()) / \
17              (kernels.max() - kernels.min() + 1e-8)
18
19    # 显示前 64 个卷积核
20    n_display = min(C_out, 64)
21    n_cols = 8
22    n_rows = (n_display + n_cols - 1) // n_cols
23
24    fig, axes = plt.subplots(n_rows, n_cols,
25                             figsize=(12, 2 * n_rows))
26    axes = axes.flatten()
27
28    for i in range(n_display):
29        # 显示第一个输入通道的卷积核
30        if C_in == 3:
31            axes[i].imshow(kernels[i].transpose(1, 2, 0))
32        else:
33            axes[i].imshow(kernels[i, 0], cmap='gray')
34        axes[i].axis('off')
35
36    for i in range(n_display, len(axes)):
37        axes[i].axis('off')
38
39    plt.suptitle(f'{layer_name} 卷积核可视化 '
40                f' (共{C_out}个, 显示{n_display}个)',
41                fontsize=14)
42    plt.tight_layout()
43    plt.show()
44
45 # 使用示例
46 # model = trained_cnn_model
47 # visualize_kernels(model, 'conv1')

```

9.7.2 特征图可视化

观察中间层的激活模式，了解网络在不同深度提取了什么样的特征：

特征图可视化

```

1 import torch
2 import torch.nn as nn
3 import matplotlib.pyplot as plt
4
5 class FeatureExtractor:
6     """提取并可视化CNN中间层的特征图"""
7     def __init__(self, model, layer_names):
8         self.model = model
9         self.features = {}
10        self.hooks = []
11
12        for name in layer_names:
13            layer = dict(model.named_modules())[name]
14            hook = layer.register_forward_hook(
15                self._hook_fn(name)
16            )
17            self.hooks.append(hook)
18
19        def _hook_fn(self, name):
20            def hook(module, input, output):
21                self.features[name] = output.detach()
22            return hook
23
24        def remove_hooks(self):
25            for hook in self.hooks:
26                hook.remove()
27
28        def visualize(self, image, layer_name, max_channels=16):
29            self.model.eval()
30            with torch.no_grad():
31                _ = self.model(image.unsqueeze(0))
32
33            feat = self.features[layer_name][0] # (C, H, W)
34            C = min(feat.shape[0], max_channels)
35
36            n_cols = 4
37            n_rows = (C + n_cols - 1) // n_cols
38            fig, axes = plt.subplots(n_rows, n_cols,
39                                    figsize=(12, 3 * n_rows))
40            axes = axes.flatten()
41
42            for i in range(C):
43                axes[i].imshow(feat[i].cpu().numpy(),
44                               cmap='viridis')
45                axes[i].axis('off')
46                axes[i].set_title(f'Ch {i}', fontsize=8)
47
48            for i in range(C, len(axes)):
49                axes[i].axis('off')
50
51            plt.suptitle(f'{layer_name} 特征? '
52                        f'(shape: {feat.shape})', fontsize=14)
53            plt.tight_layout()
54            plt.show()

```

9.7.3 Grad-CAM——类激活映射

Grad-CAM 利用目标类别的梯度在最后一个卷积层的特征图上的加权平均，生成类别的空间定位热力图：

Grad-CAM 实现

```

1 import torch
2 import torch.nn.functional as F
3 import cv2
4 import numpy as np
5
6 class GradCAM:
7     def __init__(self, model, target_layer):
8         self.model = model
9         self.target_layer = target_layer
10        self.gradients = None
11        self.activations = None
12
13        # 注册 forward hook 获取激活 ?          target_layer.
14        register_forward_hook(
15            self._save_activation
16        )
17        # 注册 backward hook 获取梯 ?          target_layer.
18        register_backward_hook(
19            self._save_gradient
20        )
21
22    def _save_activation(self, module, inp, out):
23        self.activations = out.detach()
24
25    def _save_gradient(self, module, grad_in, grad_out):
26        self.gradients = grad_out[0].detach()
27
28    def generate(self, input_image, target_class=None):
29        self.model.eval()
30        output = self.model(input_image)
31
32        if target_class is None:
33            target_class = output.argmax(1).item()
34
35        # 反向传播获取梯 ?          self.model.zero_grad()
36        one_hot = torch.zeros_like(output)
37        one_hot[0, target_class] = 1
38        output.backward(gradient=one_hot, retain_graph=True)
39
40        # 全局平均池化梯度得到权 ?          weights = self.gradients.mean(dim
41            =(2, 3),
42            keepdim=True) # (1, C, 1, 1)
43
44        # 加权组合特征 ?          cam = (weights * self.activations).sum(dim=1,
45            keepdim=True)
46        cam = F.relu(cam) # 只关注对类别有正向影响的区域
47
48        # 归一化
49        cam = cam - cam.min()
50        cam = cam / (cam.max() + 1e-8)
51
52        130
53        return cam.squeeze().cpu().numpy()
54
55    def overlay(self, input_image, cam, alpha=0.5):
56        """将热图叠加到原图上"""

```

§ 9.8

CNN 设计中的核心原则

9.8.1 感受野的指数增长

每层池化将空间分辨率减半，但卷积核的感受野在输入图上指数级增长。这是 CNN 能够捕捉大范围语义依赖的关键。

$$\text{感受野}_l = \text{感受野}_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

9.8.2 通道数递增，空间分辨率递减

现代 CNN 的设计遵循一个经典模式：

- 随深度增加，空间分辨率下降（通过池化或步长为 2 的卷积）
- 随深度增加，通道数增加（从 3 到 64、128、256、512）

这种设计将空间信息“交换”为语义信息——浅层的大尺寸特征图编码精确的空间位置（适合定位），深层的小尺寸特征图编码丰富的语义类别信息（适合分类）。

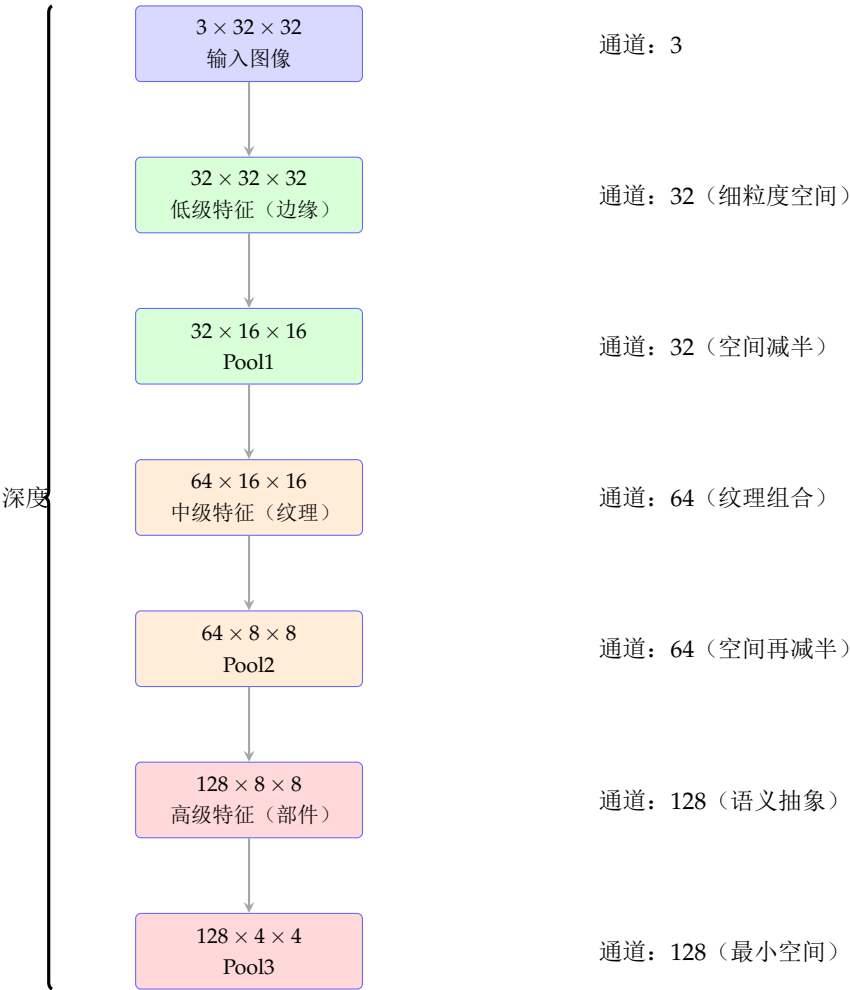


图 9.6: CNN 的特征金字塔。空间分辨率逐层降低，通道宽度逐层增加，从空间精确的低级特征过渡到语义丰富的高级特征

9.8.3 1×1 卷积——通道间的信息交换

1×1 卷积看起来像一个点（没有空间范围），但它在通道维度上执行完整的线性变换：

$$Y[o, i, j] = \sum_c K[o, c] \cdot X[c, i, j] + b[o]$$

1×1 卷积的用途：

- **改变通道数**：不改变空间尺寸的情况下调整通道维度（升降维）
- **跨通道信息融合**：学习通道之间的线性组合
- **减少计算量**：在 3×3 或 5×5 卷积前先用 1×1 降维（bottleneck 结构）
- **增加非线性**：配合 ReLU，在不增加过多参数的情况下提升网络表达能力

9.8.4 空洞卷积——扩大感受野而不增加参数

空洞卷积（Dilated / Atrous Convolution）在标准卷积核的元素之间插入“空洞”（零值间隙），使得在不增加参数量和计算量的情况下成倍扩大感受野：

$$Y[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i + m \cdot d, j + n \cdot d] \cdot K[m, n]$$

其中 d 是空洞率（dilation rate）， $d = 1$ 退化为标准卷积， $d = 2$ 时 3×3 卷积的感受野等效为 5×5 。

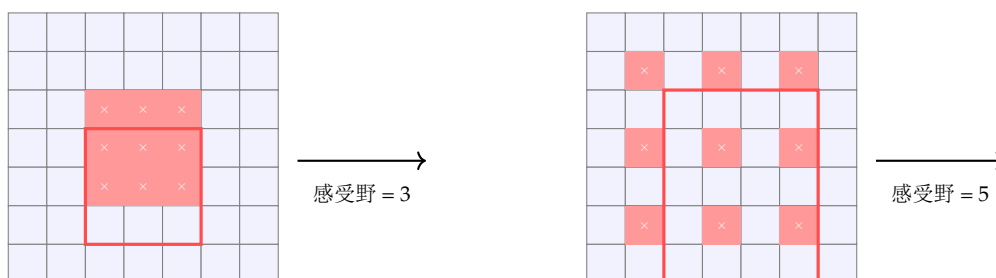


图 9.7: 空洞卷积对比。 $d = 1$ 时 3×3 卷积的感受野为 3； $d = 2$ 时同尺寸卷积核的感受野扩展为 5，但不增加参数数量和计算量。空洞卷积广泛应用于语义分割任务（如 DeepLab 系列）

空洞卷积在语义分割中尤为重要——它允许网络在不损失分辨率（不池化）的情况下获得大的感受野，同时利用多尺度的上下文信息。

§9.9 实践建议与常见陷阱

- **3×3 是标准核大小**—— 5×5 和 7×7 通常可用多个 3×3 堆叠替代，更加高效
- **BatchNorm 几乎总是有益**——在 CNN 中加入 BN 可显著加速收敛并允许更大学习率
- **用卷积替代全连接**——全连接层可用 1×1 卷积或全局平均池化替代，减少参数量
- **数据增强是免费的午餐**——对于小数据集，数据增强（翻转、裁剪、颜色抖动）的提升往往大于改进模型架构
- **检查感受野覆盖范围**——确保最深层的感受野覆盖输入图像的大部分区域

- 注意内存占用——中间特征图的存储是显存的主要消费者，可使用梯度检查点技术 `trade` 计算换内存

9.9.1 计算卷积参数的公式

给定输入尺寸 $C_{in} \times H \times W$ ，卷积核 $C_{out} \times C_{in} \times k \times k$ ：

$$\text{参数量} = C_{out} \times (C_{in} \times k^2 + 1) \quad (\text{如果 } \text{bias}=\text{True})$$

$$\text{FLOPs} \approx 2 \times C_{out} \times C_{in} \times k^2 \times H_{out} \times W_{out}$$

§9.10 本章小结

本章是全书代码最密集的一章，我们深入探讨了卷积神经网络从理论到实践的每一个层面：

- 卷积利用局部连接和参数共享将视觉任务的参数需求从 $O(H^2W^2)$ 降至 $O(k^2)$
- 用 NumPy 亲手实现的 `conv2d` 和 `im2col` 揭示了卷积的底层计算模式
- 池化层通过降采样增大感受野并引入局部平移不变性
- 完整的 PyTorch CNN 训练管线——数据增强、模块化设计、训练循环和学习率调度
- 转置卷积实现了从低分辨率特征到高分辨率输出的上采样
- 特征可视化（卷积核、特征图、Grad-CAM）是理解 CNN 内部表示的重要工具
- CNN 设计的核心模式：空间分辨率递减、通道数递增，将空间信息转化为语义信息

经典 CNN 架构——从 LeNet 到 ResNet

CNN 的发展史是一部精妙的工程进化史：LeNet-5 在 1998 年定义了卷积 + 池化 + 全连接的基本范式；AlexNet 在 2012 年开启了深度学习时代；VGG 证明了深度的力量；GoogLeNet 引入了多尺度特征融合；ResNet 用残差连接突破了深度瓶颈。本章按时间顺序逐一解析这些里程碑架构，理解每项创新背后的“为什么”。

§ 10.1

LeNet-5——CNN 的起源（1998 年）

LeNet-5 由 Yann LeCun 于 1998 年提出，用于手写数字识别（MNIST）。虽然只有约 6 万个参数，但它奠定了 CNN 的基本架构范式。

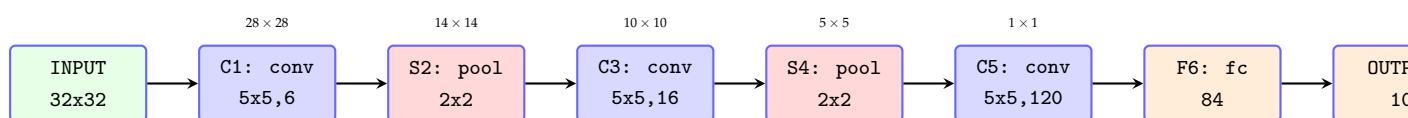


图 10.1: LeNet-5 的架构。卷积和池化交替，最后接全连接层。这个经典范式影响了此后所有 CNN 的设计

LeNet-5 的核心设计元素：

- 交替的卷积与池化——Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC，逐步降低空间分辨率并提取高层次特征
- 局部感受野——每个隐藏神经元只连接输入的一个小邻域
- 参数共享——同一卷积核在整个输入上共享参数
- 输出层使用欧氏径向基函数（RBF）而非 Softmax（后已被 Softmax 替代）

PyTorch 实现 LeNet-5

```

1 import torch.nn as nn
2 import torch.nn.functional as F
3
4 class LeNet5(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7         # 卷积部 ?          self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
8         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
9         self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
10        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
11        self.conv3 = nn.Conv2d(16, 120, kernel_size=5)
12
13        # 全连接部 ?          self.fc1 = nn.Linear(120, 84)
14        self.fc2 = nn.Linear(84, num_classes)
15
16        def forward(self, x):
17            x = self.pool1(F.relu(self.conv1(x)))    # -> 6x14x14
18            x = self.pool2(F.relu(self.conv2(x)))    # -> 16x5x5
19            x = F.relu(self.conv3(x))                # -> 120x1x1
20            x = x.view(x.size(0), -1)               # 展平
21            x = F.relu(self.fc1(x))                  # -> 84
22            x = self.fc2(x)                          # -> 10
23            return x
24
25 model = LeNet5(num_classes=10)
26 # 测试前向传播
27 import torch
28 x = torch.randn(1, 1, 32, 32)
29 out = model(x)
30 print(f"LeNet-5: {x.shape} -> {out.shape}")
31 print(f"参数量: {sum(p.numel() for p in model.parameters()):,}")

```

LeNet-5 的局限：网络较浅（仅 3 层卷积），激活函数使用 Sigmoid/Tanh（非 ReLU），没有 BatchNorm 和 Dropout，仅适用于 MNIST 级别的小规模任务。

§ 10.2

AlexNet——深度学习革命的起点（2012 年）

AlexNet 在 2012 年 ImageNet 竞赛中以 top-5 错误率 15.3% 夺冠（当年第二名 26.2%），标志了深度学习视觉时代的正式开启。

AlexNet 相比 LeNet-5 的革命性创新：

- 首次使用 **ReLU**——将训练速度提升了数倍，并缓解了梯度消失
- 双 **GPU** 并行训练——网络分两半部署在两块 GTX 580 GPU 上（当时显存仅 3GB）
- 重叠池化（Overlapping Pooling）—— 3×3 窗口，步长 2，比不重叠的 2×2 稍有改进

- **局部响应归一化（LRN）**——模拟了生物神经元的侧抑制，但后来被 BatchNorm 取代
- **Dropout**——首次应用于全连接层，有效缓解过拟合
- **数据增强**——随机裁剪和水平翻转，增加训练数据的多样性

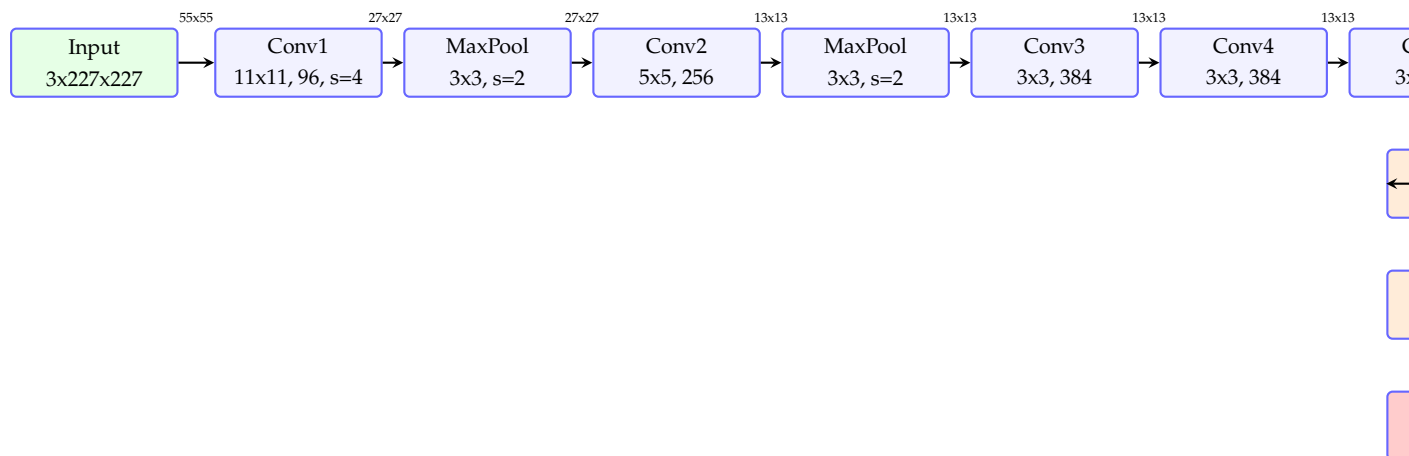


图 10.2: AlexNet 的架构。5 层卷积（前两层带池化）+ 3 层全连接。首次使用了 ReLU 和 Dropout

参数分布：AlexNet 约 6000 万参数，其中全连接层（FC6 和 FC7）占总参数的 90% 以上。这一事实直接催生了后续架构的一个设计趋势——减少甚至消除全连接层。

§ 10.3 VGG——深度的重要性（2014 年）

牛津大学 VGG 组在 2014 年做出的关键贡献不是架构上的新花样，而是极致的统一与深度：

- 所有卷积层使用统一的 3×3 卷积核（最小的非平凡尺寸）
- 池化层使用统一的 2×2 最大池化，步长 2
- 每次池化后通道数翻倍（ $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ ）
- 通过堆叠层数控制模型容量：VGG-16（16 层）和 VGG-19（19 层）

VGG 的设计原理

两个 3×3 卷积堆叠的感受野等价于一个 5×5 卷积，三个 3×3 堆叠等价于一个 7×7 。但使用小核堆叠的好处是：(1) 参数更少；(2) 中间有更多 ReLU，增加非线性；(3) 计算效率更高。

VGG 的主要缺点：

- **参数巨大**——VGG-16 有 1.38 亿参数，其中约 1.2 亿在全连接层
- **内存消耗大**——前两个 FC 层需要 470MB 的存储
- **计算开销高**——大量通道导致中间特征图占用大量显存

尽管存在这些缺点，VGG 的简单性和统一性使其成为特征提取的首选骨干网络（backbone），其预训练权重至今仍广泛用于风格迁移、图像生成等任务中。

§ 10.4

GoogLeNet / Inception——多尺度特征融合（2014 年）

GoogLeNet（与 VGG 同年）从另一个方向解决了深度网络的挑战：不再追求更深的堆叠，而是在每一层内并行使用不同尺寸的卷积核，而后再将这些多尺度特征拼接。

10.4.1 Inception 模块

Inception 模块的核心思想：网络不必在每一层决定使用哪种尺寸的卷积核，而是让网络自己学习——并行地使用 1×1 、 3×3 和 5×5 卷积以及 3×3 最大池化，然后将所有分支的输出在通道维度上拼接。

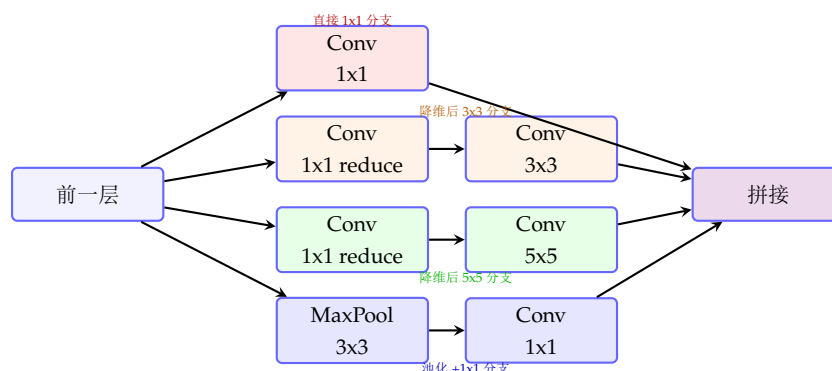


图 10.3: Inception 模块（带降维的版本）。 1×1 卷积在 3×3 和 5×5 卷积前降维，大幅减少计算量。 1×1 卷积是 Inception 经济的秘密

10.4.2 辅助分类器

GoogLeNet 在中间层添加了两个辅助分类器（auxiliary classifiers），将中间层的特征也用于分类并计算损失，提供额外的梯度信号来对抗深层网络的梯度消失。这些辅助分类器在推理时被丢弃。

简化版 Inception 模块的 PyTorch 实现

```

1 import torch
2 import torch.nn as nn
3
4 class InceptionModule(nn.Module):
5     def __init__(self, in_channels, out_1x1,
6                   red_3x3, out_3x3,
7                   red_5x5, out_5x5, out_pool):
8         super().__init__()
9         # 1x1 分支
10        self.branch1 = nn.Sequential(
11            nn.Conv2d(in_channels, out_1x1, 1),
12            nn.BatchNorm2d(out_1x1),
13            nn.ReLU(inplace=True)
14        )
15        # 1x1 -> 3x3 分支
16        self.branch2 = nn.Sequential(
17            nn.Conv2d(in_channels, red_3x3, 1),
18            nn.BatchNorm2d(red_3x3),
19            nn.ReLU(inplace=True),
20            nn.Conv2d(red_3x3, out_3x3, 3, padding=1),
21            nn.BatchNorm2d(out_3x3),
22            nn.ReLU(inplace=True)
23        )
24        # 1x1 -> 5x5 分支 (常替 ? 两个 3x3 ?)
25        self.branch3 = nn.Sequential(
26            nn.Conv2d(in_channels, red_5x5, 1),
27            nn.BatchNorm2d(red_5x5),
28            nn.ReLU(inplace=True),
29            nn.Conv2d(red_5x5, out_5x5, 3, padding=1),
30            nn.BatchNorm2d(out_5x5),
31            nn.ReLU(inplace=True),
32            nn.Conv2d(out_5x5, out_5x5, 3, padding=1),
33            nn.BatchNorm2d(out_5x5),
34            nn.ReLU(inplace=True)
35        )
36        # 池化分 ?
37        self.branch4 = nn.Sequential(
38            nn.MaxPool2d(3, stride=1, padding=1),
39            nn.Conv2d(in_channels, out_pool, 1),
40            nn.BatchNorm2d(out_pool),
41            nn.ReLU(inplace=True)
42        )
43
44        def forward(self, x):
45            b1 = self.branch1(x)
46            b2 = self.branch2(x)
47            b3 = self.branch3(x)
48            b4 = self.branch4(x)
49            return torch.cat([b1, b2, b3, b4], dim=1)
50
51 # 测试
52 x = torch.randn(2, 256, 14, 14)
53 module = InceptionModule(256, 64, 96, 128, 16, 32, 32)
54 out = module(x)
55 print(f"Inception: {x.shape} -> {out.shape}")

```

§ 10.5

ResNet——残差网络，突破深度极限（2015 年）

10.5.1 退化问题

VGG 证明了增加深度可以提升性能。但实验发现：当网络深度超过某个阈值后，训练误差和测试误差反而上升——这不是过拟合，而是**退化**（**degradation**）。理论上，更深的网络至少不应该比浅层网络更差（深层网络的后几层可以学习恒等映射，退化为浅层网络）。然而，SGD 难以学习恒等映射。

10.5.2 残差学习——Let the layers fit a residual mapping

ResNet 的核心洞察：与其让层直接学习期望映射 $\mathcal{H}(\mathbf{x})$ ，不如让层学习**残差** $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。最终输出为 $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。

如果恒等映射是最优的，优化器只需将残差分支的权重推向零即可（比学习恒等映射容易得多）。

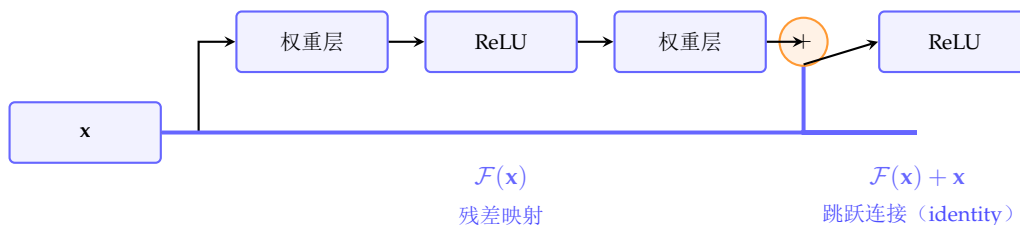


图 10.4: 残差块（Residual Block）的结构。跳跃连接（shortcut connection）将输入 $\mathbf{x}$ 直接加到残差分支的输出上，使得梯度可以通过这条“高速公路”无损地传递到浅层

10.5.3 残差连接解决梯度消失

对于残差块 $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$ ，反向传播时梯度为：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right)$$

单位矩阵 $\mathbf{I}$ 的存在保证了梯度至少有一条直接的通路到达任意浅层——即使 $\partial \mathcal{F} / \partial \mathbf{x}$ 很小，梯度也不会完全消失。这是 ResNet 能够训练 152 层甚至 1000+ 层网络的根本原因。

10.5.4 Bottleneck 残差块

对于极深网络（ResNet-50/101/152），使用 **bottleneck** 结构减少计算量：

$$\text{Conv1x1}(C_{\text{in}} \rightarrow C_{\text{mid}}) \rightarrow \text{Conv3x3}(C_{\text{mid}} \rightarrow C_{\text{mid}}) \rightarrow \text{Conv1x1}(C_{\text{mid}} \rightarrow C_{\text{out}})$$

其中 $C_{\text{mid}} = C_{\text{out}}/4$ 。 1×1 卷积先降维再升维，使得 3×3 卷积在低维空间执行，大幅减少参数数量和计算量。

ResNet 残差块和完整网络的 PyTorch 实现

```

1 import torch
2 import torch.nn as nn
3
4 class BasicBlock(nn.Module):
5     """ResNet-18/34 的残差块"""
6     expansion = 1
7
8     def __init__(self, in_channels, out_channels,
9                 stride=1, downsample=None):
10         super().__init__()
11         self.conv1 = nn.Conv2d(in_channels, out_channels,
12                                3, stride, 1, bias=False)
13         self.bn1 = nn.BatchNorm2d(out_channels)
14         self.conv2 = nn.Conv2d(out_channels, out_channels,
15                                3, 1, 1, bias=False)
16         self.bn2 = nn.BatchNorm2d(out_channels)
17         self.downsample = downsample
18         self.relu = nn.ReLU(inplace=True)
19
20     def forward(self, x):
21         identity = x
22
23         out = self.relu(self.bn1(self.conv1(x)))
24         out = self.bn2(self.conv2(out))
25
26         if self.downsample is not None:
27             identity = self.downsample(x)
28
29         out += identity
30         out = self.relu(out)
31         return out
32
33 class Bottleneck(nn.Module):
34     """ResNet-50/101/152 的 Bottleneck 残差块"""
35     expansion = 4
36
37     def __init__(self, in_channels, out_channels,
38                 stride=1, downsample=None):
39         super().__init__()
40         mid_channels = out_channels
41
42         self.conv1 = nn.Conv2d(in_channels, mid_channels,
43                                1, bias=False)
44         self.bn1 = nn.BatchNorm2d(mid_channels)
45         self.conv2 = nn.Conv2d(mid_channels, mid_channels,
46                                3, stride, 1, bias=False)
47         self.bn2 = nn.BatchNorm2d(mid_channels)
48         self.conv3 = nn.Conv2d(mid_channels,
49                                out_channels * self.expansion,
50                                1, bias=False)
51         self.bn3 = nn.BatchNorm2d(
52             out_channels * self.expansion)
53         self.relu = nn.ReLU(inplace=True)
54         self.downsample = downsample

```

10.5.5 ResNet 架构配置表

层名	ResNet-18	ResNet-34	ResNet-50	ResNet-152
conv1	$7 \times 7, 64, \text{stride } 2$ $3 \times 3 \text{ max pool, stride } 2$			
conv2_x	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
输出	average pool, 1000-d fc, softmax			
参数量	11.7M	21.8M	25.6M	60.2M

§ 10.6

DenseNet——连接一切（2017 年）

DenseNet 将 ResNet 的“跳跃连接”思想推向极致：每一层的输出都连接到所有后续层的输入。对于第 l 层：

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{l-1}])$$

其中 $[\dots]$ 表示在通道维度上拼接（而非相加）。这一设计带来了三个关键优势：

- 减轻梯度消失——所有层直接接收来自损失函数的监督信号
- 特征重用——浅层特征被所有后续层直接使用，无需逐层传递
- 大幅减少参数——由于特征重用，每层的输出通道数可以很小（12 或 32），总参数量反而少于 ResNet

§ 10.7

轻量化架构——MobileNet 与 EfficientNet

10.7.1 深度可分离卷积（Depthwise Separable Convolution）

MobileNet 的核心创新是将标准卷积分解为两步：

1. 深度卷积（Depthwise Conv）：每个输入通道使用独立的 3×3 卷积核
2. 逐点卷积（Pointwise Conv）：用 1×1 卷积融合跨通道信息

这一分解将计算量从 $O(k^2 C_{in} C_{out} HW)$ 降至 $O(k^2 C_{in} HW + C_{in} C_{out} HW)$ ，约为原来的 $1/C_{out} + 1/k^2$ 。

深度可分离卷积

```

1 import torch
2 import torch.nn as nn
3
4 class DepthwiseSeparableConv(nn.Module):
5     def __init__(self, in_channels, out_channels,
6                 kernel_size=3, stride=1):
7         super().__init__()
8         self.depthwise = nn.Conv2d(
9             in_channels, in_channels, kernel_size,
10            stride, padding=kernel_size//2,
11            groups=in_channels,          # 分组卷 ?          bias=False
12        )
13        self.pointwise = nn.Conv2d(
14            in_channels, out_channels, 1,
15            bias=False
16        )
17        self.bn1 = nn.BatchNorm2d(in_channels)
18        self.bn2 = nn.BatchNorm2d(out_channels)
19        self.relu = nn.ReLU(inplace=True)
20
21    def forward(self, x):
22        x = self.relu(self.bn1(self.depthwise(x)))
23        x = self.relu(self.bn2(self.pointwise(x)))
24        return x
25
26 # 计算量对 ?in_c, out_c = 64, 128
27 conv_std = nn.Conv2d(in_c, out_c, 3, bias=False)
28 conv_ds = DepthwiseSeparableConv(in_c, out_c, 3)
29
30 def count_params(m):
31     return sum(p.numel() for p in m.parameters())
32
33 print(f"标准卷积参数: {count_params(conv_std):,}")
34 print(f"深度可分离卷积参数: {count_params(conv_ds):,}")
35 print(f"压缩比: {count_params(conv_ds)/count_params(conv_std):.2%}")

```

10.7.2 EfficientNet——复合缩放

EfficientNet 提出了**复合缩放策略**：不单独增加深度、宽度或分辨率，而是按照固定比例同时缩放三者：

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi$$

其中 $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ （确保 FLOPS 约翻倍）， ϕ 是用户指定的复合系数。这种方法在 ImageNet 上以远少于先前模型的参数达到了 state-of-the-art。

§ 10.8 架构演进路线图与设计趋势

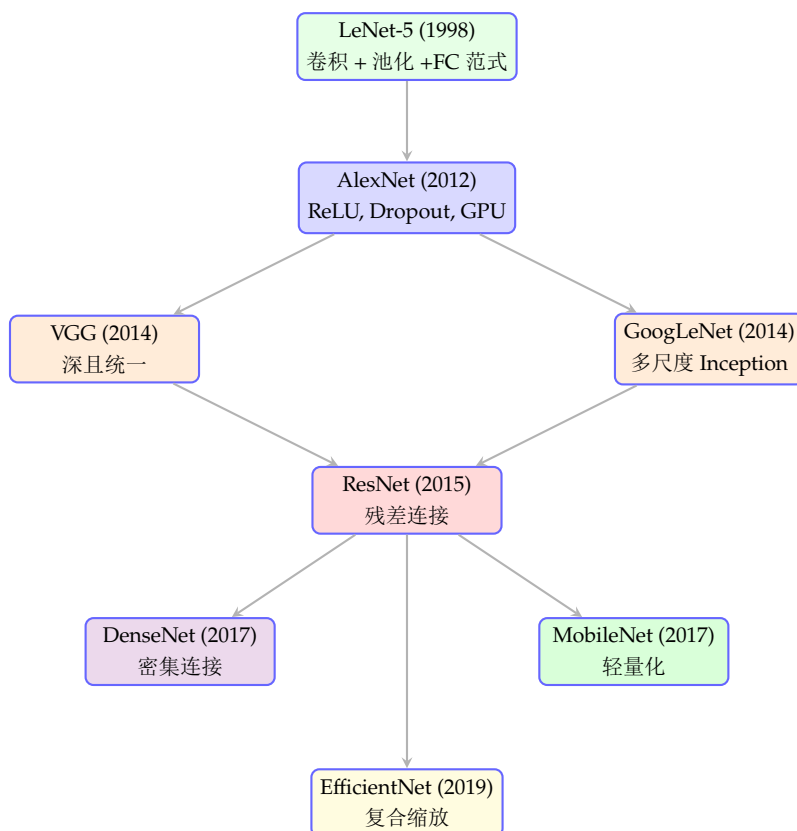


图 10.5: CNN 架构的演进路线。从 LeNet 到 AlexNet 是范式建立，从 VGG/GoogLeNet 到 ResNet 是深度突破，从 DenseNet 到 EfficientNet 是效率优化

10.8.1 设计趋势总结

1. 核尺寸趋向小且统一—— $7 \times 7 \rightarrow 5 \times 5 \rightarrow 3 \times 3$ (VGG、ResNet)，大核仅用于 stem 或由小核堆叠
2. 全连接层逐渐消失——从 FC 占 90% 参数 (AlexNet) 到全局平均池化 + 单 FC (ResNet)
3. 连接越来越密集——从串行 (VGG) 到残差 (ResNet) 到密集 (DenseNet)
4. 从追求深度到追求效率——MobileNet、EfficientNet 关注在计算预算内最大化性能
5. BatchNorm + ReLU 成为标配——几乎每个卷积层后面都跟着 BN 和 ReLU

§ 10.9 本章小结

从 LeNet-5 到 EfficientNet，CNN 架构的演进体现了深度学习设计的几个核心原则：

- 深度带来表达能力——VGG 证明了深度的重要性，ResNet 用残差连接突破了深度的瓶颈
- 多尺度增强鲁棒性——Inception 在一层内融合多尺度信息，适应不同大小的物体

- 连接密度决定信息流——从串行到残差到密集，更密集的连接使得梯度和特征流动更自由
- 效率决定实用性——深度可分离卷积和复合缩放使得 CNN 可以在移动设备上运行

CNN 擅长处理空间模式，而序列数据则需要循环神经网络（RNN）来捕捉时间依赖性。本章从基础 RNN 出发，深入长短期记忆网络（LSTM）和门控循环单元（GRU）的精妙门控设计，最终引出注意力机制——它彻底改变了对序列建模的方式，并为下一章的 Transformer 架构铺平了道路。

§ 11.1 序列建模的核心挑战

11.1.1 为什么需要特殊的网络结构

文本、语音、视频、时间序列等数据具有天然的**顺序依赖**关系——当前时刻的输出不仅依赖于当前输入，还依赖于历史信息。全连接网络和 CNN 难以处理这种可变长的时间依赖：

- **变长输入**——序列长度不固定，无法用固定大小的输入层
- **长程依赖**——句子开头的主语可能影响 100 个词后的动词形态
- **参数共享**——同样的模式可能在不同位置出现（如在任意位置的“not”都翻转句意）

11.1.2 RNN 的基本思想

RNN 的核心是一个**循环连接**——隐藏状态 $\mathbf{h}_t$ 既依赖于当前输入 $\mathbf{x}_t$ ，也依赖于前一步的隐藏状态 $\mathbf{h}_{t-1}$ ：

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

这一简单的递归定义赋予了 RNN 在理论上处理任意长序列的能力——每个时间步的信息都通过 $\mathbf{h}_t$ 传递到下一步。

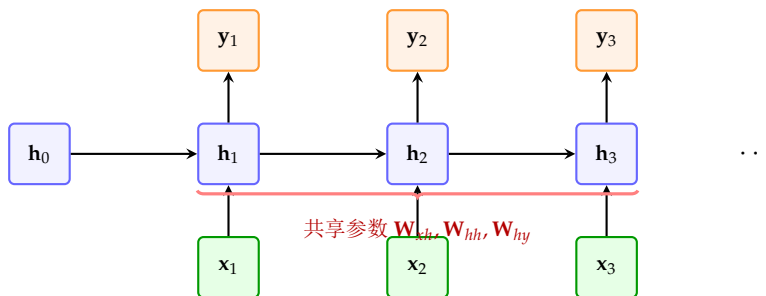


图 11.1: RNN 在时间维度上的展开。每个时间步使用相同的参数，隐藏状态 $\mathbf{h}_t$ 在时间轴上进行传递，携带历史信息

§ 11.2

循环神经网络——前向传播与反向传播

11.2.1 前向传播

Vanilla RNN 的前向传播公式：

$$\begin{aligned}\mathbf{h}_t &= \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \\ \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)\end{aligned}\quad (11.1)$$

其中 $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$, $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ 。所有时间步共享这三组参数。

11.2.2 时间反向传播 (BPTT)

RNN 的梯度需要通过时间反向传播 (Backpropagation Through Time, BPTT) 来计算。将 RNN 在时间上展开为深度为 T 的前馈网络后，应用标准反向传播。

关键是隐藏状态对前一隐藏状态的梯度：

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^\top \text{diag}(\tanh'(\mathbf{z}_t))$$

式中 $\tanh'$ 的值范围在 $[0, 1]$ 内，对于大多输入最终趋近于 0。从时间步 t 到时间步 k ($k \ll t$) 的梯度需经过 $t - k$ 次连乘：

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}$$

这揭示了 RNN 训练的核心困难：当 $\|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{i-1}}\| < 1$ 时梯度指数级衰减（消失），模型无法捕捉长程依赖；当范数大于 1 时梯度指数级增长（爆炸）。

RNN 中的梯度消失与爆炸

对于 Vanilla RNN，隐藏状态之间的雅可比矩阵 $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{i-1}}$ 的特征值若持续小于 1， $t - k$ 步后的梯度呈指数衰减。 $\tanh$ 激活函数的导数最大为 1（在输入为 0 处），实际中大多数输入落在饱和区导致导数趋近于 0。

11.2.3 RNN 的变体

多层 RNN：将多个 RNN 层堆叠，下层的隐藏状态作为上层的输入序列：

$$\mathbf{h}_t^{(l)} = \tanh(\mathbf{W}_{hh}^{(l)}\mathbf{h}_{t-1}^{(l)} + \mathbf{W}_{xh}^{(l)}\mathbf{h}_t^{(l-1)} + \mathbf{b}_h^{(l)})$$

双向 RNN：同时从前向和后向两个方向处理序列，对于句子中的单词可以同时获取其左右的上下文信息：

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$$

双向 RNN 在自然语言处理中广泛使用（如命名实体识别、词性标注），因为某位置的标注通常同时依赖左右上下文。但双向 RNN 不适用于自回归生成任务（生成时未来的 token 不可见）。

§ 11.3 LSTM——长短期记忆网络

1997年由Hochreiter和Schmidhuber提出的LSTM通过精心设计的门控机制从根本上解决了RNN的梯度消失问题。

11.3.1 LSTM 的核心思想——细胞状态

LSTM引入了细胞状态（cell state） $\mathbf{c}_t$ ——一条贯穿时间步的“传送带”，信息可以在上面畅通无阻地流动：

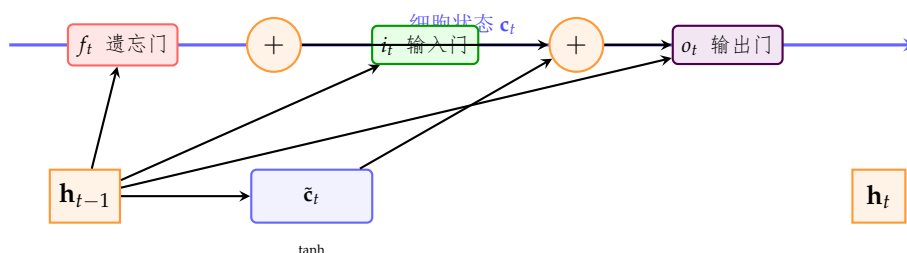


图 11.2: LSTM 的内部结构概览。细胞状态 $\mathbf{c}_t$ 如传送带贯穿各时间步，三个门控（遗忘、输入、输出）精确控制信息的选择性记忆与遗忘

11.3.2 四个门控的精确数学公式

$$\begin{aligned}
 \mathbf{f}_t &= \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f) \quad (\text{遗忘门——决定丢弃什么}) \\
 \mathbf{i}_t &= \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i) \quad (\text{输入门——决定存储什么}) \\
 \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c) \quad (\text{候选记忆——新信息}) \\
 \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{细胞状态更新}) \\
 \mathbf{o}_t &= \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o) \quad (\text{输出门——决定输出什么}) \\
 \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{隐藏状态输出})
 \end{aligned} \tag{11.2}$$

其中 σ 是 Sigmoid 函数（输出在 0 到 1 之间，充当门控的“开关”）， $\odot$ 表示逐元素乘法。

11.3.3 梯度流动——为什么 LSTM 有效

细胞状态 $\mathbf{c}_t$ 的递归更新不含 Sigmoid 或 Tanh 的非线性（门控是对 $\mathbf{c}_{t-1}$ 的逐元素缩放，而非复合函数）：

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t$$

梯度在细胞状态通路中传播时，每一时间步只乘以遗忘门的输出 $\mathbf{f}_t$ （0 ~ 1 之间的逐元素值）。如果 $\mathbf{f}_t \approx 1$ ，梯度几乎无衰减地传播。这正是 LSTM 能够捕捉数百步甚至更长距离依赖的根本原因——细胞状态提供了一条梯度高速公路，类比如 ResNet 中的跳跃连接。

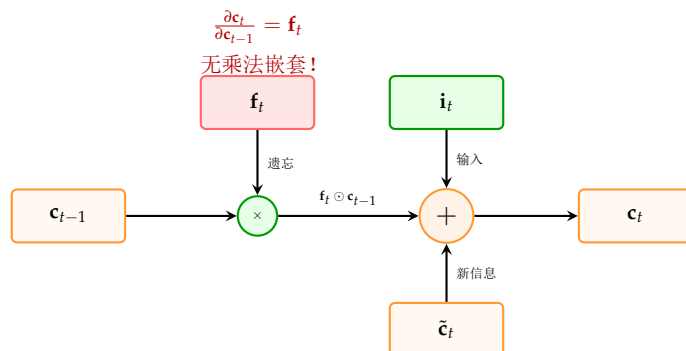


图 11.3: LSTM 细胞状态更新的关键在于：梯度在 c_t 通路上只需乘以遗忘门 f_t （而非复合函数的导数），避免了梯度消失

§ 11.4 GRU——门控循环单元

GRU (Gated Recurrent Unit) 是 LSTM 的精简版，将遗忘门和输入门合并为一个更新门，并将细胞状态和隐藏状态合并：

$$\begin{aligned}
 \mathbf{z}_t &= \sigma(\mathbf{W}_{xz}\mathbf{x}_t + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_z) \quad (\text{更新门——控制历史保留 vs 新信息引入}) \\
 \mathbf{r}_t &= \sigma(\mathbf{W}_{xr}\mathbf{x}_t + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_r) \quad (\text{重置门——控制忽略多少历史}) \\
 \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (\text{候选隐藏状态}) \\
 \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{最终隐藏状态})
 \end{aligned} \tag{11.3}$$

GRU 仅有两个门（更新门和重置门），参数量比 LSTM 少约 25%，在实践中两者的性能通常相当。更新门 $\mathbf{z}_t$ 同时扮演了 LSTM 中遗忘门和输入门的角色。当 $\mathbf{z}_t = 0$ 时，完全保留历史；当 $\mathbf{z}_t = 1$ 时，完全用新信息替换历史。

PyTorch 中的 LSTM 与 GRU

```

1 import torch
2 import torch.nn as nn
3
4 # ===== LSTM =====
5 lstm = nn.LSTM(
6     input_size=128,      # 输入特征维度
7     hidden_size=256,     # 隐藏状态维度
8     num_layers=2,        # 堆叠层数
9     batch_first=True,    # 输入 shape: (batch, seq, features)
10    bidirectional=False, # 是否双向
11    dropout=0.3           # 层间 Dropout
12)
13
14 # 输入: (batch_size, seq_len, input_size)
15 x = torch.randn(32, 50, 128)
16
17 # output: (batch, seq, hidden*directions)
18 # h_n: (num_layers*directions, batch, hidden)
19 # c_n: (num_layers*directions, batch, hidden)
20 output, (h_n, c_n) = lstm(x)
21 print(f"LSTM output: {output.shape}") # (32, 50, 256)
22 print(f"LSTM h_n: {h_n.shape}")      # (2, 32, 256)
23 print(f"LSTM c_n: {c_n.shape}")      # (2, 32, 256)
24
25 # ===== GRU =====
26 gru = nn.GRU(
27     input_size=128,
28     hidden_size=256,
29     num_layers=2,
30     batch_first=True,
31     dropout=0.3
32)
33
34 output, h_n = gru(x)
35 print(f"GRU output: {output.shape}") # (32, 50, 256)
36 print(f"GRU h_n: {h_n.shape}")      # (2, 32, 256)
37
38 # ===== 双向 LSTM =====
39 bilstm = nn.LSTM(128, 256, num_layers=2,
40                  batch_first=True, bidirectional=True)
41 output, (h_n, c_n) = bilstm(x)
42 print(f"BiLSTM output: {output.shape}") # (32, 50, 512)
43
44 # ===== 手动 LSTM (理解原理) =====
45 class LSTMCell(nn.Module):
46     def __init__(self, input_size, hidden_size):
47         super().__init__()
48         # 四个门控的线性变换合并为一个大矩阵乘法
49         self.W = nn.Linear(input_size + hidden_size,
50                             4 * hidden_size)
51
52     def forward(self, x, state):
53         h_prev, c_prev = state
54         combined = torch.cat([x, h_prev], dim=-1)

```


§ 11.5 序列到序列模型 (Seq2Seq)

Seq2Seq 模型将变长输入序列映射为变长输出序列，由**编码器**和**解码器**两个 RNN 组成。

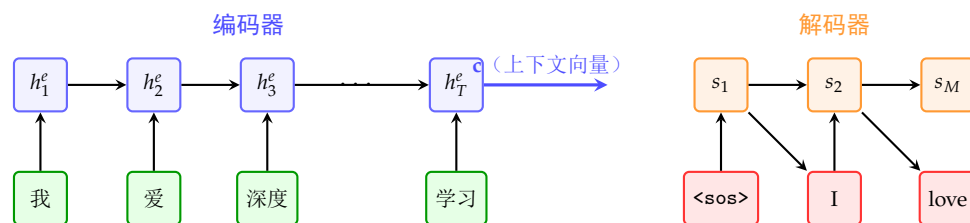


图 11.4: Seq2Seq 模型。编码器将输入序列压缩为固定长度的上下文向量 $\mathbf{c}$ ，解码器根据 $\mathbf{c}$ 自回归地生成输出序列

Seq2Seq 的信息瓶颈

基础 Seq2Seq 模型将整个输入序列压缩为一个固定长度的上下文向量 $\mathbf{c}$ 。当输入序列较长时，这一瓶颈严重限制了模型对长输入的处理能力——这就是注意力机制被引入的动机。

§ 11.6 注意力机制——从瓶颈到动态加权

11.6.1 Bahdanau 注意力 (2015 年)

Bahdanau 等人提出的注意力机制允许解码器在每一个生成步骤动态地关注输入序列中的不同位置，而非被迫使用固定的上下文向量：

$$\mathbf{c}_t = \sum_{j=1}^T \alpha_{tj} \mathbf{h}_j^e$$

其中注意力权重 α_{tj} 经 Softmax 归一化，表示了在第 t 步解码时，编码器第 j 个位置的重要性：

$$e_{tj} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_j^e), \quad \alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^T \exp(e_{tk})}$$

这里 $\mathbf{s}_{t-1}$ 是解码器的前一步隐藏状态， $\mathbf{h}_j^e$ 是编码器第 j 步的隐藏状态。注意力机制通过加性注意力 (additive attention) 计算查询 (query) 和键 (key) 之间的匹配分数。

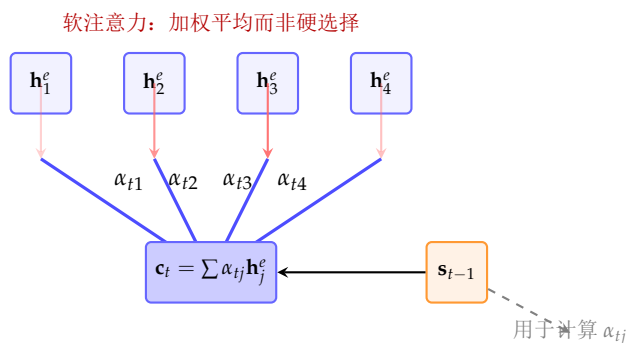


图 11.5: Bahdanau 注意力机制的示意图。解码器在每个时间步根据前一步状态 $\mathbf{s}_{t-1}$ 动态计算编码器各位的权重，生成加权的上下文向量 $\mathbf{c}_t$

11.6.2 Luong 注意力——三种评分方式

Luong 等人提出了更简洁的注意力机制，并系统对比了三种评分函数：

- 点积 (dot): $\text{score}(\mathbf{s}_t, \mathbf{h}_j) = \mathbf{s}_t^\top \mathbf{h}_j$
- 通用 (general): $\text{score}(\mathbf{s}_t, \mathbf{h}_j) = \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_j$
- 拼接 (concat): $\text{score}(\mathbf{s}_t, \mathbf{h}_j) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a [\mathbf{s}_t; \mathbf{h}_j])$

11.6.3 缩放点积注意力——通往 Transformer 的桥梁

当向量维度较大时，点积的结果会很大——因为点积值随维度 d 线性增长，导致 Softmax 进入梯度极小的区域。解决方案是除以 $\sqrt{d}$ ：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

这一缩放因子是 Transformer 中注意力机制的关键设计。在 d_k 维度下，假设查询和键的每个分量是独立的零均值单位方差随机变量，点积的均值为 0，方差为 d_k 。除以 $\sqrt{d_k}$ 将方差控制为 1。

注意力机制的 PyTorch 实现

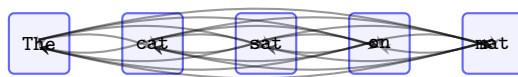
```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class BahdanauAttention(nn.Module):
7     """Bahdanau 加性注意力"""
8     def __init__(self, enc_hidden, dec_hidden, attn_dim):
9         super().__init__()
10        self.W = nn.Linear(dec_hidden, attn_dim, bias=False)
11        self.U = nn.Linear(enc_hidden, attn_dim, bias=False)
12        self.v = nn.Linear(attn_dim, 1, bias=False)
13
14    def forward(self, decoder_state, encoder_outputs):
15        """
16        decoder_state: (batch, dec_hidden)
17        encoder_outputs: (batch, seq_len, enc_hidden)
18        """
19        # (batch, attn_dim) -> (batch, 1, attn_dim)
20        dec_proj = self.W(decoder_state).unsqueeze(1)
21        # (batch, seq_len, attn_dim)
22        enc_proj = self.U(encoder_outputs)
23
24        # (batch, seq_len, 1)
25        scores = self.v(torch.tanh(dec_proj + enc_proj))
26        scores = scores.squeeze(-1)
27
28        attn_weights = F.softmax(scores, dim=-1) # (batch, seq_len)
29
30        # (batch, enc_hidden)
31        context = torch.bmm(
32            attn_weights.unsqueeze(1),
33            encoder_outputs
34        ).squeeze(1)
35        return context, attn_weights
36
37 # ===== 缩放点积注意力 =====
38 def scaled_dot_product_attention(Q, K, V, mask=None):
39     """
40     Q: (batch, ..., seq_len, d_k)
41     K: (batch, ..., seq_len, d_k)
42     V: (batch, ..., seq_len, d_v)
43     """
44     d_k = Q.size(-1)
45     scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
46
47     if mask is not None:
48         scores = scores.masked_fill(mask == 0, -1e9)
49
50     attn = F.softmax(scores, dim=-1)
51     output = attn @ V
52     return output, attn
53
54 # 测试

```

§ 11.7 自注意力——序列内部的注意力

自注意力（Self-Attention）是注意力机制的一种特殊形式：查询 **Q**、键 **K** 和值 **V** 都来自同一序列。这意味着序列中的每个位置都关注序列中的所有其他位置（包括自身），从而在单层内建立全局依赖。



每个词关注所有词（包括自身）

图 11.6: 自注意力的全连接特性。序列中的每个位置都通过注意力权重与所有其他位置建立直接联系，使得信息可以在单层内实现全局流动

自注意力相比 RNN 的核心优势：

- **恒定路径长度**——任意两个位置之间的信息传播只需 $O(1)$ 步（单层），而 RNN 需要 $O(T)$ 步
- **并行计算**——所有位置的计算相互独立，可完全并行化
- **可解释性**——注意力权重提供了位置之间依赖强度的直观可视化

其代价是：

- **计算复杂度 $O(T^2)$** ——每一层需要计算所有位置之间的注意力分数
- **记忆复杂度 $O(T^2)$** ——需要存储注意力权重矩阵
- **缺乏位置信息**——自注意力本身对位置的排列不敏感，需要额外的位置编码

§ 11.8 本章小结

本章完成了从 RNN 到注意力机制的完整探索：

- RNN 通过隐藏状态的递归传递来建模序列依赖，但梯度消失在长序列上成为致命短板
- LSTM 用细胞状态和精心设计的门控机制（遗忘门、输入门、输出门）构建了梯度的“高速公路”
- GRU 将 LSTM 的门控精简为两个门，以更少的参数实现了几乎同等的效果
- 注意力机制打破了 Seq2Seq 中的信息瓶颈——每个解码步骤可以动态关注输入的不同部分
- 缩放点积注意力将点积值除以 $\sqrt{d_k}$ ，确保 Softmax 不进入饱和区
- 自注意力在序列内部实现全局交互，是 Transformer 架构的核心组件——这就是下一章的内容

Transformer 与大规模语言模型

2017 年的“Attention Is All You Need”论文彻底改变了深度学习的格局。Transformer 摒弃了 RNN 的循环结构，完全基于注意力机制构建，以无与伦比的并行性和长程建模能力成为现代深度学习的统一架构。本章从 Transformer 的核心组件出发，深入 BERT 的双向编码和 GPT 的自回归生成，最终覆盖 Llama 和 DeepSeek 等前沿大模型的技术创新。

§ 12.1

Transformer——Attention Is All You Need

12.1.1 为什么需要 Transformer

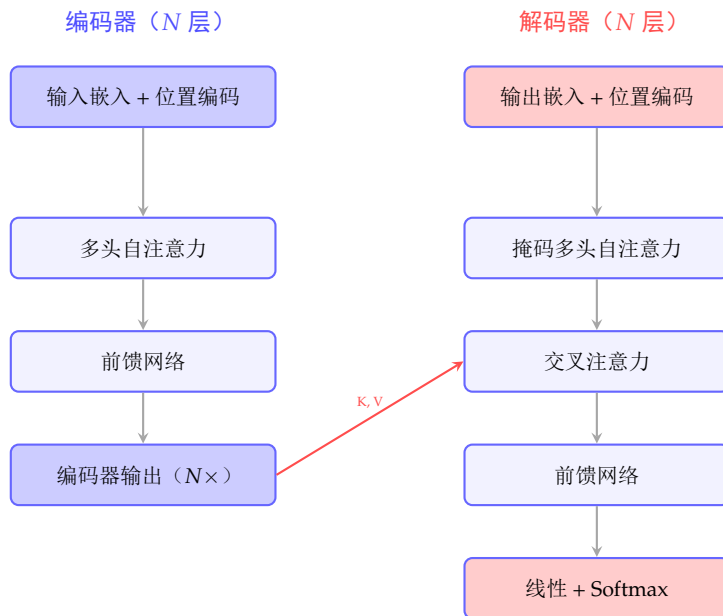
第 11 章中我们讨论了 RNN 的两个根本缺陷：

1. 顺序计算——第 t 步必须在第 $t - 1$ 步完成后才能计算，无法并行
2. 长程依赖困难——梯度需经 $O(T)$ 步反向传播，信息路径过长

Transformer 通过全注意力机制解决这两个问题：每个位置直接关注所有其他位置，信息传播路径长度为 $O(1)$ ，所有位置的计算完全并行。

12.1.2 整体架构

Transformer 由编码器（Encoder）和解码器（Decoder）组成，每部分由多个相同结构的层堆叠而成。



原始论文: $N = 6, d_{\text{model}} = 512, h = 8 \text{ heads}$

图 12.1: Transformer 的编码器-解码器架构。编码器将输入序列编码为连续表示, 解码器根据编码器输出和已生成的输出序列自回归地生成目标序列

12.1.3 多头自注意力——Transformer 的心脏

单头自注意力允许模型在单一表示子空间中关注其他位置。多头注意力通过并行运行多个注意力头, 让模型在不同的表示子空间中同时学习多类依赖关系:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (12.1)$$

$$\text{head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V)$$

其中 $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。论文中 $d_k = d_v = d_{\text{model}}/h = 64$ 。

多头注意力的优势

单头注意力只能学习一种类型的依赖模式 (如语法依存或语义相似性), 多头注意力通过低维投影将 $d_{\text{model}} = 512$ 维空间划分为 $h = 8$ 个 $d_k = 64$ 维子空间, 每个头在独立的低维子空间中学习不同类型的注意力模式, 最后合并所有头学到的信息。总计算量与单头全维度注意力相同。

12.1.4 位置编码——赋予序列顺序信息

注意力机制本身对位置完全不变 (permutation invariant) —— 交换两个输入的位置, 自注意力的输出也相应交换但值不变。为了让模型感知序列顺序, Transformer 在输入嵌入上加位置编码。

正弦位置编码 (原论文):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

其中 pos 是位置， i 是维度索引。这种设计使得：不同维度编码不同频率的正弦波，任何偏移 k 的关系 PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数——这一性质使模型可能学到了相对位置关系。

现代大模型（GPT、Llama）多使用可学习的位置编码或 **RoPE**（旋转位置编码）——后者通过在注意力计算中引入旋转矩阵来编码相对位置信息，具有更优的外推能力。

12.1.5 前馈网络

每个注意力子层之后是一个位置独立的前馈网络（Position-wise FFN）：

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

其中隐藏层维度 d_{ff} 通常为 $4 \times d_{\text{model}} = 2048$ 。前馈网络在每个位置独立应用相同的参数，为模型提供非线性变换能力。

GPT 系列使用 **GELU** 激活函数替代 ReLU；Llama 使用 **SwiGLU**——一种门控变体，将激活函数整合到门控线性单元中：

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1 \odot \text{SiLU}(\mathbf{x}\mathbf{W}_2))\mathbf{W}_3$$

12.1.6 Add & Norm——残差连接与层归一化

每个子层（注意力或前馈网络）之后都有残差连接和层归一化：

$$\text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

原论文使用后归一化（Post-LN），但现代架构（GPT-2 及以后）改用前归一化（Pre-LN）——在子层输入前而非输出后进行归一化。Pre-LN 在训练深层 Transformer 时更稳定，允许更少的学习率预热。

12.1.7 掩码自注意力——解码器中的因果性

在解码器中，自注意力必须是因果的（causal）——位置 i 只能关注位置 1 到 i （不能看到未来的 token）。这通过在 Softmax 前将未来位置的分数设置为 $-\infty$ （掩码）来实现：

$$\text{MaskedAttention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}$$

其中 $\mathbf{M}$ 是上三角矩阵， $\mathbf{M}_{ij} = -\infty$ （当 $j > i$ ），其余为 0。

从头实现 Transformer (PyTorch)

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5 import copy
6
7 class MultiHeadAttention(nn.Module):
8     def __init__(self, d_model, n_heads, dropout=0.1):
9         super().__init__()
10         assert d_model % n_heads == 0
11         self.d_k = d_model // n_heads
12         self.n_heads = n_heads
13
14         self.W_q = nn.Linear(d_model, d_model)
15         self.W_k = nn.Linear(d_model, d_model)
16         self.W_v = nn.Linear(d_model, d_model)
17         self.W_o = nn.Linear(d_model, d_model)
18         self.dropout = nn.Dropout(dropout)
19
20     def forward(self, q, k, v, mask=None):
21         batch_size = q.size(0)
22
23         # 线性投影并分割为多头
24         Q = self.W_q(q).view(batch_size, -1,
25                               self.n_heads, self.d_k).transpose(1, 2)
26         K = self.W_k(k).view(batch_size, -1,
27                               self.n_heads, self.d_k).transpose(1, 2)
28         V = self.W_v(v).view(batch_size, -1,
29                               self.n_heads, self.d_k).transpose(1, 2)
30
31         # 缩放点积注意力
32         scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)
33         if mask is not None:
34             scores = scores.masked_fill(mask == 0, -1e9)
35         attn = F.softmax(scores, dim=-1)
36         attn = self.dropout(attn)
37
38         # 合并多头
39         out = attn @ V
40         out = out.transpose(1, 2).contiguous().view(
41             batch_size, -1, self.n_heads * self.d_k)
42         return self.W_o(out), attn
43
44 class FeedForward(nn.Module):
45     def __init__(self, d_model, d_ff, dropout=0.1):
46         super().__init__()
47         self.linear1 = nn.Linear(d_model, d_ff)
48         self.linear2 = nn.Linear(d_ff, d_model)
49         self.dropout = nn.Dropout(dropout)
50
51     def forward(self, x):
52         return self.linear2(
53             self.dropout(F.gelu(self.linear1(x)))
54         )

```

§ 12.2

BERT——双向编码器表示

BERT (Bidirectional Encoder Representations from Transformers) 仅使用 Transformer 的编码器部分，通过大规模无监督预训练学习深层的双向上下文表示。

12.2.1 预训练任务

BERT 通过两个无监督任务进行预训练：

1. **掩码语言模型 (Masked Language Model, MLM)** ——随机遮盖输入中 15% 的 token (其中 80% 替换为 [MASK], 10% 替换为随机 token, 10% 保持原样)，让模型根据双向上下文预测原始 token。这迫使模型学习深层的语义理解。
2. **下一句预测 (Next Sentence Prediction, NSP)** ——给定两个句子 A 和 B，判断 B 是否是 A 的下一句。这一任务促使模型理解句子之间的关系 (但后续研究表明 NSP 的贡献有限，RoBERTa 去掉了 NSP)。

BERT 的输入表示由三部分相加构成：Token 嵌入 + 段落嵌入 (Segment Embedding, 标识两个句子) + 位置嵌入。

12.2.2 BERT 微调

预训练完成后，BERT 可以通过添加少量任务特定参数在下游任务上微调：

- **文本分类**——使用 [CLS] token 的最后一层隐藏状态输入线性分类器
- **命名实体识别**——每个 token 的最后一层隐藏状态输入线性分类器
- **问答**——输入问题和段落的拼接，预测答案的起始和终止位置
- **句子对分类**——判断两句话的关系 (蕴含、矛盾、中立)

BERT 系列的关键改进：**RoBERTa** 去掉 NSP、使用更大的 batch size 和更多数据；**ALBERT** 通过参数共享和嵌入因式分解大幅减少参数；**DeBERTa** 引入了解耦的注意力机制。

§ 12.3

GPT——生成式预训练 Transformer

GPT 系列采用 Transformer 仅解码器 (decoder-only) 架构，使用自回归语言建模目标进行预训练：给定前 $i-1$ 个 token，预测第 i 个 token。

12.3.1 GPT 的演进

GPT-1 (2018 年)：117M 参数，12 层 Transformer 解码器，首次证明了生成式预训练 + 下游微调范式的有效性。核心假设：语言模型本身可以在无监督数据上学到通用的世界知识和语言能力。

GPT-2 (2019 年)：1.5B 参数，48 层。核心贡献是发现了**零样本迁移**——一个足够大的语言模型即使不经过任何微调也能执行下游任务 (翻译、摘要、问答等)。这颠覆了“预训练 + 微调”的范式。

GPT-3 (2020 年)：175B 参数，96 层。将零样本/少样本学习推向了极致——仅通过提示词 (prompt) 中的少量示例 (in-context learning)，无需梯度更新即可适应新任务。规模足够大时，涌现出了新的能力。

GPT-4 (2023 年): 多模态 (文本 + 图像输入), 在推理能力、事实准确性和指令遵循方面有质的飞跃。具体架构和参数规模未公开。

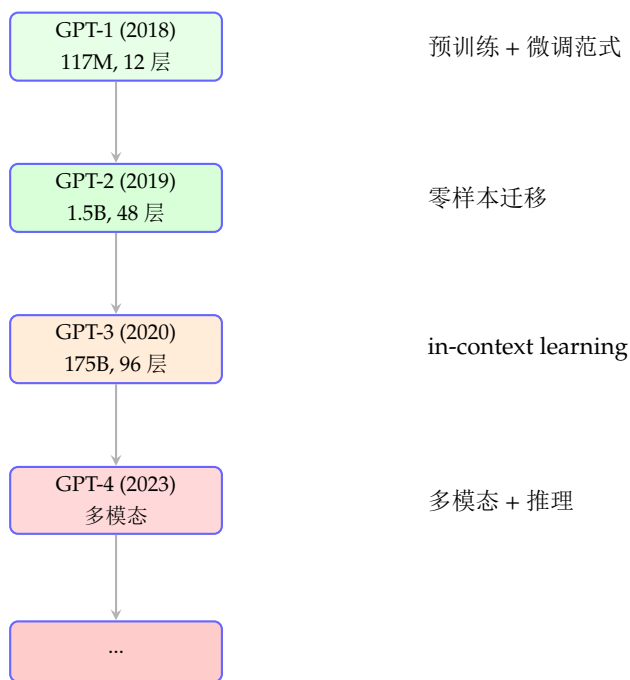


图 12.2: GPT 系列的演进路线。每代模型在规模和能力上都有质的飞跃, 完成了从任务特定微调到通用能力涌现的范式转变

12.3.2 自回归生成与 KV-Cache

GPT 在推理时逐 token 生成文本。为了提高效率, 使用 **KV-Cache**: 将已生成的 token 的 Key 和 Value 矩阵缓存, 生成下一个 token 时只需计算新 token 的 Q、K、V, 然后与缓存的 K、V 做注意力计算。

GPT 风格的因果自注意力（带 KV-Cache）

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class CausalSelfAttention(nn.Module):
7     def __init__(self, d_model, n_heads, max_len=1024):
8         super().__init__()
9         self.n_heads = n_heads
10        self.d_k = d_model // n_heads
11
12        self.q_proj = nn.Linear(d_model, d_model, bias=False)
13        self.k_proj = nn.Linear(d_model, d_model, bias=False)
14        self.v_proj = nn.Linear(d_model, d_model, bias=False)
15        self.out_proj = nn.Linear(d_model, d_model, bias=False)
16
17        # 因果掩码（上三角 = -inf）
18        mask = torch.triu(torch.ones(max_len, max_len),
19                           diagonal=1).bool()
20        self.register_buffer('causal_mask', mask)
21
22    def forward(self, x, kv_cache=None):
23        B, T, C = x.shape
24
25        q = self.q_proj(x).view(B, T, self.n_heads,
26                                 self.d_k).transpose(1, 2)
27        k = self.k_proj(x).view(B, T, self.n_heads,
28                                 self.d_k).transpose(1, 2)
29        v = self.v_proj(x).view(B, T, self.n_heads,
30                                 self.d_k).transpose(1, 2)
31
32        # KV-Cache: 如果存在缓存, 则拼接
33        if kv_cache is not None:
34            k_cache, v_cache = kv_cache
35            k = torch.cat([k_cache, k], dim=2)
36            v = torch.cat([v_cache, v], dim=2)
37
38        # 缩放点积注意力
39        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.d_k)
40
41        # 因果掩码
42        mask = self.causal_mask[:T, :k.size(2)]
43        att = att.masked_fill(mask, float('-inf'))
44        att = F.softmax(att, dim=-1)
45
46        y = att @ v
47        y = y.transpose(1, 2).contiguous().view(B, T, C)
48        y = self.out_proj(y)
49
50        return y, (k, v) # 返回KV缓存
51
52 # 自回归生成示例
53 def generate(model, start_tokens, max_new_tokens, device):
54     model.eval()

```

12.3.3 缩放定律

OpenAI 的研究发现了一个关键的经验规律——**缩放定律**（Scaling Laws）：模型在语言建模任务上的交叉熵损失随着模型参数量、训练数据量和计算量的增加，呈现出可预测的幂律关系。具体来说，损失 L 与计算量 C 的关系近似为：

$$L(C) \approx \left(\frac{C_0}{C}\right)^{\alpha_C} + L_\infty$$

其中 α_C 是缩放指数（经验上约 0.05-0.1）， L_∞ 是不可约损失。这意味着：每将计算量翻倍，损失约减少 5%——要获得显著的性能提升，需要指数级增加的资源。

Chinchilla 缩放定律（2022 年）修正了此前的结论：对于给定的计算预算，模型参数量和训练 token 数应该**同等比例**缩放。此前的大模型（如 GPT-3、Gopher）都严重训练不足——它们有太多参数却训练了太少的 token。

§ 12.4

Llama——开源大模型的前沿

Meta 的 Llama 系列代表了开源大模型的最前沿水平。

12.4.1 Llama 系列的演进

Llama 1（2023 年 2 月）：7B/13B/33B/65B 四个规模。核心创新在于：使用纯公开数据训练，证明了用高质量数据训练的小模型可以在多数基准上超越更大的模型。65B 的 Llama 在 1.4T tokens 上训练，在多个基准上接近或超越了 175B 的 GPT-3。

Llama 2（2023 年 7 月）：7B/13B/70B，使用 2T tokens 训练。关键改进包括：

- **RMSNorm** 前归一化（而非 LayerNorm），计算更高效
- **SwiGLU** 激活函数替代 ReLU，提升 FFN 的表达能力
- **RoPE**（旋转位置编码）替代绝对位置编码，提升上下文长度的外推能力
- **分组查询注意力**（Grouped-Query Attention, GQA）——将查询头分组，每组共享键和值头，减少 KV-Cache 的内存占用
- **40% 更多训练数据**（2T vs 1.4T tokens）

Llama 3（2024 年）：8B/70B/405B。使用超过 15T tokens 训练。在推理、代码、多语言和长上下文方面有显著提升。405B 的 Llama 3 在多数基准上接近或达到了 GPT-4 的水平。

Llama 的技术创新集成

Llama 系列的成功并非单一突破，而是多项精巧设计的有机结合：(1) RMSNorm 提升归一化效率；(2) SwiGLU 增强非线性表达；(3) RoPE 提供优越的位置建模；(4) GQA 平衡速度和性能；(5) 大量高质量训练数据的精心筛选。

12.4.2 RMSNorm——简化的层归一化

RMSNorm 去除了 LayerNorm 中的均值中心化步骤，仅使用均方根（Root Mean Square）进行缩放：

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \odot \gamma$$

相比 LayerNorm, RMSNorm 去掉了均值减去的计算, 速度更快, 且在 Transformer 中表现无差甚至略好。

12.4.3 RoPE——旋转位置编码

RoPE (Rotary Position Embedding) 不通过加法来注入位置信息, 而是通过旋转查询和键向量来实现位置感知:

$$\mathbf{q}_m = \mathbf{R}_m \mathbf{q}, \quad \mathbf{k}_n = \mathbf{R}_n \mathbf{k}$$

其中 $\mathbf{R}_m$ 是一个块对角旋转矩阵, 其第 i 个 2×2 块的旋转角为 $\theta_i = 10000^{-2i/d}$ 。RoPE 的优雅之处在于: 两个位置间的注意力分数自然包含了相对位置信息: $\mathbf{q}_m^\top \mathbf{k}_n = \mathbf{q}^\top \mathbf{R}_m^\top \mathbf{R}_n \mathbf{k} = \mathbf{q}^\top \mathbf{R}_{n-m} \mathbf{k}$, 仅依赖于相对位置 $n - m$ 。

12.4.4 分组查询注意力

标准多头注意力中, 每个注意力头都有自己的 Q、K、V 投影。多查询注意力 (MQA) 让所有头共享 K 和 V, 大幅减少 KV-Cache 但可能降低模型质量。分组查询注意力 (GQA) 是两种方法的折中方案——将注意力头分组, 每组内的头共享相同的 K 和 V。

GQA 在几乎不影响模型质量的情况下, 将 KV-Cache 的内存占用减少了约 $n_{\text{heads}}/n_{\text{groups}}$ 倍, 对长序列推理的效率提升尤为显著。

§ 12.5

DeepSeek——中国大模型的崛起

DeepSeek 系列代表了中国在大语言模型领域的前沿水平, 其技术创新备受国际关注。

12.5.1 DeepSeek-V2 与 Multi-head Latent Attention

DeepSeek-V2 引入了 MLA (Multi-head Latent Attention), 通过将 Key 和 Value 压缩到一个低维的潜在空间来大幅减少 KV-Cache:

$$\mathbf{c}_t^{KV} = \mathbf{W}^{DKV} \mathbf{h}_t, \quad \mathbf{k}_t = \mathbf{W}^{UK} \mathbf{c}_t^{KV}, \quad \mathbf{v}_t = \mathbf{W}^{UV} \mathbf{c}_t^{KV}$$

其中 $\mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}$ 是低维的潜在表示 ($d_c \ll n_{\text{heads}} \times d_k$)。MLA 用低维瓶颈来强制高效的 KV 表示, 比 GQA 更进一步降低了推理时的内存占用。

12.5.2 DeepSeek-V3 与混合专家模型

DeepSeek-V3 采用了 Mixture of Experts (MoE) 架构: 总参数量超 600B, 但每个 token 只激活约 37B 参数 (通过路由选择 top-K 专家)。这使得训练和推理的计算成本大幅降低。

MoE 的核心机制是路由 (routing): 对于每个 token, 一个轻量的路由器 (通常是一个线性层 + Softmax) 计算该 token 应该被发送到哪个或哪几个专家处理。

12.5.3 DeepSeek-R1——推理增强

DeepSeek-R1 通过大规模强化学习 (RL) 来提升模型的推理能力。核心理念: 不需要人工标注的推理链 (Chain-of-Thought) 监督数据, 仅通过 RL 奖励函数 (最终答案是否正确) 来激励模型自发产生更强的推理过程。R1 展示了模型可以通过 RL 自主涌现出验证、回溯、自我纠错等复杂的思维模式。

DeepSeek 的技术路线

DeepSeek 系列的技术演进清晰地展示了三个方向：(1) MLA 减少推理内存；(2) MoE 增加总容量但保持推理效率；(3) RL 增强推理能力。这三条线分别优化了推理成本、知识容量和思考深度。

§ 12.6

Transformer 的变体与优化

12.6.1 FlashAttention——IO 感知的注意力优化

标准注意力需要将整个 $T \times T$ 的注意力矩阵从 GPU 显存（HBM）读写到 SRAM，这成为了显存带宽的瓶颈。**FlashAttention** 通过分块（tiling）和重新计算（recomputation）技术，在 SRAM 中完成小块的计算并立即写回结果，消除了对完整注意力矩阵的显存读写，将注意力机制从显存带宽瓶颈变为计算瓶颈。

FlashAttention 在不改变计算结果的条件下，将标准注意力的显存复杂度从 $O(T^2)$ 降至 $O(T)$ （按比例），并实现了 2-4 倍的速度提升。FlashAttention-2 和 FlashAttention-3 进一步优化了 GPU 线程块调度和非均匀分布的负载。

12.6.2 Mamba 与状态空间模型

近来，**状态空间模型**（State Space Models, SSMs）作为一种 Transformer 的替代架构获得了大量关注。Mamba 应用了**选择性扫描机制**——与传统的 SSM 使用固定的状态转移矩阵不同，Mamba 的转移矩阵依赖于输入（输入感知），使得模型能够根据输入内容的不同选择性地忽略或保留信息。

Mamba 声称在保持训练和推理线性复杂度（ $O(T)$ 而非 Transformer 的 $O(T^2)$ ）的同时，达到了与 Transformer 相当的性能，在长序列建模方面具有天然优势。但其在大规模训练和生态兼容性方面仍在追赶 Transformer。

§ 12.7

指令微调与 RLHF

12.7.1 指令微调

预训练的大模型虽然具备强大的语言能力，但不会遵循人类的指令（它只是一个“续写”模型）。**指令微调**（Instruction Tuning）通过构造多样化的（指令-回答）对来微调模型，使其学会遵循指令。

InstructGPT/GPT-3.5 的关键发现：经过精细的指令微调后，一个 1.3B 的模型在人类评估中可以优于未微调的 175B 模型。

12.7.2 RLHF——基于人类反馈的强化学习

RLHF（Reinforcement Learning from Human Feedback）是将人类偏好编码到模型中的核心框架，包含三个步骤：

1. **监督微调**（SFT）——使用高质量的人类标注数据进行初步微调
2. **奖励模型训练**——收集人类对多个模型回答的偏好比较数据，训练一个奖励模型 $r_\phi(\mathbf{x}, \mathbf{y})$ 来预测人类偏好

3. **PPO 强化学习**——使用 PPO 算法优化 SFT 模型，最大化奖励模型的得分，同时用 KL 散度惩罚约束模型不偏离 SFT 分布太远：

$$\max_{\theta} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}, \mathbf{y} \sim \pi_{\theta}} [r_{\phi}(\mathbf{x}, \mathbf{y}) - \beta \text{KL}(\pi_{\theta}(\mathbf{y}|\mathbf{x}) \| \pi_{\text{ref}}(\mathbf{y}|\mathbf{x}))]$$

DPO (Direct Preference Optimization, 2023 年) 是对 RLHF 的重大简化：它直接在偏好数据上训练语言模型，跳过了显式的奖励模型训练和 PPO 阶段，通过一个巧妙的数学变换将 RLHF 目标转化为二元交叉熵损失。

§ 12.8 本章小结

本章覆盖了从 Transformer 到现代大模型的完整技术脉络：

- **自注意力**以 $O(1)$ 路径长度连接序列中的任意两个位置，是 Transformer 能够高效并行和长程建模的根本原因
- **BERT** 通过掩码语言模型的预训练学习双向语义表示，改写了 NLP 领域的研究范式
- **GPT** 系列用自回归生成 + 规模化证明了“大力出奇迹”的可行性，涌现出 in-context learning 等新能力
- **Llama** 通过 RMSNorm、SwiGLU、RoPE 和 GQA 等技术集成，证明了高质量数据训练的小模型可以超越更大的模型
- **DeepSeek** 的 MLA、MoE 和 RL 增强推理代表了中国大模型技术的前沿探索
- **缩放定律**揭示了模型性能与资源投入之间的可预测关系，指导着大模型的高效训练
- RLHF/DPO 等技术将人类偏好对齐到模型行为中，使大模型从“完成句子”进化为“遵循指令”